



SAP HANA Cloud, SAP HANA Database Property Graph Engine Reference

Generated on: 2026-05-24 12:34:36 GMT+0000

SAP HANA Cloud, SAP HANA Database | QRC 1/2026

Public

Original content: https://help.sap.com/docs/HANA_CLOUD_DATABASE/11afa2e60a5f4192a381df30f94863f9?locale=en-US&state=PRODUCTION&version=2026_1_QRC

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

SAP HANA Cloud Property Graph Engine Reference

This reference provides information about SAP HANA Cloud property graph engine. It is organized as follows:

- **Introduction**

Introduction to SAP HANA Cloud property graph engine.

- **SAP HANA Graph Data Model**

Description of the graph data model using a simple example.

- **Graph Workspaces**

Description of SQL statements for creating and manipulating graph workspaces.

- **Graph Data Modification**

Description of SQL statements for modifying graph data.

- **GraphScript Language**

Description of the GraphScript stored procedure language.

- **openCypher Query Language**

Description of the openCypher query language interface in SAP HANA (Cypher is a registered trademark of Neo4j, Inc.).

- **Additional Information**

A collection of additional information.

Introduction

SAP HANA Cloud property graph engine is an integral part of the SAP HANA core functionality. It expands the SAP HANA platform with native support for graph processing and enables the execution of typical graph operations on the data stored in an SAP HANA system.

SAP HANA Graph Data Model

Graphs are a powerful abstraction that can model different kinds of networks and linked data coming from many industries, such as logistics and transportation, utility networks, knowledge representation, and text processing.

In SAP HANA, a graph is made up by a set of vertices and a set of edges. Vertices are the entities in the graph while the edges describe the relationships between vertices. Each edge connects two vertices - one vertex is denoted as the source and the other as the target. Edges are always directed, yet they can be navigated in either direction. Any number of edges may connect the same two vertices. Vertices and edges can have an arbitrary number of attributes. An attribute consists of a name that is associated with a data type and a value.

Graphs can be defined on column store tables, row store tables, remote tables, views and their synonyms - these graphs are collectively referred to as graphs on tables. Additionally, graphs can be defined on JSON collections.

The whole feature set of SAP HANA can be applied to the graph data: Access control, backup and recovery, and so on. Furthermore, all SAP HANA Cloud property graph engine functions can be applied to the graph data stored in SAP HANA coming from business applications.

SAP HANA Cloud property graph engine provides graph workspaces as dedicated catalog objects for defining graphs.

Graphs on Tables

For graphs on tables, the vertices are stored in vertex tables, while the edges are stored in edge tables. Vertex and edge tables are collectively denoted as graph tables. Vertex attributes correspond to the columns of the vertex tables, and edge attributes correspond to the columns of the edge tables. The maximum number of attributes is bound by the maximum number of columns of the underlying tables. Additionally, each vertex may have an associated label, which represents the role of the vertex in the modeled domain. Edges also may have an associated label which represents the role of the edge in the modeled domain.

Graphs on JSON Collections

For graphs on collections, all the vertices and edges are stored in one collection. JSON documents can be deeply nested and there is no apriori definition of which fields exist or their data type. In general, the data stored in a collection is semi-structured and schema-less. Each JSON document which describes a graph entity (i.e., vertex or edge) must have at least one field specifying the entity type and one field specifying the identifier of the entity in the graph. Furthermore, each JSON document describing an edge must have two additional fields which define the source and the target vertex of the edge. For each graph stored in a collection an [adjacency index](#) is required. The adjacency index specifies the names of the fields in a JSON document defining the graph topology: the entity type, entity identifier, edge source vertex and edge target vertex. If a particular JSON document does not have any of the mentioned fields, then it is not considered to be part of the graph and it is ignored during any further graph processing.

Related Information

[SAP HANA Cloud](#), [SAP HANA Database SQL Reference Guide](#)

Graph Workspaces

A graph workspace is a catalog object that defines a graph. A graph workspace is uniquely identified by the database schema it resides in and the workspace name.

Graph workspace metadata information is provided by the [system views](#) GRAPH_WORKSPACES and GRAPH_WORKSPACE_COLUMNS.

Graph Workspaces on Tables, Views and Synonyms

For graphs on tables, the graph workspace consists of one or more vertex tables and one or more edge tables - the same table can be used to specify both a vertex table and an edge table. Additionally, it is possible to create a graph workspace without a vertex table if exactly one edge table is provided. Vertex attributes with the same name, must have the same data type. Similarly, edge attributes with the same name, must have the same data type.

For each graph table, an optional label can be specified. If no label is specified, the label defaults to the corresponding vertex or edge table name. Vertex labels and edge labels must be unique within a particular graph workspace.

Each vertex is uniquely identified by its associated label and one or more vertex attributes denoted as the vertex key attributes. Analogously, each edge is uniquely identified by its label and one or more edge attributes denoted as the edge key attributes.

All the source vertices defined in a specific edge table must refer to vertices from the same vertex table of the given graph workspace. Similarly, all the target vertices defined in a specific edge table must refer to vertices from the same vertex table of the given graph workspace. Consequently, if a graph workspace contains multiple vertex tables, then it is mandatory to specify the vertex table referenced by the source and target vertices of each edge table. The source and target vertex for each edge is uniquely identified by one or more edge attributes, which are denoted source and target attributes, respectively. The data type and order of the source and target attributes must match the data type and order of the key attributes in the corresponding referenced vertex table.

Graph Workspaces on Collections

The definition of a graph on a collection must specify the TYPE, vertex and edge ID, the edge SOURCE and edge TARGET fields. Each JSON document in the collection with the corresponding TYPE field having the value "vertex" describes a vertex. The key of each vertex is stored in the corresponding ID field. Additional fields of the given JSON document specify additional vertex attributes. No further restrictions apply to the additional vertex attributes: Different vertices may have different attributes or even attributes with the same name but different data types. Similarly, a JSON document with the corresponding TYPE field having the value "edge" describes an edge. The key of each edge is stored in the corresponding ID field while the source and target vertex are identified by the values of the corresponding SOURCE and TARGET fields. All the source and target vertices must reference existing vertices in the collection. As for vertices, no further restrictions apply to the additional edge attributes.

Related Information

[Graph Metadata Views](#)

CREATE GRAPH WORKSPACE Statement (Data Definition)

Creates a graph workspace.

Syntax

```
CREATE [OR REPLACE] GRAPH WORKSPACE <workspace_name>
  { <graph_tables_definition> [<cache_option>]
    | <adjacency_index_reference>
    | <graph_collection_specification> }
```

Syntax Elements

OR REPLACE

The behavior of CREATE OR REPLACE depends on the existence of a previously defined graph workspace. If the graph workspace does not exist, then a new graph workspace is created and the statement behaves like CREATE GRAPH WORKSPACE. If the workspace already exists, then the workspace is altered according to the new definition - in this case, the owner of the workspace as well as all the privileges associated with the graph workspace are preserved.

<workspace_name>

Specifies the name of the graph workspace.

```
<workspace_name> ::= [<schema_name>.]<identifier>
```

<graph_tables_definition>

Specifies the vertex and edge tables. Multiple vertex and edge tables can be specified in any order. A valid graph workspace must contain at least one edge table. Currently, each vertex table must be unique within the graph workspace.

For graph workspaces without vertex tables, see the subsection [Workspaces without vertex table](#) below.

```
<graph_tables_definition> ::= { <vertex_table_definition> | <edge_table_definition> } ...
```

<vertex_table_definition>

Specifies a vertex table.

```
<vertex_table_definition> = VERTEX TABLE <table_name> <vertex_key_label_specifications>
```

<vertex_key_label_specifications>

Specifies the key and label of the vertex table. Both single column and multi-column keys are supported. The specification of the key is mandatory while the label specification is optional. For each vertex table, exactly one key must be specified and at most one label can be explicitly defined. The two specifications can be made in any order.

```
<vertex_key_label_specifications> = { <key_columns_specification> | <label_specification> } ...
```

<edge_table_definition>

Specifies an edge table.

```
<edge_table_definition> = EDGE TABLE <table_name> <edge_key_source_target_label_specifications>
```

<edge_key_source_target_label_specifications>

Specifies the mandatory edge key, source vertex key and target vertex key of the edge table. Both single column and multi-column keys are supported. Additionally, an optional label can be specified. For each edge table, exactly one edge key, one source vertex key and one target vertex key must be specified and at most one label can be explicitly defined. The specifications can be made in any order.

```
<edge_key_source_target_label_specifications> =  
    { <key_columns_specification> | <source_columns_specification> | <target_columns_specification>
```

<table_name>

Specifies the name of a graph table (for example, vertex or edge table). A graph table can be either a table, a view or a synonym.

```
<table_name> ::= [<schema_name>.]<identifier>
```

The graph workspace and the graph tables can reside in different schemas. If any of the schemas are omitted, then the default schema is assumed.

<key_columns_specification>

Specifies the columns which uniquely identify a vertex in a vertex table or an edge in an edge table. These columns are also denoted as vertex and edge key attributes, respectively.

```
<key_columns_specification> ::= KEY [COLUMN] <column_name_list>  
<column_name_list> ::= <column_name> [ { <comma> <column_name> } ... ]  
<column_name> ::= <identifier>
```

The vertex and edge key columns can have one of the following SQL types: TINYINT, SMALLINT, INTEGER, BIGINT, NVARCHAR, or VARBINARY. For performance reasons, integer types are recommended. Each key column of a graph table backed by a database table must have the NOT NULL constraint and additionally, the key columns must also have the UNIQUE constraint.

<label_specification>

Specifies the label of the corresponding graph table. The label specification is optional and if missing, then the label defaults to the corresponding vertex or edge table name.

```
<label_specification> ::= LABEL <string_literal>
```

The label can be used later in both [GraphScript](#) and [openCypher](#).

<source_columns_specification>

Specifies the columns which uniquely identify the source vertices of the edges in the corresponding edge table.

```
<source_columns_specification> ::= SOURCE [COLUMN] <column_name_list> [<vertex_table_reference>]
```

<target_columns_specification>

Specifies the columns which uniquely identify the target vertices of the edges in the corresponding edge table.

```
<target_columns_specification> ::= TARGET [COLUMN] <column_name_list> [<vertex_table_reference>]
```

If the edge table is backed by a database table, then the source and target columns must have the NOT NULL constraint.

<vertex_table_reference>

Specifies the referenced vertex table. By default, the key columns of the specified vertex table are referenced.

```
<vertex_table_reference> ::= REFERENCES <table_name>
```

<cache_option>

Specifies the caching strategy. For each graph workspace on tables used in [GraphScript](#) a secondary index structure is built in order to speed-up GraphScript program execution. Depending on the selected caching option and underlying data store, this index can be kept in-memory and re-used in subsequent GraphScript program invocations.

```
<cache_option> ::= WITH DYNAMIC CACHE | NO CACHE | WITH STATIC CACHE RETENTION <unsigned_integer>
```

For the default option WITH DYNAMIC CACHE, the cache is managed automatically. The graph always reflects the current transactional view of the underlying source objects. If the graph is completely based on column store sources, the system tries to avoid full index updates and rather handles source updates incrementally on access, so in most cases dynamic caching is preferable.

NO CACHE disables the caching for this specific graph workspace, which may be appropriate if a graph is accessed only once or memory resources are scarce.

If the WITH STATIC CACHE RETENTION option is selected, it is mandatory to specify the time in minutes until the cache entry is invalidated. Then after initial access the system is allowed to skip updates of internal graph data structures irrespective of changes of the underlying data. If a graph is accessed after its retention period has expired, then its internal index structures are updated according to all intermediate source data updates. This update may initiate a renewed stable retention period.

<adjacency_index_reference>

Specifies the name of the referenced adjacency index for graph workspaces on collections.

```
<adjacency_index_reference> ::= ADJACENCY INDEX [<schema_name>.]<identifier>
```

<graph_collection_specification>

Specifies the name of the collection and the fields defining the graph topology.

```
<graph_collection_specification> ::= COLLECTION [<schema_name>.]<identifier> <graph_collection_fields_specification>
```

<graph_collection_fields_specification>

Specifies the names of the fields defining the graph topology - the fields are described in the section [Graph Workspaces on Collections](#). Three different sections can be distinguished according to the defined fields: The type section defines the TYPE field, the vertex section defines the vertex ID (i.e., an identifier) field, while the edge section defines the edge ID, edge SOURCE and edge TARGET fields. All the sections are mandatory and can be specified in any order. Furthermore, the definition of the ID, SOURCE and TARGET fields in the edge section is mandatory and the fields can be specified in arbitrary order. Currently, the vertex ID and edge ID fields must be identical.

```
<graph_collection_fields_specification> =
  { <type_field_specification> | <vertex_fields_specification> | <edge_fields_specification> } ..

<type_field_specification> = TYPE <field>
<vertex_fields_specification> = VERTEX <key_field>
<key_field> = KEY <field>
<edge_fields_specification> = EDGE {<key_field> | <source_field> | <target_field>} ...
<source_field> = SOURCE <field>
<target_field> = TARGET <field>
```

Workspaces without vertex table

It is possible to create a graph workspace without a vertex table. This is possible if exactly one edge table is provided.

In case the vertex table is omitted, the system will implicitly create vertex data based on the contents of the edge table. The vertex data will be the union of entities referenced in the SOURCE fields and entities referenced in the TARGET fields of the edge table.

The implicit vertices use the key attribute names KEY_1, . . . , KEY_N. The respective data types correspond to the datatypes of the source/target fields in the edge table. The vertices do not have any other additional attributes.

Description

During the creation of a new graph workspace, the SAP HANA system checks whether the creator of the graph workspace has the CREATE ANY privilege on the intended graph workspace schema. For CREATE OR REPLACE, the user must be additionally authorized to alter the graph workspace if it already exists. For that, the user must have the ALTER privilege on the workspace schema or the workspace unless the user is the owner of the workspace being replaced.

Graph Workspaces on Tables, Views and Synonyms

For graph workspaces on tables, views and synonyms, the following additional requirements are checked:

- The creator of the graph workspace is authorized to select from all the graph tables.
- At least one vertex and one edge table or exactly one edge table (i.e., workspace without vertex table) has been specified.
- Each vertex table is unique within the graph workspace.
- All the graph tables and columns exist and have supported data types.
- For graph tables backed by database tables, the above mentioned NOT NULL and UNIQUE constraints on the key, source and target columns are defined.
- All the vertex and respectively edge attributes with the same name have the same data type.
- If multiple vertex tables are defined, for each edge table the vertex table referenced by the source and respectively target vertices is specified.
- The explicitly or implicitly defined vertex and respectively edge labels are unique within the graph workspace.

Graph Workspaces on Collections

There are two options to create graph workspaces on collections:

1. The first option is to explicitly create the required hash and adjacency collection indexes using the `CREATE HASH INDEX` and `CREATE ADJACENCY INDEX` statements, respectively. Afterwards, the graph workspace can be created by simply referencing the adjacency index in the `CREATE GRAPH WORKSPACE` statement. The creator of the graph workspace must be authorized to select from the collection on which the adjacency index is defined.
2. The second option entails specifying the collection and the fields defining the graph topology in the `CREATE GRAPH WORKSPACE` statement. In this case, the SAP HANA system will first check whether the creator of the graph workspace is authorized to select from and create indexes on the specified collection. If the user is not the owner of the collection, then the user is authorized to create collection indexes if it has one of the following privileges: `CREATE ANY` or `INDEX` on the collection schema or `INDEX` on the collection.

Currently, an additional restriction applies to the vertex and edge ID fields, which must be identical. If the user has the required privileges, and the vertex and edge ID fields are identical, the system will check whether matching hash and adjacency indexes already exist. To perform this check, the vertex and edge ID, the edge SOURCE, edge TARGET and TYPE fields specified in the `CREATE GRAPH WORKSPACE` statement are mapped to the ID, SOURCE, TARGET and TYPE fields of the adjacency index, respectively. The system will try to automatically create any missing indexes before creating the graph workspace - if the matching hash or adjacency indexes already exist, then they are used and no further indexes are created. The names of the system generated hash and adjacency indexes have the prefix ***SYS_HASH_INDEX*** and ***SYS_ADJACENCY_INDEX***, respectively.

The system creates the missing indexes in the schema of the collection. The owner of the collection is also the owner of the indexes regardless of the user who created the graph workspace and the system indexes with it. If multiple graph workspaces are defined on the same fields of the same collection, then missing collection indexes are created only once during the creation of the first graph workspace and they are used by all other graph workspaces. The life cycle of the system generated collection indexes is managed by the system. The system generated indexes are automatically dropped when the last referencing graph workspace is dropped.

The life cycle of the explicitly created collection indexes is not affected in any way by the graph workspaces referencing them. However, dropping an adjacency index invalidates any referencing graph workspace. An invalidated graph workspace due to a dropped adjacency index cannot be reattached to a different adjacency index and it remains invalid until dropped.

Graph workspaces on collections can be used in [GraphScript](#) while they are currently not supported by [OPENCYPHER TABLE](#)!

A successfully created graph workspace remains valid as long as the above mentioned conditions are fulfilled.

A valid graph workspace is consistent if every vertex and respectively edge is unique and all the edge source and target vertices reference existing vertices within the graph workspace. For graph workspaces on database tables, properly defined referential constraints between the tables which define the edge and vertex tables can guarantee the consistency of the graph workspace.

Examples

Create graph workspace on tables

Create the graph workspace "COMPANY_GRAPH". For the complete list of SQL statements used to create the underlying tables of the "COMPANY_GRAPH" workspace please see [Appendix A](#).

```
CREATE GRAPH WORKSPACE COMPANY_GRAPH
  VERTEX TABLE EMPLOYEES
    KEY EID
    LABEL 'Employee'
  VERTEX TABLE PROJECTS
    LABEL 'Project'
    KEY PID
  EDGE TABLE PROJECT_ASSIGNMENTS
    SOURCE EMPLOYEE_ID REFERENCES EMPLOYEES
    TARGET PROJECT_ID REFERENCES PROJECTS
    KEY EMPLOYEE_ID, PROJECT_ID
    LABEL 'Assignments'
  EDGE TABLE PROJECT_OUTCOMES
    KEY PROJECT_ID, PRODUCT_ID
    SOURCE PROJECT_ID REFERENCES PROJECTS
    TARGET PRODUCT_ID REFERENCES PRODUCTS
  VERTEX TABLE PRODUCTS
    KEY PID
    LABEL 'Product'
  VERTEX TABLE CUSTOMERS
    KEY CID
  EDGE TABLE SALES
    KEY PRODUCT_ID, CUSTOMER
    LABEL 'Sales'
    SOURCE PRODUCT_ID REFERENCES PRODUCTS
    TARGET CUSTOMER REFERENCES CUSTOMERS;
```

The "COMPANY_GRAPH" contains multiple vertex tables. Therefore, for each edge table the referenced source and target vertex table must be specified. The label is a case-sensitive SQL string: For the edges contained in the "SALES" table, the label is 'Sales' whereas the vertices defined through the "CUSTOMERS" table have the default label 'CUSTOMERS'.

For the Open Flights graph workspace example consisting of one vertex table and one edge table, see [Appendix A](#).

Create graph workspace on a collection

In order to create a graph workspace on a collection, specify the name of the collection and the fields defining the graph topology. Missing hash and adjacency indexes are automatically created by the system before creating the graph workspace.

```
CREATE COLLECTION A_SCHEMA.EMPLOYEES;
CREATE GRAPH WORKSPACE GRAPH_SCHEMA.EMPLOYEES_GRAPH COLLECTION A_SCHEMA.EMPLOYEES
  TYPE "type"
  VERTEX
    KEY "id"
  EDGE
    KEY "id"
```

```
SOURCE "src"  
TARGET "tgt";
```

Create graph workspace on a collection by referencing an explicitly created adjacency index

First, a hash index must be defined on the identifier field of the graph entities in the collection. Second, the adjacency index must be defined on the collection containing the graph data. Afterwards, the graph workspace can be created by simply referencing the adjacency index in the create graph workspace statement.

```
CREATE COLLECTION A_SCHEMA.EMPLOYEES;  
CREATE HASH INDEX EMPLOYEES_HASH_INDEX ON A_SCHEMA.EMPLOYEES("id");  
CREATE ADJACENCY INDEX EMPLOYEES_ADJ_INDEX ON A_SCHEMA.EMPLOYEES(ID "id", SOURCE "src", TARGET "tgt");  
CREATE GRAPH WORKSPACE GRAPH_SCHEMA.EMPLOYEES_GRAPH ADJACENCY INDEX A_SCHEMA.EMPLOYEES_ADJ_INDEX;
```

For the complete list of SQL statements used to create the underlying collection, the hash index and the adjacency index please see [Appendix A](#).

CREATE ADJACENCY INDEX Statement (Data Definition)

Creates an adjacency index on a JSON collection table.

Syntax

```
CREATE ADJACENCY INDEX <index_name> ON <collection_name> (<identifier_field_list>)
```

Syntax Elements

<index_name>

Specifies the name of the adjacency index with an optional schema name. An index can only be created in the schema of the collection on which it is defined.

```
<index_name> ::= [<schema_name>.<identifier>]
<schema_name> ::= <identifier>
```

<collection_name>

Specifies the name of the JSON collection table.

```
<collection_name> ::= [[<database_name>.<schema_name>.<identifier>]
<database_name> ::= <identifier>
```

<identifier_field_list>

Specifies the identifiers of the ID, SOURCE, TARGET and TYPE fields.

```
<identifier_field_list> ::= ID <field>, SOURCE <field>, TARGET <field>, TYPE <field>
<field> ::= <identifier> [{ <period> <identifier> } [...]]
```

Description

The adjacency index specifies the names of the TYPE, ID, SOURCE and TARGET fields defining a graph stored in the JSON collection. Each JSON document which describes a graph entity (i.e., vertex or edge) must have at least one field specifying the entity type (i.e., the TYPE field) and one field specifying the identifier (i.e., the ID field) of the entity in the graph.

- The entity type field must have the data type string and it must have one of the two allowed values: "vertex" or "edge".
- The identifier field must be a positive BIGINT value.
- The uniqueness of the ID field is not enforced by the database, but for most use cases it is strongly recommended to use unique identifiers. Also ensure that edge and vertex documents shall have different IDs.

- Furthermore, each JSON document describing an edge must have two additional fields of type BIGINT which define the source and the target vertex of the edge.

If the specified fields are present in a document, then the document must adhere to all the specified rules, else the creation of the adjacency index fails. Once the adjacency index has been created, this condition is checked for all the data manipulation operations on the given collection. These consistency checks ensure that graph queries yield the same results as similar SQL queries on the collection.

In order to define an adjacency index a corresponding hash index on the ID field must exist. Currently, only one adjacency index can be defined on any given collection. The adjacency index is persisted.

Example

Below is an example:

```
CREATE COLLECTION GRAPH_COLLECTION;
INSERT INTO GRAPH_COLLECTION values('{"id": 0, "type": "vertex", "name": "John"}');
INSERT INTO GRAPH_COLLECTION values('{"id": 1, "type": "vertex", "name": "Paul"}');
INSERT INTO GRAPH_COLLECTION values('{"id": 2, "type": "vertex", "name": "Mary"}');
INSERT INTO GRAPH_COLLECTION values('{"id": 3, "type": "vertex", "name": "Sara"}');
INSERT INTO GRAPH_COLLECTION values('{"id": 4, "type": "edge", "source": 0, "target": 1, "relation": "knows"}');
INSERT INTO GRAPH_COLLECTION values('{"id": 5, "type": "edge", "source": 0, "target": 2, "relation": "knows"}');
INSERT INTO GRAPH_COLLECTION values('{"id": 6, "type": "edge", "source": 2, "target": 3, "relation": "knows"}');
INSERT INTO GRAPH_COLLECTION values('{"id": 7, "type": "edge", "source": 1, "target": 3, "relation": "knows"}');

CREATE HASH INDEX HASH_INDEX ON GRAPH_COLLECTION("id");
CREATE ADJACENCY INDEX ADJACENCY_INDEX ON GRAPH_COLLECTION (ID "id", SOURCE "source", TARGET "target");
```

First, the collection "GRAPH_COLLECTION" is created. Second, the hash index is defined on the field which will be used as the identifier for the vertices and edges. Finally, the adjacency index is created.

Related Information

[INDEXES System View](#)

[INDEX_COLUMNS System View](#)

DROP GRAPH WORKSPACE Statement (Data Definition)

Drops a graph workspace.

Syntax

```
DROP GRAPH WORKSPACE <workspace_name> [CASCADE]
```

Syntax Elements

<workspace_name>

Specifies the name of the graph workspace.

```
<workspace_name> ::= [<schema_name>.]<identifier>
```

CASCADE

Drops the graph workspace and dependent objects (for example, GraphScript procedures and table functions which reference the graph workspace).

Description

In order to drop a graph workspace, the user must be either the owner of the graph workspace or have the DROP privilege for the given graph workspace.

Graph Workspaces on Tables, Views and Synonyms

Dropping a graph workspace defined on tables, views or synonyms does not affect the underlying graph tables.

Graph Workspaces on Collections

Depending on how the graph workspace was created, there are two cases to consider:

- If an explicitly created adjacency index was referenced in the statement used to create the graph workspace, then dropping the graph workspace does not affect the underlying collection nor the referenced adjacency index.
- If the collection and the fields defining the graph topology were specified in the CREATE GRAPH WORKSPACE statement, then the user dropping the graph workspace also needs to have the privilege to drop collection indexes unless the user is the owner of the graph workspace. System generated collection indexes referenced by the graph workspace will be dropped as well while dropping the graph workspace, if there are no other graph workspaces left referencing the indexes.

For more information, see [CREATE GRAPH WORKSPACE statement](#).

Example

Drop the graph workspace "COMPANY_GRAPH".

```
DROP GRAPH WORKSPACE COMPANY_GRAPH;
```

For the complete list of SQL statements used for creating the "COMPANY_GRAPH" workspace please see [Appendix A - Open Flights, Company and Employees Graph Examples](#).

Export and Import Graph Workspaces

Graph workspace objects can be exported from or imported into an SAP HANA Cloud system using the frontend tools in the same way as for exporting/importing views or tables.

Graph Metadata Views

GRAPH_WORKSPACES System View

This system view lists all existing graph workspaces.

Structure

Column name	Data type	Description
SCHEMA_NAME	NVARCHAR(256)	The name of the schema that the workspace resides in.
WORKSPACE_NAME	NVARCHAR(256)	The name of the workspace.
CREATE_TIMESTAMP	TIMESTAMP	The workspace creation timestamp.
USER_NAME	NVARCHAR(256)	The username of the workspace creator.
IS_VALID	NVARCHAR(5)	Specifies whether this workspace is valid (TRUE or FALSE).

GRAPH_WORKSPACE_COLUMNS System View

This system view provides information on the constituent entities of graph workspaces as well as on the available vertex and edge attributes.

Besides the constituent key, source, and target attributes, the view also shows the other vertex and edge attributes that can be used within GraphScript or queries, but only if the referenced graph table already exists. The key, source, and target attributes are always shown, even if the graph tables do not exist, because this information is directly based on the workspace definition. Additionally, for graph workspaces on collections, the type attribute is also shown.

Structure

Column name	Data type	Description
SCHEMA_NAME	NVARCHAR(256)	The workspace schema name.
WORKSPACE_NAME	NVARCHAR(256)	The workspace name.
ENTITY_TYPE	NVARCHAR(6)	The entity type (VERTEX or EDGE).
ENTITY_SCHEMA_NAME	NVARCHAR(256)	The entity object schema name.
ENTITY_TABLE_NAME	NVARCHAR(256)	The entity object table or view name.
ENTITY_COLUMN_NAME	NVARCHAR(256)	The entity object column name.
ENTITY_ROLE	NVARCHAR(6)	The role of column (KEY, SOURCE, TARGET, or TYPE. NULL for additional vertex or edge properties).

Column name	Data type	Description
ENTITY_ROLE_POSITION	INTEGER	The position of the column in a composite key, source, or target (otherwise NULL).
ENTITY_LABEL	NVARCHAR(5000)	The explicitly defined label of the vertex or edge table the column belongs to (otherwise NULL).
REFERENCED_SCHEMA_NAME	NVARCHAR(256)	For the edge source and target columns, the schema name of the referenced vertex table (NULL for all other columns).
REFERENCED_TABLE_NAME	NVARCHAR(256)	For the edge source and target columns, the table name of the referenced vertex table (NULL for all other columns).

Global Cache Configuration

Depending on the cache specification in the `CREATE GRAPH WORKSPACE` statement, created graphs on tables can be temporarily kept in-memory for optimizing system responsiveness or for having temporarily stable results. This section describes how to inspect the cache and globally modify its behavior.

It is recommended that you do not change the default values, particularly on production systems, as any changes may significantly affect overall system performance. On SAP HANA Cloud systems, the configuration parameters below cannot be changed via SQL commands. Yet, if there is a need to do so, the graph cache maximum size can be changed using the SAP HANA Cockpit. For changing the global cache reclaim interval, you may create a customer case support ticket for your SAP HANA Cloud system containing the respective parameter and the desired value.

The upper limit of memory that may be used by graph caching is defined as follows. Both a relative percentage of the system memory and an absolute size in bytes are valid:

```
ALTER SYSTEM ALTER CONFIGURATION ('indexserver.ini', 'SYSTEM')
  SET ('graphengine', 'graph_cache_max_size') = '<amount_in_bytes|fraction_in_percent%>' WITH RECONFIGURE;
```

The cache is checked periodically for an excess of this limit. The least recently used entries are removed if they exceed the limit. By default the interval for checking this limit is one hour and can be changed as follows:

```
ALTER SYSTEM ALTER CONFIGURATION ('indexserver.ini', 'SYSTEM')
  SET ('cache', 'reclaim_interval') = '<amount_in_seconds>' WITH RECONFIGURE;
```

The 10 latest graph cache entries can be inspected as follows:

```
SELECT TOP 10 * FROM m_cache_entries WHERE cache_id = 'GraphCache'
ORDER BY create_time DESC;
```

All current graph cache entries may be immediately evicted as follows:

```
ALTER SYSTEM CLEAR CACHE ('GraphCache');
```

Related Information

[SAP Note 2502256 - FAQ: SAP HANA Caches](#)

[SAP Note 1999997 - FAQ: SAP HANA Memory](#)

RDF_GRAPH_FROM_GRAPH_WORKSPACE Built-In Procedure

The `RDF_GRAPH_FROM_GRAPH_WORKSPACE` built-in procedure maps a property graph workspace to an RDF graph representation. It creates a deep copy and both graph data representation models are independent. As a result, updates made to one graph model don't automatically reflect in the other.

Graph Models

Vertex Mapping

Vertices in a property graph are converted to RDF triples in the form subject–predicate–object:

- The vertex key is used to construct the subject URI.
- The attribute name is used to construct the predicate URI.
- The attribute value becomes the object of the triple.

Each vertex may therefore produce multiple RDF triples, one for each attribute.

Edge Mapping

Edges in the property graph are mapped using RDF* notation to capture relationships between vertices and their attributes. Each edge is represented as an embedded triple and associated attribute triples, as follows:

- The embedded subject corresponds to the source vertex, represented by the subject URI of that vertex.
- The embedded predicate corresponds to the edge label.
- The embedded object corresponds to the target vertex, represented by the subject URI of that vertex.
- Each edge attribute is mapped to a predicate–object pair, where the predicate is the attribute name and the object is the attribute value.

This mapping enables the preservation of both structural and attribute information from the property graph within an RDF* graph representation.

The following prefixes are allowed in the in-mapping table as listed here: <https://www.w3.org/wiki/TheUsualPrefixes>

Prefix	URI
rdf	http://www.w3.org/1999/02/22-rdf-syntax-ns#
dc	http://purl.org/dc/elements/1.1/
foaf	http://xmlns.com/foaf/0.1/
rdfs	http://www.w3.org/2000/01/rdf-schema#
xsd	http://www.w3.org/2001/XMLSchema#

Prefix	URI
owl	http://www.w3.org/2002/07/owl#
vann	http://purl.org/vocab/vann/
cc	http://web.resource.org/cc/
vs	http://www.w3.org/2003/06/sw-vocab-status/ns#
wot	http://xmlns.com/wot/0.1/
geo	http://www.w3.org/2003/01/geo/wgs84_pos#
inet	http://www.w3.org/2001/02pd/rfc65#
dca	http://dublincore.org/2000/03/13-dcagent#
uriReg	http://www.w3.org/Addressing/schemes#

To keep things simple, the examples in the documentation use the prefixes 'ex' and 'dbo' to represent the namespaces <http://example.org/> and <http://dbpedia.org/ontology/>, respectively. The documentation uses the terms vertex table and edge table. Unless otherwise specified, the same conventions apply for graph workspaces based on views, collections, or synonyms.

Syntax

The procedure accepts several parameters, some of which are optional.

```
RDF_GRAPH_FROM_GRAPH_WORKSPACE (  
  { GRAPH_WORKSPACE_SCHEMA_NAME => '<schema_name>'  
    , GRAPH_WORKSPACE_NAME => '<workspace_name>'  
    , TARGET_GRAPH_URI => '<target_graph_uri>'  
    , REPLACE => <true/false>  
    , MAPPING_IN => <table_name>  
    , MAPPING_MODE => '<mapping_mode>'  
    , MAPPING_OUT => ?  
  }  
)
```

Syntax Elements

Parameter	Data Type	Required/Optional	IN/OUT	Default Value	Description
GRAPH_WORKSPACE_SCHEMA_NAME	NVARCHAR(100)	Required	IN		Schema name of the source graph workspace.
GRAPH_WORKSPACE_NAME	NVARCHAR(100)	Required	IN		Object name of the source graph workspace.
TARGET_GRAPH_URI	NVARCHAR(100)	Optional	IN	Default Graph	Target RDF graph to insert the triples.
REPLACE	Boolean	Optional	IN	False	Replaces the existing triples in the provided TARGET_GRAPH_URI. If no TARGET_GRAPH_URI is

Parameter	Data Type	Required/Optional	IN/OUT	Default Value	Description
					provided, then the triples are replaced in the default graph.
MAPPING_IN	SQL Identifier	Optional	IN		Table to specify the mapping of vertex and edge attributes to RDF URIs.
MAPPING_MODE	NVARCHAR(100)	Optional	IN	AUTO	Specifies the mapping mode: AUTO, MAPPING_ONLY, IGNORE or STRICT.
MAPPING_OUT		Required	OUT		Generates a table with mapped value of each attribute.

In-Mapping Table

You can specify a mapping table using the **MAPPING_IN** parameter. This defines how the vertex and edge tables, along with their attributes, map to RDF URIs. This table lets you assign meaningful and consistent URIs to attributes instead of relying on system-generated identifiers.

Structure

Parameter	Data Type	Required/Optional	Description
GRAPH_WORKSPACE_SCHEMA_NAME	NVARCHAR(100)	Required	Schema name of the vertex/edge table.
GRAPH_WORKSPACE_TABLE_NAME	NVARCHAR(100)	Required	Object name of the vertex/edge table.
GRAPH_WORKSPACE_FIELD_NAME	NVARCHAR(100)	Required	Attribute name of the vertex/edge.
GRAPH_WORKSPACE_FIELD_TYPE	NVARCHAR(100)	Optional	Data type of the vertex/edge attribute.
RDF_PROPERTY_TYPE	NVARCHAR(100)	Required	Assigns an RDF property type to the attribute value, like data type.
RDF_LANGUAGE_TAG	NVARCHAR(100)	Optional	Assigns a RDF language tag, like @de, @it.
RDF_URI	NVARCHAR(100)	Required	The URI the attribute should be mapped to.

Vertex or Edge Table

A vertex or edge table can be specified in the mapping table by providing the following details:

- **GRAPH_WORKSPACE_SCHEMA_NAME:** The schema name in which the vertex or edge table resides.
- **GRAPH_WORKSPACE_TABLE_NAME:** The vertex or edge table name.
- **GRAPH_WORKSPACE_FIELD_NAME:**
- **GRAPH_WORKSPACE_FIELD_TYPE:** The graph field type, only relevant for collections.

- **RDF_PROPERTY_TYPE:** The RDF property.
- **RDF_LANGUAGE_TAG:**
- **RDF_URI:** The URI to map the tables.

The URI associated with this entry is mapped to the `rdf:type` property of the vertices if nothing is specified in the field `RDF_PROPERTY_TYPE`. If additional properties need to be assigned to the vertices, they can be specified using the `RDF_PROPERTY_TYPE` field. For example, `rdfs:subPropertyOf`. The mapping table also allows multiple properties to be assigned to a vertex table. For edges, the `rdf:type` property becomes the label of the edge, which is the predicate of the embedded triple in RDF* notation. It is also possible to assign multiple labels to edges.

Since a collection-based graph workspace is defined on a single collection, it is necessary to specify either 'VERTEX' or 'EDGE' in the `GRAPH_WORKSPACE_FIELD_TYPE` field to differentiate between the vertex and edge properties.

Attributes

The attributes of the vertex and edge table can be specified in the mapping table by providing the following details:

- **GRAPH_WORKSPACE_SCHEMA_NAME:** The schema name in which the vertex or edge table resides.
- **GRAPH_WORKSPACE_TABLE_NAME:** The vertex or edge table name.
- **GRAPH_WORKSPACE_FIELD_NAME:** The attribute name.
- **GRAPH_WORKSPACE_FIELD_TYPE:**
- **RDF_PROPERTY_TYPE:** The data type.
- **RDF_LANGUAGE_TAG:** The language tag.
- **RDF_URI:** The URI to map the attributes.

You can specify the data type of an attribute using `RDF_PROPERTY_TYPE`, which may include an XML Schema data type or `rdf:langString`. If you specify `rdf:langString` or leave `RDF_PROPERTY_TYPE` empty, you can specify a language tag in `RDF_LANGUAGE_TAG`. You may provide the language tag with or without the "@" prefix. For example, both "@de" and "de" represent German.

Example

Consider a vertex table "VERTEX_TABLE" in the schema "EXAMPLE_SCHEMA" with key attribute "ID", "HEIGHT" and "WEIGHT".

ID	HEIGHT	WEIGHT
'John'	185.2	75
'Sam'	177.8	83

Similarly, an edge table "EDGE_TABLE" with key attribute "ID", source column "SOURCE", target column "TARGET" and an attribute "CREATION_DATE"

ID	SOURCE	TARGET	CREATION_DATE
1	'John'	'Sam'	'2010-01-01 12:00:00.0000000'

These two tables create a graph workspace "GRAPH". Furthermore, we create the following mapping table "MAPPING_TABLE" to map tables and attributes to RDF URIs. It is possible to assign a different XML Schema data type to the attributes than what the procedure would assign based on the SQL type. It is the responsibility of the user to assign a compatible data type.

GRAPH_WORKSPACE_SCHEMA_NAME	GRAPH_WORKSPACE_NAME	GRAPH_WORKSPACE_FIELD_NAME	GRAPH_WORKSPACE_
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'		
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'	'ID'	
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'	'HEIGHT'	
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'	'WEIGHT'	
'EXAMPLE_SCHEMA'	'EDGE_TABLE'		
'EXAMPLE_SCHEMA'	'EDGE_TABLE'	'ID'	

These mappings are ignored if the mapping table includes the edge table's source or target attribute names. This happens because the source and target attributes refer to the subject URIs of their corresponding vertices.

Use the following command to execute the procedure by specifying the following details:

- The schema name of the graph workspace.
- The graph workspace name.
- The URI of the target RDF graph where the triples will be inserted.
- The in-mapping table.
- An out parameter, which serves as a placeholder for the resulting mapping. The out parameter allows the user to inspect the URIs assigned to each attribute, providing visibility into the mapping process.

```
CALL RDF_GRAPH_FROM_GRAPH_WORKSPACE
(
  GRAPH_WORKSPACE_SCHEMA_NAME => 'EXAMPLE_SCHEMA',
  GRAPH_WORKSPACE_NAME => 'GRAPH',
  TARGET_GRAPH_URI => '<http://example.org/graph>',
  MAPPING_IN => "EXAMPLE_SCHEMA"."MAPPING_TABLE",
  MAPPING_OUT => ?
)
```

The procedure generates the following triples from the vertex table:

```
foaf:firstName/John rdf:type ex:employees .
foaf:firstName/John dbo:weight "75"^^xsd:int .
foaf:firstName/John dbo:height "185.2"^^xsd:double .
foaf:firstName/Sam rdf:type ex:employees .
foaf:firstName/Sam dbo:weight "83"^^xsd:int .
foaf:firstName/Sam dbo:height "177.8"^^xsd:double .
```

It also generates the following triples in RDF* notation from the edge table:

```
< foaf:firstName/John <http://schema.org/isManager> foaf:firstName/Sam > foaf:key "1"^^xsd:int .
< foaf:firstName/John <http://schema.org/isManager> foaf:firstName/Sam > <http://EXAMPLE_SCHEMA/EDC
```

Since no mapping mode was specified, the procedure defaults to the AUTO mapping mode. In this mode, the edge attribute "CREATION_DATE" receives a system-generated URI, while all other URIs are derived from the in-mapping table.

Without In-Mapping Table

The procedure automatically generates the URIs if you do not provide an in-mapping table. The generated URIs are constructed using the schema name, the vertex or edge table name, and the attribute name.

i Note

During the automatic URI creation, if the procedure encounters any characters incompatible with URI generation, they are encoded using standard URI encoding. For instance, a new line character '\n' is encoded to '%0A', and a space is encoded to '%20'.

Vertex Table

The key attribute and its value are used to create the subject URI, which serves as a unique identifier for each vertex. In the example above, the subject URIs generated for the two keys are:

```
'http://EXAMPLE_SCHEMA/EXAMPLE_SCHEMA/ID/John'
'http://EXAMPLE_SCHEMA/EXAMPLE_SCHEMA/ID/Sam'
```

Every vertex is assigned an `rdf:type` property to indicate that the vertex is an instance of a class. This class is defined using the label of the vertex table. If no label is specified during the graph workspace creation, then the vertex table name becomes the default label. For this example, the class URI is generated using the schema name, table name, and label name as shown below:

```
http://EXAMPLE_SCHEMA/VERTEX_TABLE/VERTEX_TABLE
```

If you specify a label, such as 'Employees', for the vertex table, the system generates the class URI as follows:

```
http://EXAMPLE_SCHEMA/VERTEX_TABLE/Employees
```

Predicates are created from the non-key attributes. In this example, they are "HEIGHT" and "WEIGHT." Thus, the generated predicates are:

```
'http://EXAMPLE_SCHEMA/EXAMPLE_SCHEMA/HEIGHT'
'http://EXAMPLE_SCHEMA/EXAMPLE_SCHEMA/WEIGHT'
```

The attribute value becomes the object of the triple. For example, the value of the "HEIGHT" attribute is converted into the object of the corresponding RDF triple.

```
"185.2"^^xsd:double
"177.8"^^xsd:double
```

Similarly, for the "WEIGHT" attribute:

```
"75"^^xsd:long
"83"^^xsd:long
```

Hence, the procedure generates the following triples for the vertex table:

```
<http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/John> rdf:type <http://EXAMPLE_SCHEMA/VERTEX_TABLE/VERTEX_TABLE>
<http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/John> <http://EXAMPLE_SCHEMA/VERTEX_TABLE/HEIGHT> "185.2"^^xsd:double
<http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/John> <http://EXAMPLE_SCHEMA/VERTEX_TABLE/WEIGHT> "75"^^xsd:long
<http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/Sam> rdf:type <http://EXAMPLE_SCHEMA/VERTEX_TABLE/VERTEX_TABLE>
<http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/Sam> <http://EXAMPLE_SCHEMA/VERTEX_TABLE/HEIGHT> "177.8"^^xsd:double
<http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/Sam> <http://EXAMPLE_SCHEMA/VERTEX_TABLE/WEIGHT> "83"^^xsd:long
```

```
<http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/Sam> <http://EXAMPLE_SCHEMA/VERTEX_TABLE/WEIGHT> "83"^^xsd:integer
<http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/Sam> <http://EXAMPLE_SCHEMA/VERTEX_TABLE/HEIGHT> "177.8"^^xsd:float
```

Edge Table

For the edge table, URIs are generated similarly to vertex URIs. The edge label is used to create the predicate connecting the two vertices. If no edge label is specified during the creation of the graph workspace, then the table name is the label by default. So the default predicate connecting two vertices is:

```
http://EXAMPLE_SCHEMA/EDGE_TABLE/EDGE_TABLE
```

If an edge label is defined during the graph workspace creation, for instance 'isManager', then the default predicate is:

```
http://EXAMPLE_SCHEMA/EDGE_TABLE/isManager
```

Each edge attribute is represented as a predicate of the embedded triple, and the corresponding attribute value becomes the object. The triples are produced in RDF* notation as shown below:

```
<<http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/John> <http://EXAMPLE_SCHEMA/EDGE_TABLE/EDGE_TABLE> <http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/John>
<http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/John> <http://EXAMPLE_SCHEMA/EDGE_TABLE/EDGE_TABLE> <http://EXAMPLE_SCHEMA/VERTEX_TABLE/ID/John>
```

Out-Mapping Table

An out-mapping table is generated for each procedure call. This table allows you to inspect the URIs assigned to each attribute, providing visibility into the mapping process. The syntax structure of the out-mapping table is identical to that of an in-mapping table. The `GRAPH_WORKSPACE_FIELD_TYPE` is populated with the SQL type of vertex and edge attributes. In case of a collection, this field would also specify 'VERTEX' or 'EDGE' to distinguish between the properties of vertices and edges.

For the above example with an in-mapping table, the following out-mapping table would be generated:

GRAPH_WORKSPACE_SCHEMA_NAME	GRAPH_WORKSPACE_TABLE_NAME	GRAPH_WORKSPACE_FIELD_NAME	GRAPH_WORKSPACE_FIELD_TYPE
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'		
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'	'ID'	'NVARCHAR'
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'	'HEIGHT'	'DOUBLE'
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'	'WEIGHT'	'BIGINT'
'EXAMPLE_SCHEMA'	'EDGE_TABLE'		
'EXAMPLE_SCHEMA'	'EDGE_TABLE'	'ID'	'INT'
'EXAMPLE_SCHEMA'	'EDGE_TABLE'	'CREATIONDATE'	'TIMESTAMP'

And for the example without an in-mapping table, the following out-mapping table is generated:

GRAPH_WORKSPACE_SCHEMA_NAME	GRAPH_WORKSPACE_TABLE_NAME	GRAPH_WORKSPACE_FIELD_NAME	GRAPH_WORKSPACE_FIELD_TYPE
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'		

GRAPH_WORKSPACE_SCHEMA_NAME	GRAPH_WORKSPACE_TABLE_NAME	GRAPH_WORKSPACE_FIELD_NAME	GRAPH_WORK
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'	'ID'	'NVARCHAR'
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'	'HEIGHT'	'DOUBLE'
'EXAMPLE_SCHEMA'	'VERTEX_TABLE'	'WEIGHT'	'BIGINT'
'EXAMPLE_SCHEMA'	'EDGE_TABLE'		
'EXAMPLE_SCHEMA'	'EDGE_TABLE'	'ID'	'INT'
'EXAMPLE_SCHEMA'	'EDGE_TABLE'	'CREATIONDATE'	'TIMESTAMP'

Mapping Mode

The `MAPPING_MODE` parameter controls how the built-in procedure interprets and applies mapping rules between graph attributes and RDF URIs. The following mapping modes are supported:

AUTO (Default mode)

- If an in-mapping table is defined and includes a graph attribute, then the mapped URI is used for this attribute.
- Otherwise, if the mapping table is not defined or does not include a particular graph attribute, a system-generated URI is created automatically.

IGNORE

- This mode only considers the attributes defined in the mapping table.
- Any attribute that is not included in the mapping table is ignored and not considered to generate RDF triples.

STRICT

- Ensures that all vertex and edge attributes are defined in the in-mapping table.
- The procedure raises an error if any attribute is missing from the mapping table.
- RDF triples are generated only when all attributes are explicitly mapped. If the URI field is left empty for an attribute, then that attribute would be ignored during the conversion.

MAPPING_ONLY

- Allows you to inspect the URI mapping for each attribute without generating the RDF graph.
- This mode is useful for validating and troubleshooting the mapping configuration or for generating a mapping for later adaptation.
- If an unsupported data type is encountered, the corresponding error message is recorded in the `GRAPH_WORKSPACE_FIELD_TYPE` column.

Data Type Mapping

SQL data types are mapped to XML Schema data types as listed below. Other data types are not supported by the procedure.

SQL	XML
SMALLINT	short
TINYINT	short
REAL	float
INT	int
BIGINT	long
SMALLDECIMAL	decimal
DECIMAL	decimal
DOUBLE	double
BOOLEAN	boolean
DATE	date
DAYDATE	date
TIME	time
TIMESTAMP	dateTime
LONGDATE	dateTime
VARBINARY	base64Binary
VARCHAR	string
NVARCHAR	string

The JSON-language data types are mapped to XML Schema data types as listed below. Other data types, such as OBJECT, ARRAY, etc. are not supported.

JSON	XML
BOOLEAN	boolean
INTEGER	long
DOUBLE	double
DECIMAL	decimal
STRING	string

i Note

The data type integer in JSON language is mapped to BIGINT in SQL; hence, it is mapped to xsd:long.

Variants of Graph Workspaces

A graph workspace can be defined on tables, views, synonyms, or collections. Additionally, graph workspaces can have the following characteristics:

- **Edge-only:** Defined without a vertex table.
- **Composite keys:** Vertices or edges identified by multiple attributes.
- **Heterogeneous:** Containing mixed data types or structures.
- **Collection-based:** Defined on a simple collection or a collection with a JSON schema, with nested values.

Due to these variations, the procedure applies different transformation rules, as described below:

Edge-Only

If no in-mapping table is provided for edge-only graph workspaces, then the procedure generates vertex URIs using the source and target column names from the edge table. Consider the following example:

ID	SOURCE	TARGET
1	'John'	'Sam'

This generates the following RDF* triples:

```
<<http://EXAMPLE_SCHEMA/EDGE_TABLE/SOURCE/John> <http://EXAMPLE_SCHEMA/EDGE_TABLE/EDGE_TABLE> <http://EXAMPLE_SCHEMA/EDGE_TABLE/TARGET/Sam>
```

Composite Keys

For graph workspaces having composite keys (for example, vertices or edges identified by more than one key attribute), the procedure generates a blank node as the vertex identifier. For edges, this is not required because the edge key attributes are converted to predicates of the RDF triple, preserving their uniqueness without the need for a blank node.

Consider the following vertex table with key attributes "VERTEX_ID_1" and "VERTEX_ID_2":

VERTEX_ID_1	VERTEX_ID_2	NAME
1	99	'Alex'
2	98	'Kim'

These would be converted to the following triples:

```
_:b53687092225 rdf:type <http://SIMPLECOMPOSITEKEYGRAPH/VERTEX_TABLE/VERTEX_TABLE> .
_:b53687092225 <http://SIMPLECOMPOSITEKEYGRAPH/VERTEX_TABLE/vertex_id_1> "1"^^xsd:int .
_:b53687092225 <http://SIMPLECOMPOSITEKEYGRAPH/VERTEX_TABLE/vertex_id_2> "99"^^xsd:int .
_:b53687092225 <http://SIMPLECOMPOSITEKEYGRAPH/VERTEX_TABLE/name> "Alex" .
_:b75161928705 rdf:type <http://SIMPLECOMPOSITEKEYGRAPH/VERTEX_TABLE/VERTEX_TABLE> .
_:b75161928705 <http://SIMPLECOMPOSITEKEYGRAPH/VERTEX_TABLE/vertex_id_1> "2"^^xsd:int .
_:b75161928705 <http://SIMPLECOMPOSITEKEYGRAPH/VERTEX_TABLE/vertex_id_2> "98"^^xsd:int .
_:b75161928705 <http://SIMPLECOMPOSITEKEYGRAPH/VERTEX_TABLE/name> "Kim" .
```

Graph on Collection

As described in the "Graph Workspaces on Collections" section of the [Graph Workspaces](#) topic, graph workspaces based on simple collections inherently recognize only the following columns: ID, SOURCE, TARGET, and TYPE.

By default, only these columns are converted to RDF triples. Additional attributes can be included by specifying them in the in-mapping table.

Consider a collection named "EMPLOYEES" that contains data with nested values.

```
"id": {"value" : 0 } , "type": "vertex", "name": "Mary", "role": "senior manager"
"id": {"value" : 1 }, "type": "vertex", "name": "John", "role": "manager"

"id": {"value" : 5 }, "type": "edge", "src": {"value" :1}, "tgt": {"value" :0}, "relation" : "isMar"
```

And consider an in-mapping table with nested attributes. Nested attributes should be dot-separated to indicate the hierarchy. The components of the nested attributes can be specified with or without quotes.

For example:

- address.city
- "address"."city"

Both notations are valid and will be interpreted equivalently by the procedure. The default generation of the nested fields is separated by a slash character as a delimiter. For example, 'address.city' is converted to address/city. If an attribute that is not present in the collection is defined in the in-mapping table, it is ignored.

GRAPH_WORKSPACE_SCHEMA_NAME	GRAPH_WORKSPACE_NAME	GRAPH_WORKSPACE_FIELD_NAME	GRAPH_WORKSPACE_
'EXAMPLE_SCHEMA'	'EMPLOYEES'		
'EXAMPLE_SCHEMA'	'EMPLOYEES'	""id"."value""	
'EXAMPLE_SCHEMA'	'EMPLOYEES'	'name'	

Graph on JSON Schema Collection

If a graph on collection is based on a JSON schema, then the fields that are part of the JSON schema definition are also considered during generation, in addition to ID, SOURCE, TARGET, and TYPE.

Consider the following example:

```
CREATE COLLECTION CUSTOMERS JSON SCHEMA '{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "properties": {
    "id": {
      "type": "integer"
    },
    "src" : {
      "type": "integer"
    },
    "tgt" : {
      "type": "integer"
    },
    "type" : {
      "type": "string"
    },
  },
}
```

```

    "address": {
      "type": "object",
      "properties": {
        "street": {
          "type": "string"
        },
        "city": {
          "type": "string"
        },
        "zip": {
          "type": "integer"
        }
      }
    },
    "name": {
      "type": "string"
    }
  },
  "required": ["id"],
  "additionalProperties": true
}

```

And with the following data:

```

INSERT INTO CUSTOMERS VALUES({'id': 10, "name": "Smith", "address":{"street": "155 Beechwood St.",
INSERT INTO CUSTOMERS VALUES({'id': 20, "name": "Johnson", "address":{"street": "1499 Grove Street
INSERT INTO CUSTOMERS VALUES({'id': 10, "src": 10, "tgt": 20, "type": "edge", "relationship": "isf

```

Without an in-mapping table, the procedure generates the following triples for vertices and edges:

```

<http://JSON_SCHEMA/CUSTOMERS/id/10> rdf:type <http://JSON_SCHEMA/CUSTOMERS/CUSTOMERS> .
<http://JSON_SCHEMA/CUSTOMERS/id/10> <http://JSON_SCHEMA/CUSTOMERS/address/street> "155 Beechwood St." .
<http://JSON_SCHEMA/CUSTOMERS/id/10> <http://JSON_SCHEMA/CUSTOMERS/address/city> "Seattle" .
<http://JSON_SCHEMA/CUSTOMERS/id/10> <http://JSON_SCHEMA/CUSTOMERS/name> "Smith" .
<http://JSON_SCHEMA/CUSTOMERS/id/10> <http://JSON_SCHEMA/CUSTOMERS/address/zip> "20005"^^xsd:long .
<http://JSON_SCHEMA/CUSTOMERS/id/20> <http://JSON_SCHEMA/CUSTOMERS/address/city> "Long Island" .
<http://JSON_SCHEMA/CUSTOMERS/id/20> rdf:type <http://JSON_SCHEMA/CUSTOMERS/CUSTOMERS> .
<http://JSON_SCHEMA/CUSTOMERS/id/20> <http://JSON_SCHEMA/CUSTOMERS/address/zip> "11788"^^xsd:long .
<http://JSON_SCHEMA/CUSTOMERS/id/20> <http://JSON_SCHEMA/CUSTOMERS/name> "Johnson" .
<http://JSON_SCHEMA/CUSTOMERS/id/20> <http://JSON_SCHEMA/CUSTOMERS/address/street> "1499 Grove Street" .
<<http://JSON_SCHEMA/CUSTOMERS/id/10> <http://JSON_SCHEMA/CUSTOMERS/CUSTOMERS> <http://JSON_SCHEMA,

```

An edge attribute named "relationship" is also inserted into the collection. However, because this attribute is not defined in the JSON-Schema, the procedure is unaware of its existence and therefore does not generate triples. To ensure the procedure includes this attribute, specify it in an in-mapping table or add it to the JSON Schema definition.

Graph Data Modification

Any change to the edge table or the vertex table affects the edges or the vertices of the graph.

The following SQL statements create two new vertices with vertex keys PEK (Beijing) and TNA (Jinan), and an edge between them in the "FLIGHTS"."GRAPH" workspace.

For the complete list of statements for creating the example graph in [SAP HANA Property Graph Data Model](#), see [Appendix A - Open Flights, Company and Employees Graph Examples](#).

```
INSERT INTO "FLIGHTS"."NODES" VALUES (
  'PEK', 'Airport', 'Beijing Capital International Airport', 'Beijing', 'China',
  ST_GeomFromText('POINT(116.58499908447266 40.080101013183594)', 4326));
INSERT INTO "FLIGHTS"."NODES" VALUES (
  'TNA', 'Airport', 'Yaoqiang Airport', 'Jinan', 'China',
  ST_GeomFromText('POINT(117.21600341796875 36.857200622558594)', 4326));

INSERT INTO "FLIGHTS"."EDGES" ("SOURCE", "TARGET", "DIST_KM") VALUES ('PEK', 'TNA', 361.969);
```

Vertices can be modified and deleted using SQL UPDATE and DELETE statements. The following statement modifies the "COUNTRY" attribute of all vertices with "COUNTRY" 'China'.

```
UPDATE "FLIGHTS"."NODES" SET "COUNTRY" = 'PR China'
WHERE "COUNTRY" = 'China';
```

The following statement deletes those edges from the graph workspace "FLIGHTS"."GRAPH" that contain 'FRA' as "SOURCE" or "TARGET".

```
DELETE FROM "FLIGHTS"."EDGES"
WHERE "SOURCE" = 'FRA' or "TARGET" = 'FRA';
```

Vertices can be inserted, updated, and deleted in the same way as edges.

```
DELETE FROM "FLIGHTS"."NODES"
WHERE "NODE_KEY" IN ('FRA', 'AMS');
```

If the consistency of a graph workspace is not guaranteed by referential constraints, modifications of the referenced graph tables can lead to an inconsistent graph workspace. The exact behavior in case of inconsistent graph workspaces is different for each Graph Engine component. See [CREATE GRAPH WORKSPACE Statement \(Data Definition\)](#) and [DROP GRAPH WORKSPACE Statement \(Data Definition\)](#).

The uniqueness of vertices in the "FLIGHTS"."GRAPH" workspace can be checked with the following query on the vertex table:

```
SELECT "NODE_KEY", COUNT(*)
FROM "FLIGHTS"."NODES"
GROUP BY "NODE_KEY"
HAVING COUNT(*) > 1;
```

If the result of above query is not empty, then the vertex key column ("NODE_KEY") contains duplicates and the graph workspace is inconsistent.

An analogous query can be used on the edge table to check the uniqueness of edge keys:

```
SELECT "EDGE_KEY", COUNT(*)  
FROM "FLIGHTS"."EDGES"  
GROUP BY "EDGE_KEY"  
HAVING COUNT(*) > 1;
```

The correctness of the source and target vertex references can be checked with the following query:

```
SELECT "EDGE_KEY", "SOURCE", "TARGET"  
FROM "FLIGHTS"."EDGES"  
WHERE  
    "SOURCE" NOT IN (SELECT "NODE_KEY" FROM "FLIGHTS"."NODES")  
    OR  
    "TARGET" NOT IN (SELECT "NODE_KEY" FROM "FLIGHTS"."NODES");
```

If the result of the above query is not empty, then either the source or target of the resulting edge is not a valid vertex key, leading to an inconsistent graph workspace.

GraphScript Language

GraphScript is an imperative programming language that provides application developers with a high-level interface for accessing the graph data defined by a graph workspace.

The GraphScript language eases the development of application-specific graph algorithms and integrates them into SQL-based data processing. Furthermore, GraphScript provides optimized built-in algorithms to solve common graph-related problems, such as finding the shortest path from one vertex to another. These algorithms are available as built-in functions that can be reused in any GraphScript program to simplify the development of efficient solutions for customer-specific, graph-related problems.

In case of inconsistent graph workspaces, a GraphScript program will terminate with an error.

Integration into the SAP HANA Environment

The GraphScript language can be used in the following HANA SQL logic containers: Stored procedures, anonymous blocks, and table user-defined functions. Logic containers with GraphScript code support only a subset of the options and data types supported by SQLScript. Besides these differences, logic containers with GraphScript code can be created, dropped and used in the very same way as logic containers with SQLScript code.

GraphScript Procedures

GraphScript procedures do not make any modifications to the database data or its structure and are therefore marked as read-only. Similar to SQLScript procedures, GraphScript procedures support debugging from the SAP HANA Database Explorer.

Syntax

Creates a GraphScript procedure.

```
CREATE [OR REPLACE] PROCEDURE <procedure_name> (<procedure_parameters>)
LANGUAGE GRAPH [<sql_security_mode>] [<default_schema_specification>]
READS SQL DATA AS
BEGIN
    <GraphScript_procedure_body>
END
```

Syntax Elements

<procedure_name>

Specifies the name of the procedure with an optional schema name.

```
<procedure_name> ::= [<schema_name>.]<identifier>
```

<procedure_parameters>

Specifies the parameters of the procedure.

```
<procedure_parameters> ::= <in_out_inout_parameter> [ { , <in_out_inout_parameter> } [...] ]
```

```
<in_out_inout_parameter> ::= <in_out_inout_specifier> <parameter_name> <data_or_table_type>
```

```
<in_out_inout_specifier> ::= IN | OUT | INOUT
```

```
<parameter_name> ::= <identifier>
```

```
<data_or_table_type> ::= <scalar_data_type> | <table_type> | <table_type_definition>
```

```
<scalar_data_type> ::=
```

```
INTEGER
| BIGINT
| DOUBLE
| BOOLEAN
```

```
| NVARCHAR [ (<unsigned_integer>) ]
| TIMESTAMP
```

The input and output parameters of a procedure can have any of the [primitive SQL types supported by GraphScript](#) or a table type. INOUT parameters must be of a supported scalar type - table type parameters are either IN or OUT parameters. The default value of scalar output parameter variables is NULL.

<table_type>

Specifies a table type that was previously defined with the CREATE TYPE statement.

```
<table_type> ::= [<schema_name>.]<identifier>
```

<table_type_definition>

Specifies a table type that is defined within the signature of the procedure.

```
<table_type_definition> ::= TABLE (<column_list_definition>)
<column_list_definition> ::= <column_definition> [ { , <column_definition> } [...] ]
<column_definition> ::= <column_name> <scalar_data_type>
<column_name> ::= <identifier>
```

Table parameters with primary key or unique constraints are not supported. Additionally, the columns of output tables must be NULL-able. Therefore, NOT NULL constraints are not supported for output tables.

<sql_security_mode>

Specifies the security mode of the procedure. The default is DEFINER.

```
<sql_security_mode> ::= SQL SECURITY <mode>
<mode> ::= DEFINER | INVOKER
```

In the DEFINER security mode, the execution of the procedure is performed with the privileges of the definer of the procedure. In the INVOKER security mode, the execution of the procedure is performed with the privileges of the invoker of the procedure.

<default_schema_specification>

Specifies the schema for unqualified objects in the procedure. If the optional default schema specification is missing, then the current schema of the session at the time of the procedure definition is used.

```
<default_schema_specification> ::= DEFAULT SCHEMA <schema_name>
```

Description

Once a procedure has been created, the CALL procedure statement can be used to executed the procedure. Procedures can be dropped using the DROP PROCEDURE statement.

The [GraphScript Procedure Example](#) illustrates the creation and usage of a GraphScript procedure.

GraphScript Anonymous Blocks

A GraphScript anonymous block is an executable DML statement which may contain an arbitrary GraphScript program. For anonymous blocks, no corresponding object is created in the catalog.

Syntax

A GraphScript anonymous block is defined and executed in a single step by using the following syntax:

```
DO [(

```

<body> ::= !! supports the same feature set as GraphScript procedures

The mandatory language identifier for GraphScript anonymous blocks is GRAPH. Anonymous blocks are always read only. Therefore, the READS SQL DATA specification is not needed.

Syntax Elements

<anonymous_block_parameters>

Describes the parameters of the anonymous block.

```
<anonymous_block_parameters> ::= <named_parameter> [ { , <named_parameter> } [...] ]
<named_parameter> ::= {IN | OUT} <parameter_name> <parameter_type> => <parameter_value>
<parameter_name> ::= <identifier>
<parameter_type> ::= <scalar_data_type> | <table_type> | <table_type_definition>
<parameter_value> ::= ? | <identifier> | <string_literal> | <unsigned_integer> |
<signed_integer> | <signed_numeric_literal> | <unsigned_numeric_literal> | <expression>
```

Description

Anonymous blocks only support IN and OUT parameters - unlike procedures they do not support INOUT parameters. GraphScript anonymous blocks support the same data types as GraphScript procedures. Analogous to procedures, the default value of scalar output parameter variables is NULL.

Unlike the parameter definition in the CREATE PROCEDURE statement, anonymous block parameters require in-place initialization using the '=>' syntax. Input parameters can be initialized either with scalar expressions or like prepared parameters with '?'. The values of the prepared parameters must be provided when the block is executed. The anonymous block is compiled only once and it can be executed multiple times with different prepared parameter values. Output parameters must be initialized with '?'.

```
DO (IN in_val BigInt => ?, OUT out_val BigInt => ?) LANGUAGE GRAPH
BEGIN
```

```

    out_val = :in_val;
END

```

Anonymous blocks cannot be used within procedures or functions.

Currently, GraphScript anonymous blocks do not provide support for debugging.

The [GraphScript Anonymous Block Example](#) illustrates the usage of a GraphScript anonymous block with an output table parameter for which the type is defined inline in the parameter list.

GraphScript Table User-Defined Functions

Similar to SQLScript, GraphScript supports table user-defined functions (TUDF). A GraphScript TUDF can only be called within the FROM-clause of an SQL statement at the same parameter positions as table names.

Syntax

Creates a GraphScript table function.

```

CREATE [OR REPLACE] FUNCTION <function_name> [( <function_parameters> )]
    <return_clause>
LANGUAGE GRAPH [<sql_security_mode>] [<default_schema_specification>]
[READS SQL DATA] AS
BEGIN
    <GraphScript_function_body>
END

```

Syntax Elements

The optional syntax elements <sql_security_mode> and <default_schema_specification> are the same as for the GraphScript procedures defined above.

<function_name>

Specifies the name of the function with an optional schema name.

```
<function_name> ::= [<schema_name>.]<identifier>
```

<function_parameters>

Specifies the parameters of the table function.

```

<function_parameters> ::= <in_parameter> [ { , <in_parameter> } [...] ]
<in_parameter> ::= [IN] <parameter_name> <data_or_table_type>

```

Functions have only input parameters, therefore the specifier IN is optional. Functions support the same parameter data types as procedures.

<return_clause>

Defines the type of the return table.

```
<return_clause> ::= RETURNS { <table_type> | <table_type_definition> }
```

Description

GraphScript table functions support the same feature set as GraphScript procedures. Additionally, a table function must return a table of the type defined in the return clause. To this end, GraphScript provides the [return table statement](#).

Table functions are always read only. Therefore, the READS SQL DATA specification is optional.

The [GraphScript Table User-Defined Function Example](#) illustrates the usage of a GraphScript table user-defined function which determines all the employees who work on projects that produce products bought by a specific customer based on the Company graph described in [Appendix A - Open Flights, Company and Employees Graph Examples](#).

Language Fundamentals

The GraphScript grammar consists of basic language elements, supports variables, and provides options for specifying the control flow.

Grammar

In general, GraphScript's grammar consists of three different fundamental types of constructs:

- Statements: self-containing language constructs that can have side effects
- Expressions: language constructs that return a value of a given type
- Terminal symbols: special characters and symbols

Statements

A GraphScript program consists of an arbitrary number of statements.

```
<script> ::= <statements>
<statements> ::= { <statement> }
```

The following types of statements are supported in GraphScript:

```
<statement> ::=
<definition>
| <assignment>
| <conditional>
| <foreach_loop>
| <return_table_statement>
| <temp_attribute>
| <while_loop>
| <traversal>
| <end_traversal>
```

Expressions

An expression is a language construct that returns a value of a given type.

```
<expr> ::=
<literal_expr>
| <attr_access_expr>
| <local_var_expr>
| <arithmetic_expr>
| <relational_expr>
| <logical_expr>
| <collection_init_expr>
| <function_expr>
| <set_operation>
| <concat_expr>
| <index_expr>
| <cell_expr>
| <filter_expr>
| <proj_expr>
```

```
| <clos_expr>
| <order_expr>
```

GraphScript supports the following expressions:

- Literal expressions (`literal_expr`)
- Attribute access expressions (`attr_access_expr`)
- Local variable expressions (`local_var_expr`)
- Arithmetic expressions (`arithmetic_expr`)
- Relational expressions (`relational_expr`)
- Logical expressions (`logical_expr`)
- Collection initializer list expressions (`collection_init_expr`)
- Function expressions (`function_expr`)
- Set operations (`set_operation`)
- Concatenation expressions (`concat_expr`)
- Positional access expressions (`index_expr`)
- Cell access expressions (`cell_expr`)
- Filter expressions (`filter_expr`)
- Projection expressions (`proj_expr`)
- Closure expressions (`clos_expr`)
- Order expressions (`order_expr`)

Terminal Symbols

The following list summarizes special characters and symbols that are used in GraphScript.

```
<comma> ::= ,
<single_quote> ::= '
<double_quote> ::= "
<l_paren> ::= (
<r_paren> ::= )
<l_curly> ::= {
<r_curly> ::= }
<l_square> ::= [
<r_square> ::= ]
<pipe> ::= |
<semicolon> ::= ;
<colon> ::= :
<dot> ::= .
<equals> ::= =
<minus> ::= -
<plus> ::= +
<lower> ::= <
<lower_equal> ::= <=
<greater> ::= >
<greater_equal> ::= >=
<leads_to> ::= =>
<equal> ::= ==
<unequal> ::= !=
<membership_in> ::= IN
```

```

<membership_not_in> ::= NOT IN
<underscore> ::= _
<asterisk> ::= *
<slash> ::= /
<modulo> ::= %
<circumflex> ::= ^
<any_character> ::= !! any character
<hash_symbol> ::= #
<dollar_sign> ::= $
<questionmark> ::= ?
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<pos_digit> ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<letter> ::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q |
           r | s | t | u | v | w | x | y | z | A | B | C | D | E | F | G | H | I | J |
           K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z

```

Comments

GraphScript supports both single-line and multiline comments.

A single-line comment starts with "--"; all characters that follow in the line are treated as a comment. A multiline comment is enclosed in "/*" and "*/".

Examples

```
INTEGER v = 23; -- this is a single line comment
```

```

FOREACH e IN Edges(:g) {
/* this is a multi line comment
  INTEGER i = :e.attr;
  INTEGER v = 23;*/
}

```

Local Variables

GraphScript supports local variables that split large and complex statements into smaller ones, improving the script's overall readability. After initial assignment or declaration (see [Assignments](#)), the type of a variable can no longer be modified. A local variable expression can be used anywhere that an expression is supported, as long as the corresponding expression types match. The names of variables are case insensitive. Variable references that are used for read access must be preceded by a colon (:), while references for write access have no leading colon.

```

<variable> ::= <letter> { <letter> | <digit> | <underscore> }
<variable_reference> ::= <colon> <letter> { <letter> | <digit> | <underscore> }
<local_var_expr> ::= <variable_reference>

```

Examples

```

aVariable
aSecondVariable12
:aVariable

```

Definition Statements

A definition statement declares and defines a local variable of a specific type. Objects of primitive types are default-initialized to NULL. Additionally, it's possible to do this explicitly by using NULL. Vertices, edges, and graphs have to be initialized in the declaration. A variable is visible and accessible in the current scope and all inner scopes, such as in loops and conditional statements. Variables defined inside loops and conditional statements aren't accessible outside the statements. GraphScript doesn't support variable shadowing; in other words, a variable defined in an outer scope can't be redefined in an inner scope.

```
<definition> ::= <vertex_or_edge> <variable> = <expression> ;
| <primitive_type> <variable> [ = <expression> ] ;
| <primitive_type> <variable> NULL ;
| <graph> <variable> = <expression> ;
| <multiset_type> <variable> = <expression> ;
| <sequence_type> <variable> = <expression> ;
```

Examples

```
GRAPH g1 = GRAPH("FLIGHTS","GRAPH");
GRAPH g2 = GRAPH("GRAPH");
GRAPH g3 = GRAPH(mygraph);
VERTEX v(:g1,1);
EDGE e(:g1,'3');
INTEGER i;
DOUBLE d = 23.5;
NVARCHAR s = 'Dave';
```

Assignments

An assignment statement binds the result of an expression to a local variable. Alternatively, if the left-hand side of the assignment statement is an attribute access expression, the value of the right-hand side expression is assigned to the corresponding attribute.

```
<assignment> ::= <variable> = <expr> ;
| <variable> = NULL ;
| <variable> = <projection_expr> ;
| <attr_access_expr> = <expr> ;
```

GraphScript is statically typed and the data type of the object that is bound to the variable can't change during the lifetime of a single query run. Only variables of primitive types can be set to NULL. All variables have to be initialized before they can be referenced or accessed in a GraphScript expression.

Examples

```
GRAPH g = GRAPH("MYWORKSPACE");
INTEGER i;
i = 23;
VERTEX v(:g,1);
v.attr = 23;
```

Identifiers

GraphScript uses identifiers to reference schema names, graph workspace names, and attribute names. Similar to SQL, identifiers are case-sensitive when enclosed by double quotes, and case-insensitive when used without double quotes. Best practice is to always use double quotes.

Identifiers may look like literal values of type NVARCHAR, but they use double quotes instead of single quotes. The two data types are not interchangeable.

```
<simple_identifier> ::= ( <letter> | <underscore> ) { <letter> | <underscore> |
    <digit> | <hash_symbol> | <dollar_sign> }
<quoted_identifier> ::= <double_quote> { <any_charater> } <double_quote>
<identifier> ::= <simple_identifier> | <quoted_identifier>
```

Conditional Statements

A conditional statement consists of an IF body and an optional ELSE body. Each body contains an arbitrary number of statements enclosed in curly braces. If the expression <expr> in the IF branch evaluates to TRUE, the corresponding statements in the IF body are executed, otherwise the statements in the ELSE branch are executed.

```
<conditional> ::= IF '(' <expr> ')'
    '{' <statements> '}'
    [ ELSE '{' <statements> '}' ]
```

Examples

```
INTEGER i;
IF (2 < 3) {
    IF (TRUE) {
        i = 5;
    } ELSE {
        i = 17;
    }
}
```

FOREACH Loops

A FOREACH loop iterates over all elements in a container (Multiset, Sequence or Map). The iteration order is not specified and might differ between two script executions. The iteration variables defined in the loop head, e.g., element value, are scoped to the loop body and therefore are not available in the outside scope. Furthermore, all iteration variables are read-only, i.e., new values cannot be assigned to them. Similarly, when iterating over a named container, assignments to the container variable are not allowed in the loop body.

The special loop statements BREAK and CONTINUE are only available inside a loop body. BREAK stops the loop execution and CONTINUE skips to the next iteration.

Multiset and Sequence consist of single-value elements. One iteration variable must be defined before the IN keyword to refer to the element which is processed in the current iteration. The expression following the IN keyword must evaluate to a Multiset or Sequence container. For Sequences, a second iteration variable can be optionally defined. It refers to the ordinal position of the element inside the Sequence.

Maps contain key-value pairs. Hence, two iteration variables must be defined before the IN keyword. They refer to the key and value part of an element pair, respectively.

```
<foreach_loop> ::= FOREACH <variable> IN <expr> [WITH ORDINALITY AS <variable>] '{' <statements> '}'
    | FOREACH '(' <variable> ',' <variable> ')' IN <expr> '{' <statements> '}'
```

Examples

```

GRAPH g = GRAPH("MYWORKSPACE");
INT sumVal = 0;
FOREACH v IN VERTICES(:g) {
    sumVal = :sumVal + :v.attr;
}

SEQUENCE<VERTEX> seq = [ VERTEX(:g,1), VERTEX(:g,2) ];
FOREACH v IN :seq WITH ORDINALITY AS ord {
    v.positionAttr = :ord;
}

MAP<VERTEX,DOUBLE> costMap(:g, 1024L);
-- some code filling costMap
FOREACH (v,cost) IN :costMap {
    -- iteration over vertex-cost pair of costMap assigned to variables v and cost
}

```

WHILE Loops

A WHILE loop repeats the statements specified in the loop body as long as the condition in the loop head evaluates to true. Within a WHILE loop, the keywords BREAK and CONTINUE can be used. BREAK stops the execution of the loop and CONTINUE skips to the next loop iteration.

```
<while_loop> ::= WHILE '(' <expr> ')' '{' <statements> '}'
```

Examples

```

INTEGER a = 10;
WHILE (:a > 0) {
    a = :a - 1;
}

a = 10;
INTEGER b = 3;
WHILE (:a > 0) {
    a = :a - 1;
    IF( :a % 2 == 0 ) {
        CONTINUE;
    }
    b = :b - 1;
    IF(:b == 0) {
        BREAK;
    }
}

```

Graph Primitive Types and Operations

Types and operations that make up the domain-specific core of the GraphScript language.

Graph Primitives

The central domain-specific entities of graph processing are GRAPH, VERTEX, EDGE, and WEIGHTEDPATH.

- GRAPH

A GRAPH variable specifies a structured collection of vertices and edges.

A graph refers to a [graph workspace catalog object](#) specified by workspace name, an inverse graph for an already-defined graph, or a subgraph of an already-defined graph. Alternatively, a graph can be created from [two input tables](#) containing vertices and edges.

You must initialize a GRAPH variable at the time of declaration. There's no default initialization for GRAPH variables.

- VERTEX

The VERTEX data type specifies a node entity in the graph.

An existing vertex object can be retrieved from a GRAPH using its key. Supported keys are composed out of components with a `scalar_key_type` (INTEGER, BIGINT, NVARCHAR) type. If the graph workspace contains multiple vertex tables, then it is mandatory to specify the vertex label in addition to the vertex key. The creation of new vertices within GraphScript isn't supported.

You must initialize a VERTEX variable at the time of declaration. There's no default initialization for VERTEX variables.

- EDGE

The EDGE data type specifies a relationship between two vertices in the graph.

An existing edge object can be retrieved from a GRAPH by using its key. Like for VERTEX, the keys need to be composed out of components with a `scalar_key_type` type. If the graph workspace contains multiple edge tables, then also the edge label must be specified in addition to the edge key. The creation of new edges isn't supported.

You must initialize an EDGE variable at the time of declaration. There's no default initialization for variables of type EDGE.

- WEIGHTEDPATH

The WEIGHTEDPATH data type specifies a path with an associated weight. A WEIGHTEDPATH consists of a sequence of zero, one, or more vertices connected by edges.

You create a WEIGHTEDPATH by calling one of the built-in shortest path functions. GraphScript supports several operations on WEIGHTEDPATH variables, such as extracting its vertices and edges and calculating the length and the weight of a path. You must initialize a WEIGHTEDPATH variable at the time of declaration. There's no default initialization for WEIGHTEDPATH variables. The WEIGHTEDPATH type must be specialized with a weight type. The following weight types are currently supported: INT, DOUBLE, and BIGINT. These types lead to the following WEIGHTEDPATH types: WEIGHTEDPATH<INT>, WEIGHTEDPATH<DOUBLE>, and WEIGHTEDPATH<BIGINT>.

Basic Graph Operations

Constructors

Creates a graph object. Two different variations are available:

```
<graph_function> ::= GRAPH '(' <schema_name>, <workspace_name> ')'
<schema_name> ::= <identifier>
<workspace_name> ::= <identifier>
```

Creates a graph object based on the specified workspace in the specified schema. If the schema parameter is omitted, then the SQL schema inference rules apply. In addition to directly referring to a workspace database object, a synonym or public synonym of a workspace object could be referenced as well.

```
<graph_function> ::= GRAPH '('
    <edge_table_variable>,
    <source_column_identifier>,
    <target_column_identifier>,
    <edge_id_column_identifier>,
    <vertex_table_variable>,
    <vertex_id_column_identifier> ')'

<edge_table_variable> ::= <variable_reference>
<source_column_identifier> ::= <identifier>
<target_column_identifier> ::= <identifier>
<edge_id_column_identifier> ::= <identifier>
<vertex_table_variable> ::= <variable_reference>
<vertex_id_column_identifier> ::= <identifier>
```

Creates a graph object based on the specified edge and vertex tables, which are passed as arguments to the GraphScript procedure. Besides the edge and vertex table variables, the column identifiers of edge source, edge target, edge key, and vertex key need to be specified.

Examples

```
GRAPH g1 = GRAPH("FLIGHTS", "GRAPH");
```

```
CREATE TYPE flightsEdgeType AS TABLE(
    edge_key BIGINT,
    source NVARCHAR(100),
    target NVARCHAR(100),
    dist_km DOUBLE
);
CREATE TYPE flightsVertexType AS TABLE(
    node_key NVARCHAR(100),
    name NVARCHAR(100)
);
```

```
CREATE PROCEDURE graph_from_tabular_input (
    IN i_edges flightsEdgeType, IN i_vertices flightsVertexType)
LANGUAGE GRAPH READS SQL DATA AS
BEGIN
    GRAPH tmp_graph = GRAPH(:i_edges, "SOURCE", "TARGET", "EDGE_KEY", :i_vertices, "NODE_KEY");
    -- ...
END;
```

VERTICES

Returns all vertices in a graph.

- `VERTICES (GRAPH) => MULTISET`

EDGES

Returns all edges in a graph.

- `EDGES (GRAPH) => MULTISET`

Temporary Attribute Definition

A temporary attribute can be used to store vertex-centric or edge-centric state information at script runtime. Read access to temporary attributes is the same as for persistent attributes; write access is only allowed for temporary attributes.

You can specify temporary attributes only during creation of the graph object. If you expect a temporary attribute to be set for more than 10-15% of vertices or edges add the `DENSE` keyword to the specification. The `DENSE` keyword allows the compiler to use a more efficient implementation for more frequently used attributes.

```
<graph_function> ::= GRAPH '('
    <edge_table_variable>,
    <source_column_identifier>,
    <target_column_identifier>,
    <edge_id_column_identifier>,
    <vertex_table_variable>,
    <vertex_id_column_identifier> ')'
    WITH TEMPORARY ATTRIBUTES '(' <attr_list> ')'
```

```
<attr_list> ::= <attr_spec> | <attr_list> , <attr_spec>
<attr_spec> ::= <vertex_or_edge> <primitive_type> <identifier> [ = <expr> ] [ DENSE ]
```

Examples

```
GRAPH g1 = GRAPH("FLIGHTS", "GRAPH")
    WITH TEMPORARY ATTRIBUTES(VERTEX INTEGER distance = 0, EDGE DOUBLE price = 0.0);
GRAPH g2 = GRAPH("ROADS", "NETWORK")
    WITH TEMPORARY ATTRIBUTES(VERTEX INTEGER typeId DENSE);
```

With Attributes Definition

JSON documents can be deeply nested and there is no apriori definition of which fields exist or their data type. In general, the data stored in a collection is semi-structured and schema-less. GraphScript is a statically typed language which means that the data type of each variable must be known at compilation time. For graph workspaces defined on collections, the `WITH ATTRIBUTES` specification can be used to declare the expected data type of a persistent vertex or edge attribute.

```
<graph_function> ::= GRAPH '('
    <schema_name_identifier>,
    <workspace_name_identifier> ')'
    WITH ATTRIBUTES '(' <attr_list> ')'
```

```
<attr_list> ::= <attr_spec> | <attr_list> , <attr_spec>  
<attr_spec> ::= <vertex_or_edge> <primitive_type> <identifier> [ = <expr> ]
```

Examples

```
GRAPH g = GRAPH("GRAPH_SCHEMA", "EMPLOYEES_GRAPH")  
  WITH ATTRIBUTES (VÉRTEX NVARCHAR "role", EDGE BIGINT "weight");
```

Basic Vertex Operations

Constructors

Constructs a [vertex](#) reference from a graph or a weighted path, a label and a (composite) key. If multiple vertex tables are defined in the graph workspace, it is mandatory to provide a label in the vertex constructor. The label comes directly after the graph or path variable and right before the key components (scalar_key_type) in the argument list of the VERTEX constructor. If no vertex with the given key exists, then the program terminates with an error. Use the VERTEX_EXISTS function to determine whether a vertex exists.

- VERTEX(GRAPH, NVARCHAR, scalar_key_type[, scalar_key_type]) => VERTEX
- VERTEX(WEIGHTEDPATH, NVARCHAR, scalar_key_type[, scalar_key_type]) => VERTEX

In the case that only a single vertex table is defined, the label can be omitted.

- VERTEX(GRAPH, scalar_key_type[, scalar_key_type]) => VERTEX
- VERTEX(WEIGHTEDPATH, scalar_key_type[, scalar_key_type]) => VERTEX

A special case is the construction of a vertex from a compatible vertex. Vertices are compatible if their host graphs are defined in the same graph workspace, e.g., a weighted path, a subgraph and an inversed graph are compatible with the graph they are constructed from, as they all share the same graph workspace.

- VERTEX(GRAPH, VERTEX) => VERTEX
- VERTEX(WEIGHTEDPATH, VERTEX) => VERTEX

VERTEX_EXISTS

Checks if the given vertex, or a vertex with the given (composite) vertex key exists in a graph or a weighted path. If multiple vertex tables are defined in the graph workspace, it is mandatory to provide a label for the check by key. The label comes directly after the graph or path variable.

- VERTEX_EXISTS(GRAPH, VERTEX) => BOOL
- VERTEX_EXISTS(GRAPH, NVARCHAR, scalar_key_type[, scalar_key_type]) => BOOL
- VERTEX_EXISTS(WEIGHTEDPATH, VERTEX) => BOOL
- VERTEX_EXISTS(WEIGHTEDPATH, NVARCHAR, scalar_key_type[, scalar_key_type]) => BOOL

In the case that only a single vertex table is defined, the label can be omitted.

- VERTEX_EXISTS(GRAPH, scalar_key_type[, scalar_key_type]) => BOOL
- VERTEX_EXISTS(WEIGHTEDPATH, scalar_key_type[, scalar_key_type]) => BOOL

LABEL

Returns the label of a given vertex. If no label was specified for a vertex table in the graph workspace, the name of the table is returned.

- `LABEL (Vertex) => NVARCHAR`

IN_DEGREE

Returns the number of incoming edges of a vertex.

- `IN_DEGREE (VERTEX) => BIGINT`

OUT_DEGREE

Returns the number of outgoing edges of a vertex.

- `OUT_DEGREE (VERTEX) => BIGINT`

DEGREE

Returns the number of incoming and outgoing edges of a vertex.

- `DEGREE (VERTEX) => BIGINT`

EDGES

Returns all the incoming and outgoing edges of a vertex.

- `EDGES (VERTEX) => MULTISSET`

IN_EDGES

Returns all the incoming edges of a vertex.

- `IN_EDGES (VERTEX) => MULTISSET`

OUT_EDGES

Returns all the outgoing edges of a vertex.

- `OUT_EDGES (VERTEX) => MULTISSET`

Vertex Attribute Access

An attribute access expression allows you to retrieve the corresponding attribute value for a given vertex variable and attribute name. The attribute name is an identifier. The return type of the expression is derived from the underlying specified attribute type.

`<attr_access_expr> ::= <variable_reference> . <identifier>`

Examples

```
INTEGER cost = :v.cost;  
NVARCHAR name = :v."NAME";
```

Basic Edge Operations

Constructors

Constructs a [edge](#) reference from a graph or a weighted path, a label and a (composite) key. If multiple edge tables are defined in the graph workspace, it is mandatory to provide a label in the edge constructor. The label comes directly after the graph or path variable and right before the key components (scalar_key_type) in the argument list of the EDGE constructor. If no edge with the given key exists, then the program terminates with an error. Use the EDGE_EXISTS function to determine whether a edge exists.

- `EDGE(GRAPH, NVARCHAR, scalar_key_type[, scalar_key_type]) => EDGE`
- `EDGE(WEIGHTEDPATH, NVARCHAR, scalar_key_type[, scalar_key_type]) => EDGE`

In the case that only a single edge table is defined, the label can be omitted.

- `EDGE(GRAPH, scalar_key_type[, scalar_key_type]) => EDGE`
- `EDGE(WEIGHTEDPATH, scalar_key_type[, scalar_key_type]) => EDGE`

A special case is the construction of an edge from a compatible edge. Edges are compatible if their host graphs are defined in the same graph workspace, e.g., a weighted path, a subgraph and an inversed graph are compatible with the graph they are constructed from, as they all share the same graph workspace.

- `EDGE(GRAPH, EDGE) => EDGE`
- `EDGE(WEIGHTEDPATH, EDGE) => EDGE`

EDGE_EXISTS

Checks if the given edge, or a edge with the given (composite) edge key exists in a graph or a weighted path. If multiple edge tables are defined in the graph workspace, it is mandatory to provide a label for the check by key. The label comes directly after the graph or path variable.

- `EDGE_EXISTS(GRAPH, EDGE) => BOOL`
- `EDGE_EXISTS(GRAPH, NVARCHAR, scalar_key_type[, scalar_key_type]) => BOOL`
- `EDGE_EXISTS(WEIGHTEDPATH, EDGE) => BOOL`
- `EDGE_EXISTS(WEIGHTEDPATH, NVARCHAR, scalar_key_type[, scalar_key_type]) => BOOL`

In the case that only a single edge table is defined, the label can be omitted.

- `EDGE_EXISTS(GRAPH, scalar_key_type[, scalar_key_type]) => BOOL`
- `EDGE_EXISTS(WEIGHTEDPATH, scalar_key_type[, scalar_key_type]) => BOOL`

LABEL

Returns the label of a given edge. If no label was specified for a edge table in the graph workspace, the name of the table is returned.

- LABEL (Edge) => NVARCHAR

SOURCE

Returns the source vertex of an edge.

- SOURCE (EDGE) => VERTEX

TARGET

Returns the target vertex of an edge.

- TARGET (EDGE) => VERTEX

Edge Attribute Access

An attribute access expression allows you to retrieve the corresponding attribute value for a given edge variable and attribute name. The attribute name is an identifier. The return type of the expression is derived from the underlying specified attribute type.

```
<attr_access_expr> ::= <variable_reference> . <identifier>
```

Examples

```
DOUBLE weight = :e.weight;  
BIGINT edgeKey = :e."EDGE_KEY";
```

Basic Weighted Path Operations

The `WEIGHTEDPATH`, like a subgraph, contains copies of the corresponding vertices and edges from the parent graph that form a path. All vertices and edges of a `WEIGHTEDPATH` originate from the parent graph but are different objects. A vertex or edge of a `WEIGHTEDPATH` has all persistent and temporary attributes of a vertex or edge in the parent graph at the time of path creation. Modifications of temporary attributes in the parent graph aren't propagated to the corresponding temporary attributes of a `WEIGHTEDPATH` and the other way around.

(Constructors)

`WEIGHTEDPATH` objects can't be constructed directly but are the result of several built-in graph algorithms.

WEIGHT

Calculates the weight for a `WEIGHTEDPATH`.

- `WEIGHT(WEIGHTEDPATH<BIGINT>) => BIGINT`
- `WEIGHT(WEIGHTEDPATH<INTEGER>) => INTEGER`
- `WEIGHT(WEIGHTEDPATH<DOUBLE>) => DOUBLE`

LENGTH

Calculates the length (number of edges) for a `WEIGHTEDPATH`.

- `LENGTH(WEIGHTEDPATH<BIGINT>) => BIGINT`
- `LENGTH(WEIGHTEDPATH<INTEGER>) => INTEGER`
- `LENGTH(WEIGHTEDPATH<DOUBLE>) => DOUBLE`

VERTICES

Returns all vertices in a path.

- `VERTICES(WEIGHTEDPATH) => SEQUENCE`

EDGES

Returns all edges in a graph.

- `EDGES(WEIGHTEDPATH) => SEQUENCE`

IS_EMPTY

Tests whether the given path is empty, i.e., it has no vertices. The `SHORTEST_PATH` function returns an empty path if no connection is found from start to target vertex.

- `IS_EMPTY(WEIGHTEDPATH<BIGINT>) => BOOL`
- `IS_EMPTY(WEIGHTEDPATH<INTEGER>) => BOOL`
- `IS_EMPTY(WEIGHTEDPATH<DOUBLE>) => BOOL`

Temporary Attributes

Temporary attributes are defined on graphs. Paths and weighted paths are views on the graph they are based on. They inherit all temporary attributes from the graph, and changes to temporary attributes done on the path are visible on the graph and vice versa.

Examples

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH")
  WITH TEMPORARY ATTRIBUTES(VERTEX INTEGER distance = 0, VERTEX DOUBLE price = 0.0);
VERTEX v_source(:g, 'FRA');
VERTEX v_target(:g, 'JFK');
v_target.price = 32.8;
WEIGHTEDPATH<BIGINT> p = SHORTEST_PATH(:g, :v_source, :v_target);
VERTEX p_source(:p, 'FRA');
p_source.distance = 7; -- now also v_source.distance is 7
DOUBLE target_price = :v_target.price; -- is 32.8 as set on the original graph
```

Closures in GraphScript

Certain built-in graph algorithms and graph traversal algorithms can be customized by providing closure expressions, e.g. as weight functions or traversal hooks. A closure expression can either be defined inline or as a named closure using the CLOSURE keyword.

Closure Expression

A closure expression consists of a list of parameters, an optional return type, and a list of statements. The possible combination of parameters and return type depend on the context where the closure is used. A weight function always needs a return type, the type of the weight. A traversal hook cannot have a return type.

The built-in algorithms that make use of closures are documented on this page: [Built-in Graph Algorithms](#).

The graph traversal algorithms are documented on this page: [Graph Traversal Statements](#).

```
<clos_expr> ::= '(' <parameter_list> ')' [ '=>' <return_type> ] '{' <statement_list> '}'
```

Examples

The following example demonstrates the usage of a closure expression as a vertex hook defined inline in a traversal statement:

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
Int sum = 0;
TRAVERSE BFS :g FROM Vertex(:g, 1)
  ON VISIT VERTEX (Vertex v) { sum = :sum + :v."value"; };
```

Named Closures

In addition to defining a closure expression inline in a call of an algorithm, you can also define a closure beforehand and assign it a name. This closure can then be used in a later algorithm call, and it can also be reused for multiple calls. This reduces the need for code duplication and makes the code more readable.

When defining the closure, you need to specify a graph on which the closure will be applied. When using the closure in an algorithm, this algorithm must use the same graph. A named closure is defined with the CLOSURE keyword, followed by the name of the closure, then the graph the closure applies to, and finally the closure expression.

```
<clos_definition> ::= CLOSURE <identifier> '(' <graph_variable>, <clos_expr> ')' ';' ;
```

Examples

In the following example, two named closures are defined, weightFunction and estimateFunction. Afterwards, these are used in a call to the SHORTEST_PATH built-in algorithm.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
CLOSURE weightFunction(:g, (Edge e, Double weight, BigInt hopCnt) => Double {
  IF (:hopCnt + 1L > 3L AND :weight + :e."E_WEIGHT" > 17.0) {
```

```
        END TRAVERSE;
    } ELSE {
        return :e."E_WEIGHT";
    }
});
CLOSURE estimateFunction(:g, (Vertex v1, Vertex v2) => Double {
    return :v2.dst - :v1.dst;
});
WeightedPath<Double> shortestPath = SHORTEST_PATH(:g, Vertex(:g, 1), Vertex(:g, 2), :weightFunction
```

Built-In Graph Algorithms

Betweenness Centrality

Computes the betweenness centrality, which is a measure of centrality based on shortest paths. The betweenness centrality is the number of shortest paths for every pair of vertices that pass through an edge or vertex. Therefore vertices or edges which lie often on shortest paths between other vertices will receive a high betweenness centrality value.

```
<vbc_function> ::= BETWEENNESS_CENTRALITY '(' <graph_variable> ')'
<weighted_vbc_function> ::= BETWEENNESS_CENTRALITY '(' <graph_variable> , <clos_expr> ')'

<ebc_function> ::= BETWEENNESS_EDGE_CENTRALITY '(' <graph_variable> [ , <variable_reference> ] ')'
<weighted_ebc_function> ::= BETWEENNESS_EDGE_CENTRALITY '(' <graph_variable> , <clos_expr> [ , <var:

<graph_variable> ::= <variable_reference>
<vertex_centrality_map> ::= <variable_reference>
```

The betweenness centrality can be calculated for vertices with the function `BETWEENNESS_CENTRALITY` and for edges with the function `BETWEENNESS_EDGE_CENTRALITY`. Both functions return a mapping of vertices or edges to the corresponding centrality value. If no further parameter is specified, they compute the centrality based on unweighted shortest paths.

If an optional weight function is passed to those function the centrality is based on weighted shortest paths. The weight function requires an edge as input parameter and returns the weight as a numeric value. It can be provided as an inline closure expression or as a named closure, see [Closures](#).

The `BETWEENNESS_EDGE_CENTRALITY` function additionally provides an optional map parameter from vertices to centrality values, which also stores vertex centrality values if both are needed.

Examples

The following example demonstrates the usage for unweighted betweenness centrality for either edge or vertex centrality

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");

MAP<VERTEX, DOUBLE> airport_centrality = BETWEENNESS_CENTRALITY(:g);
MAP<EDGE, DOUBLE> valuable_connections = BETWEENNESS_EDGE_CENTRALITY(:g);
```

In addition, the algorithm returns the centrality based on weighted shortest paths if a weight function has been provided.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");

MAP<VERTEX, DOUBLE> airport_centrality(:g, 0L);
MAP<EDGE, DOUBLE> valuable_connections = BETWEENNESS_EDGE_CENTRALITY(
    :g, :airport_centrality);
```

If the centrality of vertices and edges is needed, both can be computed in one step by passing a reference to a vertex centrality map as the last argument in the `BETWEENNESS_EDGE_CENTRALITY` call.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");

MAP<VERTEX, DOUBLE> airport_centrality(:g, 0L);
MAP<EDGE, DOUBLE> valuable_connections = BETWEENNESS_EDGE_CENTRALITY(
    :g, (EDGE e) => INTEGER{ RETURN :e."DIST_KM"; }, :airport_centrality);
```

```
MAP<VERTEX, DOUBLE> weighted_airport_centrality(:g, 0L);
MAP<EDGE, DOUBLE> weighted_valuable_connections = BETWEENNESS_EDGE_CENTRALITY(
    :g, :weighted_airport_centrality);
```

Community detection

```
<community_function> ::= COMMUNITIES_LOUVAIN('(<graph>[,<runs>[,<weight>[,<metric>[,<epsilon>]]]]')
```

Returns a SEQUENCE of MULTISSET of VERTEX, representing partitioning of the vertices of a graph in communities. Each multiset holds the vertices that belong to the same community. A community is a group of vertices which are better connected to each other as compared to the vertices outside the community. In other words, vertices are densely connected within a community and sparsely connected with the vertices of other communities. The Louvain method, see [Reference 1](#), finds communities by optimizing the modularity metric (modularity is the only supported metric currently) locally for every vertex's neighborhood iteratively. It ends when the metric does not improve further.

As the order in which the vertices are processed is random, the result is not deterministic and can change between each function call. To get better results and to avoid local minima, this function allows to specify a number of runs, where a run is a complete execution of the Louvain method until a result has been found that can not be further optimized. After all the runs have been completed, the function picks the best result and returns it. Metric specific limitations like the resolution limit, see [Reference 2](#), for modularity can not be overcome this way.

The optional weight function can be provided as an inline closure or as a named closure, see [Closures](#).

Parameters

- **graph**: Graph to apply the community detection on
- **runs**: Number of Louvain runs from which the best result is picked later on (Default: 1)
- **weight**: Function that provides an edge weight for each edge in the graph (Default: (Edge e) => DOUBLE{ return 1.0; })
- **metric**: Community metric that the Louvain algorithm uses to optimize communities (Default: 'MODULARITY')
- **epsilon**: Minimum change in the metric value between iterations.

Examples

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
SEQUENCE<MULTISSET<VERTEX>> communities = COMMUNITIES_LOUVAIN(:g, 10, (Edge e) => DOUBLE{ return 1.0; })
```

Besides finding pure communities, it is desirable to work with the newly found communities. Such a workflow could be structured:

1. Find communities
2. Iterate over the communities
 - a. Create a subgraph of the current community
 - b. Apply graph algorithm to the subgraph
3. Transform the communities to maps

4. Project the result into a map

The related GraphScript code could look like the following:

```
GRAPH g = ...
SEQUENCE<MULTISET<VERTEX>> communities = COMMUNITIES_LOUVAIN(:g, 5);

-- Community based calculations
FOREACH community in :communities
{
    Graph communityGraph = SUBGRAPH(:g, :community);
    ""
}

-- Result projection
MAP<VERTEX, BIGINT> communityMap = TO_ORDINALITY_MAP(:communities);
myTable = SELECT :v."KEY", :communityMap [:v] FOREACH v in VERTICES(:g);
```

References

1. Blondel, Vincent D., et al. "Fast unfolding of communities in large networks." Journal of statistical mechanics: theory and experiment 2008.10 (2008): P10008.
2. Fortunato, Santo, and Marc Barthelemy. "Resolution limit in community detection." Proceedings of the national academy of sciences 104.1 (2007): 36-41.

Modularity

```
<community_function> ::= MODULARITY '(' <graph>,<communities>[,< weight>]')'
```

This function computes the modularity of communities that partition a graph. The return value is **DOUBLE**. Modularity is a quality metric for communities that quantifies if there are many edges inside communities and fewer between them.

The optional weight function can be provided as an inline closure or as a named closure, see [Closures](#).

Parameter

- **graph**: Graph where the community is derived from
- **communities**: Communities that partition <graph> in the form of SEQUENCE<MULTISET<VERTEX>>
- **weight**: Function that provide an edge weight for each edge in the graph (Default: (Edge e) => DOUBLE{ return 1.0; })

Examples

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");

SEQUENCE<MULTISET<VERTEX>> communities = [...]
DOUBLE metric = MODULARITY(:g, :communities, (Edge e) => DOUBLE{ return 1.0; })
```

Edges Between

Returns all edges between a start set or vertex and a target set or vertex in a graph.

- `EDGES (GRAPH, MULTISSET, MULTISSET) => MULTISSET`
- `EDGES (GRAPH, MULTISSET, VERTEX) => MULTISSET`
- `EDGES (GRAPH, VERTEX, MULTISSET) => MULTISSET`
- `EDGES (GRAPH, VERTEX, VERTEX) => MULTISSET`
- `EDGES (GRAPH, SEQUENCE, SEQUENCE) => MULTISSET`
- `EDGES (GRAPH, SEQUENCE, MULTISSET) => MULTISSET`
- `EDGES (GRAPH, MULTISSET, SEQUENCE) => MULTISSET`
- `EDGES (GRAPH, SEQUENCE, VERTEX) => MULTISSET`
- `EDGES (GRAPH, VERTEX, SEQUENCE) => MULTISSET`

Is Reachable

Tests for the existence of at least one path from vertex v1 to vertex v2.

- `IS_REACHABLE (GRAPH, VERTEX, VERTEX) => BOOL`

Reachable Vertices

Returns all vertices that are reachable from the given source vertex or vertices.

The result set will include the source vertices.

- `REACHABLE Vertices (Graph, Vertex, Nvarchar) => MULTISSET<VERTEX>`
- `REACHABLE Vertices (Graph, MultiSet<Vertex>, Nvarchar) => MULTISSET<VERTEX>`
- `REACHABLE Vertices (Graph, Sequence<Vertex>, Nvarchar) => MULTISSET<VERTEX>`

The first argument is the graph to process. The second argument defines the start vertex/vertices. The third argument selects the edges which the algorithm follows when it finds connected vertices.

The following values are possible:

- 'OUTGOING' traverses the edges as given in the input graph.
- 'INCOMING' traverses the edges of the input graph in reverse direction.
- 'ANY' traverses the input graph as if it was undirected.

Inverse Graph

Constructs the inverse graph of a given graph.

```
<inversegraph_function> ::= INVERSEGRAPH '(' <parent_graph_variable> ')'  
<parent_graph_variable> ::= <variable_reference>
```

Returns the inverse graph of the given parent graph. All vertices of the parent graph are copied into the resulting inverse graph without any modification. All edges of the parent graph are copied into the resulting inverse graph with swapped edge direction. The values for all vertices as well as edge attributes are copied from the parent graph. Attribute handling is identical to subgraphs.

On inverse graphs, the `SOURCE (:e)` and `TARGET (:e)` functions return the inverse result, whereas accessing the edge's source and target key attributes returns the original values.

Examples

In the following example, we manually construct a multiset `neighborsFRA` of all direct neighbors of `FRA` independently of the edge direction using `INVERSEGRAPH`. Note that we use a `FOREACH` loop to get the corresponding vertices in Graph `G` by key, as using the `UNION` operation for multisets from different graphs isn't permitted.

This example is an artificial one for the sake of demonstrating the `INVERSEGRAPH` function. The same result could be achieved much more efficiently by calling the `NEIGHBORS` function with a distance window from -1 to 1.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
MULTISET<VERTEX> neighborsFRA = NEIGHBORS(:g, VERTEX(:g, 'FRA'), 1, 1);
GRAPH gi = INVERSEGRAPH(:g);
FOREACH v IN NEIGHBORS(:gi, { VERTEX(:gi, 'FRA') }, 1, 1) {
    neighborsFRA = :neighborsFRA UNION { VERTEX(:g, :v."NODE_KEY") };
}
```

Subgraph

Constructs a subgraph of a graph based on a set of vertices or edges.

```
<subgraph_function> ::= SUBGRAPH '(' <parent_graph_variable>, <container_of_vertices> ')'  
<parent_graph_variable> ::= <variable_reference>  
<container_of_vertices> ::= <expr>
```

Returns the subgraph of the given parent graph induced by the given container of vertices. The subgraph contains a copy of all vertices in the input vertex container as well as all edges of the parent graph whose source and target are both in the given vertex container.

```
<subgraph_function> ::= SUBGRAPH '(' <parent_graph_variable>, <container_of_edges> ')'  
<parent_graph_variable> ::= <variable_reference>  
<container_of_edges> ::= <expr>
```

Returns the subgraph of the given parent graph induced by the given container of edges. The subgraph contains the given edges as well as all source and target vertices of the given edge container.

A subgraph is a view on the parent graph, so all vertices and edges of a subgraph are the same than the vertices and edges of the parent graph. Modifications of temporary attributes in the parent graph are visible on the subgraph and vice versa.

Examples

The following example demonstrates the usage of the subgraph function in combination with temporary attributes. The temporary attribute `continent` is defined on graph `g_flights` before deriving the subgraph `g_EuropeanFlights` from `g_flights` and can also be accessed for a vertex of `g_EuropeanFlights`.

```

GRAPH g_flights = GRAPH("FLIGHTS", "GRAPH")
  WITH TEMPORARY ATTRIBUTES(VERTEX NVARCHAR continent);
-- read continents of nodes from input table...
GRAPH g_EuropeanFlights = SUBGRAPH(
  :g_flights, v IN VERTICES(:g_flights) WHERE :v.continent == 'Europe');

VERTEX v_amsterdamEUR(:g_EuropeanFlights, 'AMS');
v_amsterdamEUR.continent = 'Europe';

VERTEX v_amsterdam(:g_flights, 'AMS');
NVARCHAR cnt = :v_amsterdam.continent;
--> cnt is 'Europe'

```

Neighbors

Explores the vicinity of a given set of start vertices with a distance window and optional edge direction.

```

<neighbors_function> ::=
  NEIGHBORS '(' <parent_graph_variable>, <start_vertices>,
    <lower_bound>, <upper_bound> [, <edge_direction>] ')'

<parent_graph_variable> ::= <variable_reference>
<start_vertices> ::= <expr>
<lower_bound> ::= <expr>
<upper_bound> ::= <expr>
<edge_direction> ::= <expr>

```

Start vertices can be either a single vertex expression, or expressions returning a multiset or sequence of vertices.

In the following section, we define the distance of a vertex V to the start_vertices set as the length (counted in hops) of the shortest of all paths from any member of the start_vertices set to V.

For non-negative lower_bound and upper_bound values, the NEIGHBORS function returns any vertex whose distance to the start_vertices set isn't smaller than the lower_bound value and not larger than the upper_bound value.

The optional edge direction parameter can be used to specify which edges the algorithm traverses when evaluating the distance of a vertex to the start_vertices set. The following values are possible:

'OUTGOING' traverses the edges as given in the input graph. 'INCOMING' traverses the edges of the input graph in reverse direction. 'ANY' traverses the input graph as if it was undirected.

When the optional direction parameter isn't used, negative lower_bound and/or upper_bound values compute the result as described following:

- If both lower_bound and upper_bound are negative, that is, NEIGHBORS(:g, :v, -a, -b) is called, the function computes the distance processing the edges in reverse direction and adjusted interval borders: NEIGHBORS(:g, :v, -a, -b) returns the same result as NEIGHBORS(:g, :v, b, a, 'INCOMING').
- If lower_bound is negative and upper_bound is non-negative, that is, NEIGHBORS(:g, :v, -a, b) is called, then edges are traversed in both outgoing direction (up to b hops) and in incoming direction (up to a hops). NEIGHBORS(:g, :v, -a, b) returns NEIGHBORS(:g, :v, a, 0) UNION NEIGHBORS(:g, :v, 1, b).

The computation performed in the example of the INVERSEGRAPH function can be rewritten using a single NEIGHBORS call with a corresponding distance range:

```

GRAPH g = GRAPH("FLIGHTS", "GRAPH");
MULTISET<VERTEX> neighborsFrankfurt = NEIGHBORS(:g, [ VERTEX(:g, 'FRA') ], -1, 1);

```

If the direction parameter is used, negative bound values are treated as 0. Any bounds interval for which the upper bound is smaller than the lower bound returns an empty result.

Page Rank

Computes the score of vertices based on the incoming edges and the scores of the corresponding source vertices. The algorithm was initially introduced by Google for weighting the importance of websites. The underlying assumption is that more important sites receive more links from other pages.

The theory is based on an theoretical person randomly browsing through the sites by clicking links. At any step, the person has a propability d to continue also called the damping factor. The resulting scores form a propability distribution representing the likelihood of the person to arrive a particular page v , denoted as $PR(v)$. Furthermore, if a page has no outgoing links, the algorithm assumes that the surfer starts randomly at any other page.

The resulting formula can be written as:

$$PR(v) = \frac{1 - d}{|V|} + d \sum_{u \in N(v)} \frac{PR(u)}{L(u)}$$

where $N(v)$ is the set of vertices which link to v and $L(u)$ the number of outgoing edges of vertex u .

The underlying implementation uses this formula to iteratively update a candidate solution until the iteration limit has been reached or all the values converged. A value is converged if the update does not deviate by a certain amount epsilon during an iteration.

```
<page_function> ::= PAGE_RANK '(' <graph_variable>. [, <damping_factor> [ , <iteration_limit>
[ , <epsilon> ] ] ] ')'
```

```
<graph_variable> ::= <variable_reference>
<damping_factor> ::= <double_constant> | <expr>
<iteration_limit> ::= <integer_constant> | <expr>
<epsilon> ::= <double_constant> | <expr>
```

Example

The following example demonstrates the usage of the page rank function.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");

MAP<VERTEX, DOUBLE> default_rank = PAGE_RANK(:g);
MAP<VERTEX, DOUBLE> custom_rank = PAGE_RANK(:g, 0.20, 1000, 0.00001);
```

The defaults for the optional parameters are:

Parameter	Type	Default
damping factor	DOUBLE	0.85
iteration limit	INT	100

Parameter	Type	Default
epsilon	DOUBLE	10 ⁻⁸

Shortest Path

A common operation on graphs is the calculation of a shortest path from a source vertex to a target vertex. The built-in function `SHORTEST_PATH` returns a [WEIGHTEDPATH](#) which contains a shortest path in a given graph between source and target. The resulting path is empty if there exists no path in the given graph. The built-in function `IS_EMPTY` checks if a path is empty.

There are three major forms of the `SHORTEST_PATH` function:

1. `SHORTEST_PATH( GRAPH g, VERTEX source, VERTEX target [, NVARCHAR direction] ) => WEIGHTEDPATH<BIGINT>`
2. `SHORTEST_PATH( GRAPH g, VERTEX source, VERTEX target, CLOSURE weight [, NVARCHAR direction] ) => WEIGHTEDPATH`
3. `SHORTEST_PATH( GRAPH g, VERTEX source, VERTEX target, CLOSURE weight, CLOSURE estimate [, NVARCHAR direction] ) => WEIGHTEDPATH`

The first form calculates a shortest path on an unweighted graph. Every edge has the same weight of 1. The weight of a path is the number of edges inside the path (hop distance). The return value is a `WEIGHTEDPATH` with a weight type of `BIGINT`.

Example:

```
WEIGHTEDPATH<BIGINT> path = SHORTEST_PATH(:mygraph, :source_vertex, :target_vertex);
```

The second form calculates a shortest path on a weighted graph with non-negative weights. The 4th parameter defines a weight function as closure expression which assigns a weight to every visited edge. The weight of a path is the sum of all edge weights inside the path. The weight type is defined by the return type of the weight function. The resulting `WEIGHTEDPATH` uses the same weight type. The weight type must be one of the following types: `INTEGER`, `BIGINT` or `DOUBLE`. The value of the returned weight must be non-negative.

Example:

```
WEIGHTEDPATH<Double> path = SHORTEST_PATH(
    :mygraph, :source_vertex, :target_vertex,
    (Edge e) => Double {
        RETURN :e.weight;
    }
);
```

The third form is a variation of the second. It requires an additional estimate function as 5th parameter to guide the search in the correct direction. This prunes the search space and leads to faster execution. The estimate function takes two vertices as parameters and estimates the weight of the shortest path between them. The estimation is not allowed to overestimate the actual weight. The return type is the same as the weight type defined in the weight function (4th parameter).

Example:

```
WEIGHTEDPATH<Double> path = SHORTEST_PATH(
    :mygraph, :source_vertex, :target_vertex,
    (Edge e) => Double {
        RETURN :e.length;
    },
```

```

(Vertex a, Vertex b) => Double {
  -- euclidean distance
  Integer distX = :a.x - :b.x;
  Integer distY = :a.y - :b.y;
  Double dist = SQRT( Double(:distX * :distX + :distY * :distY) );
  RETURN :dist;
}
);

```

All three forms have an optional last parameter, the traversal direction. It defines which edges are eligible in the search. Three values are accepted: 'OUTGOING' (default), 'INCOMING' and 'ANY'. OUTGOING only follows an edge in the direction defined in the graph, from the source of the edge to its target. INCOMING does the opposite and conceptually reverses every edge in the graph. It walks an edge from its target to its source. ANY follows edges in either direction. Conceptually, it treats the edges as undirected.

Weight Function

When searching for shortest paths on a weighted graph (forms 2 and 3), every visited edge must have an associated weight. To calculate the weight, it is required to provide a weight function as closure expression in the 4th parameter.

The weight function must take an edge as first parameter and return the weight of this edge. The weight type must be one of the following types: INTEGER, BIGINT or DOUBLE. The value of the weight must be non-negative.

Next to the mandatory edge parameter, two optional parameters can be requested from the built-in function, the accumulated weight and hop distance.

The accumulated weight must be the second parameter and have the same type as the return value, the weight type. It contains the weight of the shortest path from the source vertex to the current edge (first parameter).

The hop distance must be the third parameter and is of type BIGINT. It describes the number of edges required to reach the current edge.

One common use-case for these optional parameters are depth-limited searches which prune the search space when a depth limit is crossed. This can be easily implemented by comparing one of the optional parameters with the limit and calling `END TRAVERSE` in case of going past the limit. When `END TRAVERSE` is called, the execution of the weight function is aborted and the current edge is discarded from the search. Other open paths are still explored. The `END TRAVERSE ALL` statement is not allowed.

As a closure expression, the weight function can read and modify variables visible in the outer scope.

The weight function can be provided as an inline closure or as a named closure, see [Closures](#).

Estimate Function

In form 3, the 5th parameter is a second closure expression, the estimate function. The function must take two vertices as parameters and return an estimated weight of the shortest path between these two vertices. The return type must be the weight type as defined in the weight function.

There are additional constraints on the estimation. The value must be non-negative and never overestimate the actual weight of the shortest path, if a path between the vertices exists.

The `END TRAVERSE` and `END TRAVERSE ALL` statements are not allowed in the estimate function.

Similarly to the weight function, the estimate function can read and modify variables visible in the outer scope.

The estimate function can be provided as an inline closure or as a named closure, see [Closures](#).

Examples

The following example demonstrates the usage of the `SHORTEST_PATH` function. For a more in-depth explanation of the semantics for temporary attributes of vertices and edges in a path, see the previous subgraph example.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH")
  WITH TEMPORARY ATTRIBUTES(VERTEX INTEGER vertexAttr = 0, EDGE INTEGER edgeAttr = 42);
VERTEX v_source(:g, 'FRA');
VERTEX v_target(:g, 'JFK');

WEIGHTEDPATH<BIGINT> p = SHORTEST_PATH(:g, :v_source, :v_target);

SEQUENCE<VERTEX> sv = VERTICES(:p);
SEQUENCE<EDGE> se = EDGES(:p);

BIGINT lengthOfP = LENGTH(:p);
BIGINT weightOfP = WEIGHT(:p);

-- Get first edge of path
EDGE e = :se[1L];
-- read edge attribute value --> 42
INTEGER edgeAttrValue = :e.edgeAttr;
```

The following example demonstrates the use of custom weight functions of various types. The first path is constructed using a weight function depending on a temporary edge attribute, the second based on a constant, and the third one based on a non-temporary edge attribute.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH")
  WITH TEMPORARY ATTRIBUTES(EDGE INTEGER price = 1);
VERTEX v_source(:g, 'FRA');
VERTEX v_target(:g, 'JFK');

WEIGHTEDPATH<INTEGER> cheapest_path = SHORTEST_PATH(
  :g, :v_source, :v_target,
  (EDGE e) => INTEGER {
    RETURN :e.price;
  }
);

WEIGHTEDPATH<BIGINT> least_stopovers = SHORTEST_PATH(
  :g, :v_source, :v_target, (EDGE e) => BIGINT { RETURN 1L; });

WEIGHTEDPATH<DOUBLE> shortest = SHORTEST_PATH(
  :g, :v_source, :v_target, (EDGE e) => DOUBLE { RETURN :e."DIST_KM"; });
```

The following example demonstrates the use of the `END TRAVERSE` statement to apply a weight limit on the path. If an edge crosses the limit, it gets pruned from the search by calling `END TRAVERSE` in the weight function. In the `IF` condition, the optional second parameter of the weight function is used, which contains the accumulated weight to reach the current edge. `SHORTEST_PATH` returns an empty path if no path below the weight limit exists.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
VERTEX v_source(:g, 'FRA');
VERTEX v_target(:g, 'JFK');
INTEGER maximal_price = 500;

CLOSURE weightFct(:g, (EDGE e, INTEGER w) => INTEGER {
  IF ((:w + :e.price) < :maximal_price) {
    RETURN :e.price;
  } ELSE {
    END TRAVERSE;
  }
});

WEIGHTEDPATH<INTEGER> cheapest_path = SHORTEST_PATH(:g, :v_source, :v_target, :weightFct);
```

The following example demonstrates a limit on both weight and hop distance. In the IF condition, the optional second and third parameters of the weight function is used, which contain the accumulated weight and the hop distance to reach the current edge. `SHORTEST_PATH` returns an empty path if no path below the weight and hop distance limit exists.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
VERTEX v_source(:g, 'FRA');
VERTEX v_target(:g, 'JFK');

CLOSURE weightFct(:g, (EDGE e, INTEGER w, BIGINT h) => INTEGER {
  IF (:w + :e.price < 500 AND :h + 1L < 2L) {
    RETURN :e.price;
  } ELSE {
    END TRAVERSE;
  }
});

WEIGHTEDPATH<INTEGER> cheapest_and_shortest_path = SHORTEST_PATH(:g, :v_source, :v_target, :weightFct);
```

The following example demonstrates how to use the edge direction parameter to let the `SHORTEST_PATH` function work as if it was operating on a graph with undirected edges.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
VERTEX v_source(:g, 'FRA');
VERTEX v_target(:g, 'JFK');

WEIGHTEDPATH<BIGINT> p = SHORTEST_PATH(:g, :v_source, :v_target, 'ANY');
```

Shortest Paths One-to-All

```
<spoa_function> ::= SHORTEST_PATHS_ONE_TO_ALL '(' <parent_graph_variable>,
  <source_vertex>, <distance_attribute_name> [ , <clos_expr> ] [ , <edge_direction> ] ')'
```

<parent_graph_variable> ::= <variable_reference>
 <start_vertex> ::= <expr>
 <distance_attribute_name> ::= <identifier>
 <edge_direction> ::= <expr>

Returns a subgraph of the given parent graph (first parameter) built from the shortest paths from the start vertex (second parameter) to all other reachable vertices.

It contains all the attributes from the parent graph and an additional vertex attribute (third parameter) containing the distance from the start vertex to each reachable vertex. The type of the distance attribute is either `BIGINT` or the return type of the optional weight function. By default, the weight metric of the `SHORTEST_PATHS_ONE_TO_ALL` built-in function is the hop distance, that is, the number of edges on the path, but it's also possible to supply a custom weight function (optional fourth parameter). For details on the weight function, see the Shortest Path section. It can be provided as an inline closure or as a named closure, see [Closures](#).

The optional `edge_direction` parameter can be used to control how the direction of the edges of the input graph is treated by the shortest path search. By default, the algorithm processes edges according to their original direction as given in the input graph. For possible values and discussion of the edge direction parameter, see the Shortest Path section.

The resulting subgraph is a tree, that is, a directed, acyclic graph where each vertex has only one incoming edge, except for the start vertex, which has no incoming edge.

Examples

The following example demonstrates the usage of the `SHORTEST_PATHS_ONE_TO_ALL` function.

```

GRAPH g = GRAPH("FLIGHTS", "GRAPH");
VERTEX dubai(:g, 'DXB');
GRAPH all_dubai_flights = SHORTEST_PATHS_ONE_TO_ALL(:g, :dubai, "hop_distance_from_Dubai");

```

The following example demonstrates the use of custom weight functions of various types. The first graph is constructed using a weight function depending on a temporary edge attribute. The second graph uses a fixed constant weight. The third graph is based on a non-temporary edge attribute.

```

GRAPH G = GRAPH("FLIGHTS", "GRAPH")
  WITH TEMPORARY ATTRIBUTES(EDGE INTEGER costs = 1);
VERTEX v_source(:G, 'FRA');
GRAPH cheapest = SHORTEST_PATHS_ONE_TO_ALL(
  :g, :v_source, "total_costs", (EDGE e) => INTEGER { RETURN :e.costs; });
GRAPH shortest_hop = SHORTEST_PATHS_ONE_TO_ALL(
  :g, :v_source, "hop_distance", (EDGE e) => BIGINT{ RETURN 1L; });
GRAPH shortest_total = SHORTEST_PATHS_ONE_TO_ALL(
  :g, :v_source, "total_distance", (EDGE e) => DOUBLE{ RETURN :e."DIST_KM"; });

```

The following example demonstrates the use of weight and hop distance bound inside a weight function. The IF clause adds the bounds conditions and returns the subgraph with shortest paths to all reachable vertices, where every path has a distance of less than 1000 kms and stopovers are less than 2.

```

GRAPH G = GRAPH("FLIGHTS", "GRAPH");
VERTEX v_source(:G, 'FRA');
GRAPH bound_hierarchy = SHORTEST_PATHS_ONE_TO_ALL(
  :g, :v_source, "total_distance", (EDGE e, DOUBLE w, BIGINT h) =>
    DOUBLE{ IF(:w + :e."DIST_KM" < 1000.0 AND :h + 1L < 2L){RETURN :e."DIST_KM";}
    ELSE{END TRAVERSE;}});

```

The following example demonstrates how to use the edge direction parameter to let the SHORTEST_PATHS_ONE_TO_ALL function work as if it was operating on a graph with undirected edges.

```

GRAPH g = GRAPH("FLIGHTS", "GRAPH");
VERTEX v_source(:g, 'AMS');
GRAPH any_hierarchy = SHORTEST_PATHS_ONE_TO_ALL(:g, :v_source, "hop_distance", 'ANY');

```

K-shortest Paths

```

<ksp_function> ::= K_SHORTEST_PATHS '(' <parent_graph_variable>,
  <source_vertex>, <target_vertex>, <num_paths> [ , <clos_expr> ] ')'

```

```

<parent_graph_variable> ::= <variable_reference>
<start_vertex> ::= <expr>
<target_vertex> ::= <expr>
<num_paths> ::= <expr>

```

Returns a sequence of [WEIGHTEDPATH](#) instances containing the num_paths (fourth parameter) shortest paths within the given parent graph from a start vertex (second parameter) to a target vertex (third parameter). By default, the weight metric of the K_SHORTEST_PATHS built-in function is the hop distance, but it's also possible to supply a custom weight function (optional fourth parameter). See [Shortest Path](#) for details on the weight function. The function creates paths that might contain cycles. The paths in the sequence are ordered by increasing length.

The optional weight function can be provided as an inline closure or as a named closure, see [Closures](#).

Please note that K_SHORTEST_PATHS doesn't support the weight and hop distance parameters inside the weight function. Furthermore, the END TRAVERSE statement is not supported.

Examples

The following example demonstrates the usage of the K_SHORTEST_PATHS function.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
VERTEX v_source(:g, 'FRA');
VERTEX v_target(:g, 'JFK');
SEQUENCE<WEIGHTEDPATH<BIGINT>> paths = K_SHORTEST_PATHS(
    :g, :v_source, :v_target, 3);

FOREACH p IN :paths {
    -- process the path
    BIGINT l = LENGTH(:p);
    BIGINT w = WEIGHT(:p);
    FOREACH v in VERTICES(:p) {
        -- ....
    }
}
```

The following example demonstrates the use of a custom weight function of various types.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH")
    WITH TEMPORARY ATTRIBUTES(EDGE INTEGER price = 1);
VERTEX v_source(:g, 'FRA');
VERTEX v_target(:g, 'JFK');
SEQUENCE<WEIGHTEDPATH<INTEGER>> cheapest_paths = K_SHORTEST_PATHS(
    :g, :v_source, :v_target, 3, (EDGE e) => INTEGER{ RETURN :e.price; });
SEQUENCE<WEIGHTEDPATH<BIGINT>> least_stopovers = K_SHORTEST_PATHS(
    :g, :v_source, :v_target, 3, (EDGE e) => BIGINT{ RETURN 1L; });
SEQUENCE<WEIGHTEDPATH<DOUBLE>> shortest_paths = K_SHORTEST_PATHS(
    :g, :v_source, :v_target, 3, (EDGE e) => DOUBLE{ RETURN :e."DIST_KM"; });
```

Max Flow

The built-in MAX_FLOW function computes the maximum flow possible from a source vertex to a sink vertex. It returns a MAP, which contains the final flow through each edge in a flow network needed to achieve maximum flow.

```
<mf_function> ::= MAX_FLOW '(' <parent_graph_variable>,
    <source_vertex>, <sink_vertex>, [ , <clos_expr> ] [ , <flow_value>] [ , <flow_paths>] ')'

<parent_graph_variable> ::= <variable_reference>
<start_vertex> ::= <expr>
<sink_vertex> ::= <expr>
<flow_value> ::= <variable_reference>
<flow_paths> ::= <variable_reference>
```

Various forms of MAX_FLOW function are possible with several optional parameters, which are listed in next section:

- MAX_FLOW(GRAPH, VERTEX, VERTEX) => MAP
- MAX_FLOW(GRAPH, VERTEX, VERTEX, SCALAR) => MAP
- MAX_FLOW(GRAPH, VERTEX, VERTEX, SEQUENCE) => MAP
- MAX_FLOW(GRAPH, VERTEX, VERTEX, SCALAR, SEQUENCE) => MAP
- MAX_FLOW(GRAPH, VERTEX, VERTEX, CLOSURE) => MAP
- MAX_FLOW(GRAPH, VERTEX, VERTEX, CLOSURE, SCALAR) => MAP
- MAX_FLOW(GRAPH, VERTEX, VERTEX, CLOSURE, SEQUENCE) => MAP

- `MAX_FLOW(GRAPH, VERTEX, VERTEX, CLOSURE, SCALAR, SEQUENCE) => MAP`

Capacity Function

When searching for maximum flow, every visited edge must have an associated capacity. To calculate the capacity, it is required to provide a capacity function as closure expression in the 4th parameter.

The capacity function must take an edge as parameter and return the capacity of this edge. The capacity type must be one of the following types: `INTEGER`, `BIGINT` or `DOUBLE`. The value of the capacity must be non-negative. The capacity function can be provided as an inline closure or as a named closure, see [Closures](#).

Please note that `MAX_FLOW` does not support any other optional arguments inside the capacity function. If no capacity function is specified, then every edge is assumed to have a capacity of 1 and type `BIGINT`.

Flow Value

To obtain the maximum flow value from the source to the sink, a scalar variable of type `INTEGER`, `BIGINT`, or `DOUBLE` can be provided to the function to store the result. This variable must be declared beforehand and passed to the function as the fourth parameter if no capacity function is defined, or as the fifth parameter if a capacity function is defined.

Flow Paths

To obtain the paths that constitute the maximum flow, a sequence of [WEIGHTEDPATH](#) variable can also be provided to the function to store the result. This variable must be declared beforehand and passed to the function as the fourth parameter if no capacity function is defined, or as the fifth parameter if a capacity function is defined. If both flow value and flow paths are needed, the flow value variable should be provided first, followed by the variable for flow paths. The resulting flow paths are sorted in descending order of flow, with the fattest path, i.e., the path with the highest flow appearing first in the sequence of `WEIGHTEDPATH`.

Please note that when `WEIGHT` function is executed on a `WEIGHTEDPATH` from maximum flow, it returns the flow through this `WEIGHTEDPATH` and not the cumulative of the flow of edges in this `WEIGHTEDPATH`.

Examples

The following example demonstrates the usage of the `MAX_FLOW` function.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
VERTEX v_source(:g, 'FRA');
VERTEX v_sink(:g, 'JFK');

MAP<EDGE, BIGINT>> flow_map = MAX_FLOW(:g, :v_source, :v_sink);
```

The following example demonstrates the usage of the `MAX_FLOW` function with all optional parameters.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
VERTEX v_source(:g, 'FRA');
VERTEX v_sink(:g, 'DEL');

INTEGER flow_value = 0;
SEQUENCE<WEIGHTEDPATH<INTEGER>> flow_paths(:g);

CLOSURE capacityFct(:g, (EDGE e) => INTEGER {
```

```

    IF (:e."DIST_KM" < 7000.0) {
        RETURN :e."PASSENGER_CAPACITY";
    } ELSE {
        END TRAVERSE;
    }
});

```

```

MAP<EDGE, BIGINT>> flow_map = MAX_FLOW(:g, :v_source, :v_sink, :capacityFct, :flow_value, :flow_pa

FOREACH p IN :flow_paths {
    -- process the path
    BIGINT l = LENGTH(:p);
    BIGINT w = WEIGHT(:p);
    FOREACH v in VERTICES(:p) {
        -- ....
    }
}

```

Strongly Connected Components

```

<scc_function> ::= STRONGLY_CONNECTED_COMPONENTS '(' <parent_graph_variable> ')'
<parent_graph_variable> ::= <variable_reference>

```

Returns a sequence of sequences of vertices, representing the strongly connected components in the parent graph. The foreach loop statement can be used to iterate over the result and access the individual components. Each strongly connected component is represented as a sequence of vertices and can be used in the same way as any other sequence of vertices.

Examples

The following example demonstrates the usage of the strongly connected components function. The result of the function is a container of containers. The built-in function COUNT can be used to determine the total number of strongly connected components in the result. One can iterate over the result and use a temporary vertex attribute to store the component number to which the vertex belongs.

```

GRAPH g = GRAPH("FLIGHTS", "GRAPH")
WITH TEMPORARY ATTRIBUTES(VERTEX INTEGER componentNumber);
SEQUENCE<SEQUENCE<VERTEX>> sccResult = STRONGLY_CONNECTED_COMPONENTS(:g);

BIGINT numComponents = COUNT(:sccResult);
INTEGER componentCounter = 0;
FOREACH component IN :sccResult {
    componentCounter = :componentCounter + 1;
    FOREACH v IN :component {
        v.componentNumber = :componentCounter;
    }
}

```

Graph Traversal Statements

Many advanced graph algorithms are built on top of basic graph algorithms, extending them with a bit of custom code at specific points during the execution. For this exact purpose, GraphScript provides the TRAVERSE statement. It is an expressive way to describe algorithm variations with a set of well-defined extension points. Basic traversal algorithms like breadth-first search (BFS), depth-first search (DFS) and Dijkstra's algorithm can be extended with so-called hook functions. They are executed when specific events happen during the traversal, for example, a vertex or an edge is visited for the first time.

The following example illustrates the basic structure of the statement. It calculates the vertex preorder and postorder in a DFS started from a single vertex.

```
GRAPH g = GRAPH("EXAMPLE", "GRAPH")
    WITH TEMPORARY ATTRIBUTES (
        VERTEX INTEGER preorder = -1,
        VERTEX INTEGER postorder = -1
    );
INTEGER pre = 1;
INTEGER post = 1;
VERTEX startVertex(:g, 1);

TRAVERSE DFS :g FROM :startVertex
    ON VISIT VERTEX (VERTEX v) {
        v.preorder = :pre;
        pre = :pre + 1;
    }
    ON EXIT VERTEX (VERTEX v) {
        v.postorder = :post;
        post = :post + 1;
    };
```

First, there is some setup code to declare a graph with two temporary vertex attributes and two counters for the preorder and postorder. The TRAVERSE keyword starts the traversal statement, followed by a specifier identifying the basic traversal algorithm, DFS in this case. The supported specifiers are BFS, DFS and DIJKSTRA. Next, the graph is referenced on which the traversal will run. The header line is completed by the keyword FROM and the start vertex description, which accepts any expression resulting in a vertex of the referenced graph.

After the header follows a list of clauses defining the extensions to the basic traversal. Here, two hook functions are defined: one for the preorder calculation using ON VISIT VERTEX and a second one for the postorder calculation using ON EXIT VERTEX. The VISIT VERTEX hook is executed when a vertex is reached the first time. Similarly, the EXIT VERTEX hook is executed right before the DFS backtracks from the vertex. ON EXIT VERTEX is only available in DFS as the other algorithms do not backtrack.

The vertex hook functions can be provided either as inline closure expression or as a named closure, see [Closures](#).

The next sections describe the available extension points for each supported traversal algorithm.

Breadth-First Search (BFS)

BFS visits adjacent edges and vertices first before stepping deeper into the graph. It supports two hook functions: ON VISIT VERTEX and ON VISIT EDGE which are executed when the elements are visited the first time.

Next to a single start vertex, BFS also supports multiple start vertices in form of a Multiset. Any expression resulting in a Multiset of vertices is permitted, including ad hoc definitions with curly braces shown in the next example.

```
TRAVERSE BFS :g FROM { Vertex(:g, 1), Vertex(:g, 2) }
    ON VISIT VERTEX (VERTEX v) {
```

```
    ...
};
```

BFS supports various traversal directions. It is optionally passed as an argument to the BFS specifier.

```
TRAVERSE BFS('INCOMING') :g FROM :startVertex
  ON VISIT VERTEX (VERTEX v) {
    ...
  };
```

Please note the single quotes around the parameter INCOMING. The direction parameter is of type string. It can be either a reference to a string variable or a literal string value as shown here.

Three string values are supported: OUTGOING which is the default, INCOMING, and ANY. OUTGOING follows the direction of edges from the source vertex to the target vertex. INCOMING walks edges in the reverse direction from target to source. ANY treats edges as undirected edges, ignoring the direction stored in the data.

Depth-First Search (DFS)

DFS follows one adjacent edge deeper into the graph as far as possible before backtracking. In addition to the common hooks ON VISIT VERTEX and ON VISIT EDGE, it supports ON EXIT VERTEX as well. The VISIT hooks are executed when visiting the element the first time. The EXIT VERTEX hook runs right before backtracking from the vertex.

DFS does not support any other traversal directions than the default OUTGOING. The start vertex must be a single vertex.

Dijkstra's Algorithm (DIJKSTRA)

DIJKSTRA searches for shortest paths in a weighted graph. Vertices are visited in ascending order of their shortest distance from the start vertex. In order to calculate the weight of each edge, a mandatory WITH WEIGHT clause must be provided. The following example stores the shortest distance of each visited vertex in a temporary vertex attribute, similar to the built-in function SHORTEST_PATHS_ONE_TO_ALL.

```
TRAVERSE DIJKSTRA :g FROM :startVertex
  WITH WEIGHT (EDGE e) => DOUBLE {
    return :e."weight";
  }
  ON VISIT VERTEX (VERTEX v, DOUBLE distance) {
    v."distance" = :distance;
  };
```

The weight function defined by WITH WEIGHT calculates the weight of the edge e, which is passed as a parameter and returns the weight value with a return statement. It can be provided as inline closure or as a named closure, see [Closures](#). In the example, an edge attribute is directly used as the edge weight, but more complex calculations are also possible. The data type of the weight must be specified after the parameter list, following an arrow. Supported weight types are DOUBLE, INTEGER, and BIGINT.

The vertex hook after ON VISIT VERTEX uses an optional second parameter after the visited vertex. It contains the accumulated weight of the shortest path from the start vertex to the visited vertex and has the same data type as specified in the weight function. The next section describes in more detail the various signatures of the hook functions.

The start vertex has to be a single vertex. DIJKSTRA supports the traversal directions OUTGOING, INCOMING, and ANY. The traversal direction is optionally passed as an argument to the DIJKSTRA specifier (for example, TRAVERSE DIJKSTRA('ANY') :g). The default value is OUTGOING.

DIJKSTRA is currently the only algorithm to support the collection of walked paths. This enables the extraction of shortest paths from the start vertex to any visited vertex. The following example code illustrates how to search for the k-nearest neighbors on a weighted graph and extract the shortest path to each neighbor. The example searches for the three closest gas stations.

```
MAP<VERTEX,DOUBLE> stations(:g, 1024L);
-- declare result variable
TRAVERSERESULT<DOUBLE> shortest_path_tree(:g);

TRAVERSE DIJKSTRA :g FROM :startVertex
  WITH RESULT :shortest_path_tree -- pass to traversal
  WITH WEIGHT (EDGE e) => DOUBLE {
    return :e."weight";
  }
  ON VISIT VERTEX (VERTEX v, DOUBLE distance) {
    if (:v."TYPE" == 'Gas station') {
      stations[:v] = :distance;
      if (Count(:stations) == 3L) { -- end traversal when three found
        END TRAVERSE ALL;
      }
    }
  };

FOREACH (station,distance) IN :stations {
  -- extract path from start vertex to each gas station
  WEIGHTEDPATH<DOUBLE> p = GET_PATH(:shortest_path_tree, :station);
  ...
}
```

First, a variable of type TRAVERSERESULT must be declared before the TRAVERSE DIJKSTRA statement. Note that it must use the same weight type as TRAVERSE DIJKSTRA. Next, the variable is passed to TRAVERSE by using the WITH RESULT clause. Finally, shortest paths to any visited vertex can be constructed after the TRAVERSE statement by calling the GET_PATH built-in function with the result variable and the destination vertex.

Hook Functions

The extension points to the graph traversal are so-called traversal hooks. Closure expressions must be passed to the hooks which act as callbacks for well-defined events during the traversal, they can be provided either inline or as named closures, see [Closures](#).

Closures receive internal information of the traversal as function parameters. Additionally, they capture all variables from the scope of the TRAVERSE statement by reference. This means that outside variables are available inside the hook functions for read and write operations.

```
BIGINT reachable_vertices = 0L;
TRAVERSE BFS :g FROM :startVertex
  ON VISIT VERTEX (VERTEX v) {
    reachable_vertices = :reachable_vertices + 1L;
  };

```

The first parameter of vertex hook functions is always the vertex where the event happened, even if the parameter is unused inside the function. It cannot be omitted in the parameter list. Similarly, edge hooks always receive the current edge as first parameter. Additional parameters are optional and depend on the particular traversal algorithm. Next subsection explains it in more detail.

The available traversal hooks differ slightly between traversal algorithms. All graph traversals support the hooks ON VISIT VERTEX and ON VISIT EDGE, which are executed when a vertex or an edge is visited the first time. ON EXIT VERTEX is only supported by DFS and runs right before the algorithm backtracks from the vertex.

Furthermore, DIJKSTRA requires a weight function defined by WITH WEIGHT. It is the only function which has a return value: the calculated weight for the passed edge. The weight type must be specified after the parameter list. Supported weight types are DOUBLE, INTEGER, and BIGINT.

There are two special statements, `END TRAVERSE` and `END TRAVERSE ALL`, which prune edges and vertices from the search space. A more detailed description follows in the next subsections.

Hook functions allow the usage of return statements without an expression to leave the function body early. Those can for instance be used in combination with `END TRAVERSE` or `END TRAVERSE ALL` statements to not execute the subsequent instructions within the hook function.

```
TRAVERSE DFS :g FROM :startVertex
  ON VISIT VERTEX (VERTEX v) {
    if (:v == Vertex(:g, 1)) {
      END TRAVERSE ALL;
      return;
    }
    -- Other statements, that should not be executed when the if statement evaluates to 'true'.
  };
```

Additional Parameters

Traversal algorithms support various parameters within the hook functions. The first parameter is always a vertex in a vertex hook and an edge in an edge hook. Other parameters are optional and need to be specified in a particular order. The following table provides an overview of which parameters are available for each traversal algorithm and their position in the parameter list.

Algorithm	Weight	Hops	Direction*
BFS	Unsupported	2	3
DFS	Unsupported	2	3
DIJKSTRA	2	3	4

i Note

* Direction parameter is only available for edge hook

- Weight DIJKSTRA provides the accumulated weight as second optional parameter. This parameter calculates the distance from the start vertex along the shortest path by accumulating the weight of all edges. The accumulated weight has the same type as the weight type specified in the weight function.

```
TRAVERSE DIJKSTRA :g FROM :startVertex
  WITH WEIGHT (EDGE e) => DOUBLE {
    return :e."weight";
  }
  ON VISIT VERTEX (VERTEX v, DOUBLE distance) {
    v."distance" = :distance;
  };
```

- Hops

BFS and DFS provide the hop distance as an optional second parameter, and DIJKSTRA has it as a third parameter. The parameter is of type `BIGINT` and contains the number of edges it took to reach the current vertex or edge.

```
TRAVERSE DFS('INCOMING') :g FROM :startVertex
  ON VISIT VERTEX (VERTEX v, BIGINT hops) {
    v."hops" = :hops;
  };
```

- Direction

BFS and DIJKSTRA provide an optional direction parameter of type B00L at third and fourth position, respectively. It is only available for the edge hook. It describes if the edge is traversed in the direction of the edge (true value) or in the reverse direction (false value). This indicator is only relevant with traversal direction ANY.

```
TRAVERSE BFS('ANY') :g FROM :startVertex
  ON VISIT EDGE (EDGE e, BIGINT hops, B00L forwards) {
    e."hops" = :hops;
    e."forwards" = :forwards;
  };
```

Following example shows a DIJKSTRA algorithm with all supported parameters in an edge hook.

```
TRAVERSE DIJKSTRA('ANY') :g FROM :startVertex
  WITH WEIGHT (EDGE e) => DOUBLE {
    return :e."weight";
  }
  ON VISIT VERTEX (VERTEX v, DOUBLE distance, BIGINT hops) {
    v."distance" = :distance;
    v."hops" = :hops;
  }
  ON VISIT EDGE (EDGE e, DOUBLE distance, BIGINT hops, B00L direction)
  {
    if(:direction) {
      e."direction" = 'OUTGOING';
    }
    else {
      e."direction" = 'INCOMING';
    }
  }
};
```

END TRAVERSE Statement

The **END TRAVERSE** statement is only valid in a hook function or a weight function. When executed, further exploration is stopped at the current vertex or edge, pruning the search space.

In **VISIT VERTEX**, this means that the adjacent edges of the vertex are not visited. Similarly in **VISIT EDGE**, the edge is not traversed and the adjacent vertex is not visited through this edge. Be aware that there might be other paths still visiting the adjacent vertices and edges.

The following example illustrates how to avoid certain edges in a graph like toll roads by using **END TRAVERSE** in a **VISIT EDGE** hook.

```
TRAVERSE BFS :g FROM :startVertex
  ON VISIT EDGE (EDGE e) {
    if(:e."type" == 'toll road') {
      END TRAVERSE;
    }
  }
  ON VISIT VERTEX (VERTEX v, BIGINT hops) {
    v."hops" = :hops;
  };
```

END TRAVERSE does not have any effect in **EXIT VERTEX**. Use **END TRAVERSE ALL** to terminate the graph traversal.

END TRAVERSE ALL Statement

END TRAVERSE ALL is another statement only valid in a traversal hook. It terminates the graph traversal by discarding all elements for further exploration. Use it as an early exit, e.g., stop exploring when the destination is found.

```
TRAVERSE DIJKSTRA :g FROM :startVertex
  WITH WEIGHT (EDGE e) => DOUBLE {
    return :e."weight";
  }
  ON VISIT VERTEX (VERTEX v, DOUBLE distance) {
    if(:v == :destination) {
      dest_distance = :distance;
      END TRAVERSE ALL;
    }
  };
```

Please note that some other hook functions will run even after executing END TRAVERSE ALL. In BFS, all vertex or edge hooks on the same level as the vertex or edge hook calling END TRAVERSE ALL will be executed nonetheless. In DFS, backtracking is still executed which means that the EXIT VERTEX hooks are still run for already visited vertices.

Working with Collections

The SAP HANA JSON Document Store has no apriori definition of which field exists or their data type. In addition, they could be deeply nested. GraphScript on the other hand is statically typed, meaning the type associated with each and every single variable must be known before compilation of the source code. As a result, following considerations must be made in order to work with collections.

If a field is accessed within GraphScript, a type needs to be provided. Fields which are already defined in adjacency index are exempt from this rule as their type is already known (for ID, SOURCE and TARGET fields it is BIGINT, while for TYPE field it is NVARCHAR). The type can be specified in two ways:

- Using a type casting during attribute access

```
GRAPH g = GRAPH("GRAPH_SCHEMA", "EMPLOYEES_GRAPH");
VERTEX v(:g, 1L);
NVARCHAR role = TO_NVARCHAR(:v."role");

Edge e(:g, 5L);
BIGINT src = :e."src";           -- no need to provide a type for SOURCE field of adjacency index
```

- Using WITH ATTRIBUTES to define the field type

```
GRAPH g = GRAPH("GRAPH_SCHEMA", "EMPLOYEES_GRAPH") WITH ATTRIBUTES (VERTEX NVARCHAR "role");
VERTEX v(:g, 1L);
NVARCHAR role = :v."role";
```

As stated earlier, collections can also be nested. Consider the following example where the field "address" has nested fields "street" and "city" as shown below:

```
{
  "name": 'Paul',
  "address": {
    "street": '10 Downing Street',
    "city": 'London'
  }
}
```

In order to access a field of a nested JSON document, for instance "city" in the above example, the unique access path which is a concatenation of the field names with "." (dot) as a separator must be specified.

```
VERTEX v(:g, 1L);
NVARCHAR city = TO_NVARCHAR(:v."address"."city");
```

Container Types and Operations

A container is a composite value comprising zero or more elements of the same type. Containers are typically used for temporarily holding and processing intermediate results.

GraphScript offers two container types: MULTISSET and SEQUENCE. Both container types support the following data types: VERTEX, EDGE, INTEGER, BIGINT, DOUBLE, TIMESTAMP, NVARCHAR, and BOOLEAN. Furthermore, Graphscript supports nested multisets (for example, multiset of multisets), and nested sequences (for example, sequence of sequences).

The element type is a part of the type specification and is written in angular brackets after the MULTISSET or SEQUENCE keyword.

```
<multiset_type> ::=
    MULTISSET '<' <vertex_or_edge> '>'
    | MULTISSET '<' <primitive_type> '>'
    | MULTISSET '<' MULTISSET '<vertex_or_edge>' '>'
    | MULTISSET '<' MULTISSET '<primitive_type>' '>'

<sequence_type> ::=
    SEQUENCE '<' <vertex_or_edge> '>'
    | SEQUENCE '<' <primitive_type> '>'
    | SEQUENCE '<' <weightedpath> '>'
    | SEQUENCE '<' SEQUENCE '<vertex_or_edge>' '>'
    | SEQUENCE '<' SEQUENCE '<primitive_type>' '>'
```

Both container types can be used in FOREACH statements, filter expressions, and projection expressions. There are also container-specific element types and operations.

Containers of vertices or edges can only store elements from a single graph. Container variables can not be reassigned to containers with elements from another graph.

MULTISET

A multiset is an unordered container, and there is no ordinal position to reference individual elements of a multiset. Furthermore, it is also possible to declare a nested multiset (for example, a container with an unordered collection of multisets).

Constructors

Variables of type MULTISET can be initialized with a comma-separated enumeration of elements within curly brackets.

```
<multiset_init_expr> ::= '{' <initializer_list> '}'
<initializer_list> ::= <expr> | <initializer_list> , <expr>
```

```
INTEGER x = 13;
MULTISET<INTEGER> m1 = {5, 1, 5, :x, 13};
MULTISET<MULTISET<INTEGER>> m2 = {:m1, {2, 3, 1}};
```

By default, a multiset of a primitive type (INTEGER, BIGINT, DOUBLE, TIMESTAMP, NVARCHAR, and BOOLEAN) is initialized as empty.

```
MULTISET<DOUBLE> m3;
MULTISET<MULTISET<DOUBLE>> m4;
```

An empty multiset of a primitive type can also be initialized explicitly with the multiset constructor or using curly brackets.

```
MULTISET<BIGINT> m5 = MULTISET<BIGINT>();
MULTISET<MULTISET<BIGINT>> m6 = MULTISET<MULTISET<BIGINT>>();
MULTISET<BIGINT> m7 = {};
MULTISET<MULTISET<BIGINT>> m8 = {};
```

An empty multiset of VERTEX or EDGE type requires a graph variable for initialization.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
MULTISET<VERTEX> m9(:g);
MULTISET<MULTISET<VERTEX>> m10(:g);
```

Additionally, a multiset can be initialized by an operation or function returning a multiset.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
MULTISET<EDGE> m11 = EDGES(:g);
MULTISET<MULTISET<EDGE>> m12 = {EDGES(:g)};
```

Functions

COUNT

Returns the size of the given multiset. The result type is BIGINT.

```
MULTISET<DOUBLE> m15 = {0.0, 0.1, 0.2};
BIGINT m15count = COUNT(:m15); -- m15count is 3L
```

```
MULTISET<MULTISET<DOUBLE>> m16 = {{0.0, 0.1}, {0.2}};
BIGINT m16count = COUNT(:m16); -- m16count is 2L
```

DISTINCT

Returns a multiset containing unique elements from the given multiset.

```
MULTISET<BIGINT> m11 = {2L, 2L, 1L};
MULTISET<BIGINT> m12 = DISTINCT(:m11); -- m12 is {2L, 1L}

MULTISET<MULTISET<BIGINT>> m13 = {{2L, 2L}, {1L, 3L}, {3L, 1L}};
MULTISET<MULTISET<BIGINT>> m14 = DISTINCT(:m13); -- m14 is {{2L, 2L}, {3L, 1L}}
```

IS_EMPTY

Returns true if the multiset has no elements.

```
MULTISET<DOUBLE> nonEmptyMS = {0.0, 0.1, 0.2};
BOOL b1 = IS_EMPTY(:nonEmptyMS); -- b1 is FALSE

MULTISET<DOUBLE> emptyMS;
BOOL b2 = IS_EMPTY(:emptyMS); -- b2 is TRUE
```

RANDOM_ELEMENT

Returns a randomly chosen element from the given multiset.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
VERTEX random_vertex = RANDOM_ELEMENT(VERTICES(:g));
MULTISET<INT> ms = {1,2,3};
INT random_int = RANDOM_ELEMENT(:ms);
```

SEQUENCE

Returns a sequence containing elements from the given multiset. The order of elements in the resulting sequence is arbitrary. Returning a nested sequence from nested multisets is currently not supported.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
MULTISET<VERTEX> mv = VERTICES(:g);
SEQUENCE<VERTEX> sv = SEQUENCE<VERTEX>(:mv);
```

TO_ORDINALITY_MAP

This function converts a SEQUENCE of MULTISET of VERTEX or EDGE to a MAP with VERTEX or EDGE as the key and a BIGINT as the value. It places the vertices or edges as keys in the map with the ordinalities of their multiset as values. Keys that appear multiple times will be ignored after they have been discovered the first time. For a reverse operation that creates a SEQUENCE of MULTISET from a MAP, see the [TO_GROUP_SEQUENCE](#) section.

```
SEQUENCE<MULTISET<VERTEX>> groupSeq = [{:v3, :v1, :v3}, {:v2}];
MAP<VERTEX, BIGINT> ordMap = TO_ORDINALITY_MAP(:groupSeq);
:mapCommunities1[:v1] == 0L; -- true
```

```
:mapCommunities1[:v2] == 1L; -- true
:mapCommunities1[:v3] == 0L; -- true
```

Operations

Set operations take two multisets and produce a new multiset. The types of both arguments and the result have to match. If there are multisets of vertices or edges, the vertices or edges must come from the same graph.

In case of `UNION ALL`, the second argument can be a single element.

```
<set_op_expr> ::= <expr> <set_op> <expr>
<set_op> ::= <union> | <union_all> | <intersect> | <intersect_all> | <except> | <except_all>
```

```
MULTISET<BIGINT> m17 = {1L, 2L} UNION {2L, 3L}; -- m17 is {1L, 2L, 3L}
MULTISET<MULTISET<BIGINT>> m18 = {{1L}, {3L, 2L}} UNION {{2L, 3L}}; -- m18 is {{1L}, {3L, 2L}}
```

UNION ALL

Returns a multiset containing all elements from both arguments. Duplicates aren't removed.

```
MULTISET<BIGINT> m19 = {1L, 2L} UNION ALL {2L, 3L}; -- m19 is {1L, 2L, 2L, 3L}
m19 = :m19 UNION ALL 1L; -- m19 is {1L, 2L, 2L, 3L, 1L}

MULTISET<MULTISET<BIGINT>> m20 = {{1L}, {3L, 2L}} UNION ALL {{1L}}; -- m20 is {{1L}, {3L, 2L}, {1L}}
m20 = :m20 UNION ALL {2L, 3L}; -- m20 is {{1L}, {3L, 2L}, {1L}, {2L, 3L}}
```

UNION

Returns all unique elements from both arguments by removing the duplicates found. `UNION` has the same result as `UNION ALL` with a subsequent `DISTINCT`.

INTERSECT ALL

Returns all elements that are present in both arguments. Duplicates aren't removed.

INTERSECT

Returns all unique elements that are present in both arguments. Duplicates are removed. `INTERSECT` has the same result as `INTERSECT ALL` with a subsequent `DISTINCT`.

EXCEPT ALL

Returns all elements that are present in the left-hand argument but not in the right-hand argument. Duplicates aren't removed.

```
MULTISET<NVARCHAR> m21 = {'a', 'a', 'b', 'b'};
MULTISET<NVARCHAR> m22 = {'a'};
MULTISET<NVARCHAR> m23 = :m21 EXCEPT ALL :m22; -- m23 is {'b', 'b'}
```

```
VERTEX v1(:g, 1);
```

```

VERTEX v2(:g, 2);
MULTISET<MULTISET<VERTEX>> m24 = {{:v1}, {:v1,:v2}, {:v1,:v2,:v1}, {:v2,:v1}, {:v2}};
MULTISET<MULTISET<VERTEX>> m25 = {{:v1}, {:v2, :v1}, {:v1,:v1,:v2}};
MULTISET<MULTISET<VERTEX>> m26 = :m24 EXCEPT ALL :m25; -- m26 is {{:v2}, {:v1, :v2}}

```

EXCEPT

Returns all unique elements that are present in the left-hand argument but not in the right-hand argument. Duplicates are removed. EXCEPT has the same result as EXCEPT ALL with a subsequent DISTINCT.

```

MULTISET<NVARCHAR> m27 = {'a', 'a', 'b', 'b'};
MULTISET<NVARCHAR> m28 = {'a'};
MULTISET<NVARCHAR> m29 = :m27 EXCEPT :m28; -- m29 is {'b'}

MULTISET<MULTISET<NVARCHAR>> m30 = {{'a', 'b'}, {'a', 'c'}, {'c', 'a'}, {'a', 'd', 'b'}};
MULTISET<MULTISET<NVARCHAR>> m31 = {{'a'}, {'a', 'd', 'b'}};
MULTISET<MULTISET<NVARCHAR>> m32 = :m30 EXCEPT :m31; -- m32 is {{'a', 'b'}, {'a', 'c'}}

```

Comparison

Compares two multisets for equality using '=' or inequality using '!=' operator.

```

MULTISET<INTEGER> m33 = {1, 2, 2, 3};
BOOL b1 = :m33 == {2, 1, 3, 2}; -- b1 is true
BOOL b2 = :m33 != {2, 3, 1}; -- b2 is true

MULTISET<MULTISET<INTEGER>> m34 = {{1, 2}, {2, 3}};
BOOL b3 = :m34 == {{2, 1}, {3, 2}}; -- b3 is true
BOOL b4 = :m34 != {{2, 3}, {1}}; -- b4 is true

```

Filter Expressions

A filter expression evaluates a predicate for each element of an input container. The matching elements are returned as a multiset. Within the WHERE clause of the filter expression, it is possible to access temporary variables, and temporary and non-temporary attributes of vertices and edges, to use built-in functions, and to call functions. For filter expressions on sequence containers, an optional ordinal variable can be used to access the ordinal position of the element currently being processed.

```

<filter_expr> ::= <variable> IN <expr> [ WITH ORDINALITY AS <variable> ]
WHERE <expr>

```

Examples

```

INT tmp = 2;
v IN Vertices(:g) WHERE :v.color == 'green' AND :v.numEyes != 2;
v IN Vertices(:g) WHERE :v.id <= COUNT(Vertices(:g)) AND :v.numEyes == :tmp;

```

Order Expressions

An order expression sorts the elements of a mutiset and generates a sequence. For vertices and edges, it is necessary to specify the attributes to determine the order. If multiple attributes are given, they are used sequentially in case of equality in the preceding attribute. It is possible to specify the direction of order, either ASC for ascending or DESC for descending. If no direction is

specified, then the default order is ascending. NULL values are placed first in case of ascending and last for descending order. Currently, this is not supported for nested multisets.

```

<order_expr> ::= <expr> ORDER BY <arguments>
| <expr> ORDER [ <direction> ]

<arguments> ::= <l_paren> <argument_list> <r_paren>
| <argument>

<argumentList> ::= <argument>
| <argument_list> <comma> <argument>

<argument> ::= <identifier> [ <direction> ]

<direction> ::= ASC
| DESC

```

Examples

```

SEQUENCE<INT> s26 = {2,3,1} ORDER ASC;
SEQUENCE<VERTEX> s27 = {VERTEX(:g,1), VERTEX(:g,2)} ORDER BY ("KEY" ASC, "Attr" DESC);

```

For graph workspaces defined on collections, it is necessary to specify the type of the attributes on the basis of which the sequence should be ordered. This can be done during graph initialization using `WITH ATTRIBUTES`.

Examples

```

GRAPH g = GRAPH("GRAPH_SCHEMA", "EMPLOYEES_GRAPH") WITH ATTRIBUTES (VERTEX BIGINT "id");
SEQUENCE<VERTEX> s28 = MULTISET<VERTEX>(Vertices(:g)) ORDER BY "id" DESC;

```

SEQUENCE

A sequence is an ordered collection. It is possible to access and modify any element through its position, which is between 1L and the size of the sequence. Furthermore, it is also possible to declare a nested sequence i.e. a container with an ordered collection of sequences.

The elements of a sequence can be of type `WEIGHTEDPATH`; however, this is limited to the result of `K_SHORTEST_PATHS` algorithm. Currently, a sequence of `WEIGHTEDPATH` can only be read but not modified.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
VERTEX v1(:g, 'FRA');
VERTEX v2(:g, 'JFK');
SEQUENCE<WEIGHTEDPATH<BIGINT>> swp = K_SHORTEST_PATHS(:g, :v1, :v2, 3);
```

Constructors

SEQUENCE variables can be initialized with a comma-separated enumeration of elements within square brackets.

```
<sequence_init_expr> ::= '[' <initializer_list> ']'
<initializer_list> ::= <expr> | <initializer_list> , <expr>
```

```
NVARCHAR b = 'blue';
SEQUENCE<NVARCHAR> s1 = [ 'red', 'green', :b ];
SEQUENCE<SEQUENCE<NVARCHAR>> s2 = [:s1, ['yellow', :b]];
```

By default, a sequence of a primitive type is initialized as empty.

```
SEQUENCE<DOUBLE> s3;
SEQUENCE<SEQUENCE<DOUBLE>> s4;
```

An empty sequence of a primitive type can also be initialized explicitly with the sequence constructor or square brackets.

```
SEQUENCE<BIGINT> s5 = SEQUENCE<BIGINT>();
SEQUENCE<SEQUENCE<BIGINT>> s6 = SEQUENCE<SEQUENCE<BIGINT>>();
SEQUENCE<BIGINT> s7 = [];
SEQUENCE<SEQUENCE<BIGINT>> s8 = [];
```

An empty sequence of `VERTEX` or `EDGE` type requires a graph variable for initialization.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
SEQUENCE<VERTEX> s9(:g);
SEQUENCE<SEQUENCE<VERTEX>> s10(:g);
```

Additionally, a sequence can be initialized by an operation or function returning a sequence.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
SEQUENCE<VERTEX> sv = SEQUENCE<VERTEX>(VERTICES(:g));
```

Functions

COUNT

Returns the size of the given sequence. The result type is BIGINT.

```
SEQUENCE<BIGINT> sCount = [1L, 2L, 3L];
BIGINT c1 = COUNT(:sCount); -- sCount is 3L

SEQUENCE<SEQUENCE<BIGINT>> sCount2 = [[1L, 2L], [3L, 4L, 5L]];
BIGINT c2 = COUNT(:sCount2); -- sCount2 is 2L
```

IS_EMPTY

Returns true if the sequence has no elements.

```
SEQUENCE<DOUBLE> nonEmptySeq = [0.0, 0.1, 0.2];
BOOL b1 = IS_EMPTY(:nonEmptySeq); -- b1 is FALSE

SEQUENCE<DOUBLE> emptySeq;
BOOL b2 = IS_EMPTY(:emptySeq); -- b2 is TRUE
```

MULTISET

Returns a multiset containing elements from the given sequence. The order of elements is lost.

```
SEQUENCE<NVARCHAR> sColors = ['red', 'green', 'blue'];
MULTISET<NVARCHAR> mColors = MULTISET<NVARCHAR>(:sColors); -- mColors is {'green', 'blue', 'red'}
```

When initializing a new variable of type MULTISET, a shorthand notation can be used.

```
MULTISET<NVARCHAR> mshort(:sColors); -- same as mColors
```

RANDOM_ELEMENT

Returns a randomly chosen element from the given sequence.

```
INT random_int = RANDOM_ELEMENT([1,2,3]);
SEQUENCE<NVARCHAR> seq = ['a', 'b', 'c'];
NVARCHAR random_char = RANDOM_ELEMENT(:seq);
```

TO_ORDINALITY_MAP

This function converts a SEQUENCE of SEQUENCE of VERTEX or EDGE to a MAP with VERTEX or EDGE as the key and a BIGINT as the value. It places the vertices or edges as keys in the MAP with the ordinalities of the SEQUENCE, where they are located, as values. Keys that appear multiple times will be ignored after they have been discovered the first time.

```
SEQUENCE<SEQUENCE<VERTEX>> groupSeq = [[:v3, :v1, :v3], [:v2]];
MAP<VERTEX, BIGINT> ordMap = TO_ORDINALITY_MAP(:groupSeq);
:mapCommunities1[:v1] == 0L; -- true
:mapCommunities1[:v2] == 1L; -- true
:mapCommunities1[:v3] == 0L; -- true
```

Operations

Concatenation

Returns a sequence by concatenating two sequences or concatenating a sequence with a single element.

```
SEQUENCE<DOUBLE> s11 = [0.1, 0.2];
SEQUENCE<DOUBLE> s12 = :s11 || 0.3; -- s12 is [0.1, 0.2, 0.3]
SEQUENCE<DOUBLE> s13 = :s12 || :s12; -- s13 is [0.1, 0.2, 0.3, 0.1, 0.2, 0.3]

SEQUENCE<SEQUENCE<DOUBLE>> s14 = [[0.1, 0.2]];
SEQUENCE<SEQUENCE<DOUBLE>> s15 = :s14 || [[0.3]]; -- s15 is [[0.1, 0.2], [0.3]]
SEQUENCE<SEQUENCE<DOUBLE>> s16 = :s15 || :s15; -- s16 is [[0.1, 0.2], [0.3], [0.1, 0.2], [0.3]]
s14 = :s14 || [0.3, 0.1]; -- s14 is [[0.1, 0.2], [0.3, 0.1]]
```

Comparison

Compares two sequences for equality using '==' or inequality using '!=' operator.

```
SEQUENCE<INTEGER> s17 = [1, 2, 2, 3];
SEQUENCE<INTEGER> s18 = [1, 2, 2, 3];
SEQUENCE<INTEGER> s19 = [2, 1, 2, 3];
BOOL b1 = :s17 == :s18; -- b1 is true
BOOL b2 = :s17 == :s19; -- b2 is false
BOOL b3 = :s18 != :s19; -- b3 is true

SEQUENCE<SEQUENCE<INTEGER>> s20 = [[1, 2], [2, 3]];
SEQUENCE<SEQUENCE<INTEGER>> s21 = [[2, 3], [1, 2]];
BOOL b4 = :s20 == [[1, 2], [2, 3]]; -- b4 is true
BOOL b5 = :s20 == :s21; -- b5 is false
```

Positional Access

It is possible to access and modify any element of a sequence via its position, which is between 1L and the size of the sequence.

```
SEQUENCE<INTEGER> s22 = [1, 2, 3];
INTEGER a1 = :s22[1L]; -- a1 is 1
s22[3L] = 5; -- s22 is [1, 3, 5];

SEQUENCE<SEQUENCE<INTEGER>> s23 = [[1], [2, 3]];
SEQUENCE<INTEGER> s24 = :s23[1L]; -- s24 is [1]
s23[2L] = [4, 5, 6]; -- s23 is [[1], [4, 5, 6]]
```

In a nested sequence, it is possible to access and modify any element of a sequence in a nested sequence via comma-separated positions. The first index refers to the sequence's position in a nested sequence, and is between 1L and the size of the nested sequence. The second index refers to the element's position in this sequence, and is between 1L and the size of this sequence.

```
SEQUENCE<SEQUENCE<INTEGER>> s25 = [[1], [2, 3]];
INTEGER a2 = :s25[2L,1L]; -- a2 is 2
s25[2L,2L] = 4; -- s25 is [[1], [2, 4]];
```

Filter Expressions

A filter expression evaluates a predicate for each element of an input container. The matching elements are returned as a multiset. Within the WHERE clause of the filter expression, it is possible to access temporary variables, and temporary and non-temporary attributes of vertices and edges, to use built-in functions, and to call functions. For filter expressions on sequence containers, an optional ordinal variable can be used to access the ordinal position of the element currently being processed. In case of sequences it is possible to sort the result by specified attributes with an optional direction.

```
<filter_expr> ::= <variable> IN <expr> [ WITH ORDINALITY AS <variable> ]
WHERE <expr>
```

Examples

```
v IN [ Vertex(:g, 1), Vertex(:g, 2) ] WITH ORDINALITY AS i WHERE :i > 2L;
```

Order Expressions

An order expression sorts the elements of a sequence. For vertices and edges, it is necessary to specify the attributes to determine the order. If multiple attributes are given, they are used sequentially in case of equality in the preceding attribute. It is possible to specify the direction of order, either ASC for ascending or DESC for descending. If no direction is specified, then the default order is ascending. NULL values are placed first in case of ascending and last for descending order. Currently, this is not supported for nested sequences.

```
<order_expr> ::= <expr> ORDER BY <arguments>
| <expr> ORDER [ <direction> ]

<arguments> ::= <l_paren> <argument_list> <r_paren>
| <argument>

<argumentList> ::= <argument>
| <argument_list> <comma> <argument>

<argument> ::= <identifier> [ <direction> ]

<direction> ::= ASC
| DESC
```

Examples

```
SEQUENCE<INT> s26 = [2,3,1] ORDER ASC;
SEQUENCE<VERTEX> s27 = [VERTEX(:g,1), VERTEX(:g,2)] ORDER BY ("KEY" ASC, "Attr" DESC);
```

For graph workspaces defined on collections, it is necessary to specify the type of the attributes on the basis of which the sequence should be ordered. This can be done during graph initialization using WITH ATTRIBUTES.

Examples

```
GRAPH g = GRAPH("GRAPH_SCHEMA", "EMPLOYEES_GRAPH") WITH ATTRIBUTES (VERTEX BIGINT "id");
SEQUENCE<VERTEX> s28 = SEQUENCE<VERTEX>(Vertices(:g)) ORDER BY "id" DESC;
```

MAP

A map is an associative container that contains key-value-pairs with unique keys.

The keys of a map can be of type VERTEX, EDGE, or any scalar type except DOUBLE and BOOL. The keys must not be NULL. Any scalar data type can be used to store the mapped values.

During the construction, a default value can be specified. If no default value is specified, the default is assumed NULL. The default value must have the same type as the type of the value. When reading a map entry, if no value for the given key was stored beforehand, the default value is returned.

Constructors

MAP variables must be initialized with a constructor call which takes the expected amount of entries as parameter of type BIGINT. Optionally, a default value can be specified as the last parameter. If the key type is VERTEX or EDGE, the corresponding graph variable must be passed as the first parameter.

```
<map_type> ::=
    MAP '<' <vertex_or_edge>, <primitive_type> '>'
    | MAP '<' <primitive_type>, <primitive_type> '>'

<map_constructor> ::=
    <map_type> '(' <expected_size> [, <default_value>] ')'
    | <map_type> '(' <graph_variable>, <expected_size> [, <default_value>] ')'
<map_definiton> ::= <map_type> '=' <map_constructor> ';'
```

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH");
BIGINT expected_size = 100L;
MAP<VERTEX, NVARCHAR> mapVertex(:g, :expected_size, 'default value');
MAP<BIGINT, DOUBLE> mapInt(10L);
```

Functions

COUNT

Returns the number of entries stored in the map. The result type is BIGINT.

```
MAP<INTEGER, INTEGER> mapInt(10L);
mapInt[1] = 2;
mapInt[2] = 4;
BIGINT mapCount = COUNT(:mapInt); -- mapCount is 2L
```

IS_EMPTY

Returns true if the map has no elements.

```
MAP<INTEGER, INTEGER> emptyMap(10L);
BOOL b1 = IS_EMPTY(:emptyMap); -- b1 is TRUE
emptyMap[26] = 5;
BOOL b2 = IS_EMPTY(:emptyMap); -- b2 is FALSE
```

TO_GROUP_SEQUENCE

This function converts a MAP with VERTEX or EDGE as the key and a BIGINT as the value to a SEQUENCE of MULTiset of the key. It groups the keys with the same value in a MULTiset. Vertices or edges with no or a default value will be ignored. These groupings are placed in the SEQUENCE with the ordinality of the value. For example, a vertex with value 3 would be put in the MULTiset that is located in the outer SEQUENCE at position 4 (positions starts with 1, so a 1 is added to the value). A negative value will result in an exception.

For a reverse operation that creates a MAP from a SEQUENCE of MULTiset, or a MAP from a SEQUENCE of SEQUENCE, see [TO_ORDINALITY_MAP section \(MULTiset\)](#) or [TO_ORDINALITY_MAP section \(SEQUENCE\)](#).

```
MAP<VERTEX, BIGINT> ordMap (:g, 3L);
ordMap[:v1] = 0L;
ordMap[:v2] = 1L;
ordMap[:v3] = 0L;
SEQUENCE<MULTiset<VERTEX>> groupSeq= TO_GROUP_SEQUENCE(:ordMap);
: groupSeq == [{:v1,:v3}, {:v2}]; -- true
```

Operations

Access

Operator [] can be used to read and write to a map via a given key.

```
MAP<INTEGER, INTEGER> mapInt(10L, 42);
INTEGER defaultVal = :mapInt[4]; -- default is 42
mapInt[10] = 5; -- store value 10 at key 5
INTEGER val = :mapInt[10]; -- val is 5
```

PRIORITYQUEUE

A priority queue is a collection of elements ordered by their associated priority values

The elements can be of type VERTEX or EDGE. The priority value can be of type INTEGER, BIGINT, or DOUBLE. Elements are inserted at a position corresponding to their priority. Elements can be fetched from the front of the queue. The front element is the one with the lowest priority value.

Constructors

PRIORITYQUEUE variables must be initialized with a constructor call which takes the corresponding graph as only parameter.

```
<priorityqueue_type> ::=
    PRIORITYQUEUE '<' <vertex_or_edge>, <primitive_type> '>'

<priorityqueue_constructor> ::=
    <priorityqueue_type> '(' <graph_variable> ')'
<priorityqueue_definiton> ::= <priorityqueue_type> '=' <priorityqueue_constructor> ';'

GRAPH g = GRAPH("FLIGHTS", "GRAPH");
PRIORITYQUEUE<VERTEX, BIGINT> priorityQueueBigIntVertex(:g);
```

Functions

COUNT

Returns the number of entries stored in the priority queue. The result type is BIGINT.

```
PRIORITYQUEUE<EDGE, INTEGER> priorityQueueIntEdge(:g);
PUSH(:priorityQueueIntEdge, :edge0, 2);
PUSH(:priorityQueueIntEdge, :edge1, 4);
BIGINT priorityQueueCount = COUNT(:priorityQueueIntEdge); -- priorityQueueIntEdgeCount is 2L
```

IS_EMPTY

Returns true if the priority queue has no elements.

```
PRIORITYQUEUE<EDGE, INTEGER> emptyPriorityQueue(:g);
BOOL b1 = IS_EMPTY(:emptyPriorityQueue); -- b1 is TRUE
PUSH(:emptyPriorityQueue, :edge0, 2);
BOOL b2 = IS_EMPTY(:emptyPriorityQueue); -- b2 is FALSE
```

PUSH

Inserts the element with the given priority into the priority queue. In case the element is already present an exception is thrown.

```
PRIORITYQUEUE<EDGE, INTEGER> priorityQueue2(:g);
PUSH(:priorityQueue2, :edge0, 2); -- inserts edge0 with priority 2
PUSH(:priorityQueue2, :edge1, 4); -- inserts edge1 with priority 4
PUSH(:priorityQueue2, :edge1, 1); -- error since edge1 has already been inserted
```

FETCH

Returns the element with the lowest priority value and removes it from the priority queue. In case the priority queue is empty an exception is thrown.

```
PRIORITYQUEUE<EDGE, INTEGER> priorityQueue1(:g);
PUSH(:priorityQueue1, :edge0, 2);
PUSH(:priorityQueue1, :edge1, 4);
EDGE edgeFirst = FETCH(:priorityQueue1); -- edgeFirst is edge0
EDGE edgeSecond = FETCH(:priorityQueue1); -- edgeSecond is edge1
BOOL b = IS_EMPTY(:priorityQueue1); -- b is TRUE
```

GET_TOP_PRIORITY

Returns the priority of the element with the lowest priority value.

```
PRIORITYQUEUE<EDGE, INTEGER> priorityQueue3(:g);
PUSH(:priorityQueue3, :edge0, 2); -- inserts edge0 with priority 2
PUSH(:priorityQueue3, :edge1, 4); -- inserts edge1 with priority 4
INTEGER topWeight = GET_TOP_PRIORITY(:priorityQueue3); -- topWeight is 2
```

Vector Types

REAL_VECTOR

A REAL_VECTOR is a fixed-sized vector of single-precision floating point values. It cannot be modified in GRAPHSCRIPT.

Typically, a REAL_VECTOR is computed by machine learning methods and each element represents a characteristic of an item, so that the similarity of the REAL_VECTOR corresponds to the similarity of the items.

HALF_VECTOR

A HALF_VECTOR is a fixed-sized vector of half-precision floating point values. It cannot be modified in GRAPHSCRIPT.

Typically, a HALF_VECTOR is computed by machine learning methods and each element represents a characteristic of an item, so that the similarity of the HALF_VECTOR corresponds to the similarity of the items.

HALF_VECTOR can be used instead of REAL_VECTOR to reduce memory consumption.

All functions available on REAL_VECTOR are also available on HALF_VECTOR

Constructors

REAL_VECTOR variables can be initialized with a string containing a comma-separated enumeration of elements within square brackets.

```
REAL_VECTOR v(' [2,3,5]');
```

Functions

CARDINALITY

Returns the number of elements.

```
REAL_VECTOR v(' [2,3,5]');
BIGINT i = CARDINALITY(:v); -- i is 3L
```

MEMBER_AT

Returns the element of the vector at the given index.

```
REAL_VECTOR v(' [2,3,5]');
DOUBLE d = MEMBER_AT(:v, 1L); -- d = 2.0
```

L2DISTANCE

Returns the euclidean distance between two REAL_VECTOR.

```
REAL_VECTOR v1(' [2,3,5] ');
DOUBLE d1 = L2DISTANCE(:v1, :v1); -- d1 = 0.0 -- same vector, distance is 0
REAL_VECTOR v2(' [3,5,7] ');
DOUBLE d2 = L2DISTANCE(:v1, :v2); -- d2 = 3.0 -- euclidean distance is square root of (1*1 + 2*2 +
```

L2NORM

Returns the length (euclidean norm) of a REAL_VECTOR.

```
REAL_VECTOR v1(' [3,0,4] ');
DOUBLE d = L2NORM(:v1); -- d = 5.0
```

L2NORMALIZE

Returns a vector with the same direction and length 1.

```
REAL_VECTOR v1(' [3,0,4] ');
REAL_VECTOR v2 = L2NORMALIZE(:v1); -- v2 = [0.6, 0, 0.8]
```

COSINE_SIMILARITY

Returns the cosine of the angle between two REAL_VECTOR.

```
REAL_VECTOR v1(' [1,0,0] ');
DOUBLE d1 = COSINE_SIMILARITY(:v1, :v1); -- d = 1.0 -- same vector, angle is 0 degree
REAL_VECTOR v2(' [0,1,0] ');
DOUBLE d2 = COSINE_SIMILARITY(:v1, :v2); -- d = 0.0 -- orthogonal vector, angle is 90 degree
```

SUBVECTOR

Returns a subvector with the given dimension starting at the given start index from a vector.

```
REAL_VECTOR v1(' [1,2,3,4,5,6] ');
REAL_VECTOR v2 = SUBVECTOR(:v1,2L,3L); -- v2 = [2,3,4];
```

TO_HALF_VECTOR

Converts a REAL_VECTOR to a HALF_VECTOR.

```
REAL_VECTOR rv(' [1,2,3] ');
HALF_VECTOR hv = TO_HALF_VECTOR(:rv); -- hv = [1,2,3];
```

TO_REAL_VECTOR

Converts a HALF_VECTOR to a REAL_VECTOR.

```
HALF_VECTOR hv(' [1,2,3] ');  
REAL_VECTOR rv = TO_REAL_VECTOR(:hv); -- rv = [1,2,3];
```

Related Information

[REAL_VECTOR and HALF_VECTOR Data Types](#)

[Vector Data Types](#)

Table Operations

Tables are the central means for exchanging larger chunks of data between GraphScript procedures and their outside database environment. Tables can't be instantiated within GraphScript procedures, so they're always passed as input or output parameters. All column types must be compatible with GraphScript types. If an input table contains an unsupported column type, it cannot be accessed in the GraphScript procedure.

Cell Read Access

Reads the value at a certain index position of a specified table column. Returns NULL if the desired cell index is larger than the row count of the table. The index is 1-based, for example, the first element is accessed with the index number 1L.

```
INTEGER i = :tab."IntColumn"[1L];
```

Cell Write Access

Sets the value at a certain index position of a specified table column. If the index is larger than the row count of the table, then the table is resized accordingly and padded with NULL values if necessary.

```
tab."IntColumn"[5L] = 123;
```

Row Count

The row count of a table can be determined using the COUNT function expression.

```
BIGINT numRows = COUNT(:tab);
```

Examples

```
CREATE TYPE tt AS TABLE(NAME NVARCHAR, ATTR INTEGER);

CREATE PROCEDURE myTableProc (IN i_tab tt, OUT o_i INTEGER, OUT o_numRows BIGINT)
LANGUAGE GRAPH READS SQL DATA AS
BEGIN
    o_i = :i_tab.ATTR[1L];
    i_tab."NAME"[3L] = "John";
    o_numRows = COUNT(:i_tab);
END;
```

Projection Expression

A shorthand notation to convert data in a container to a tabular format is provided by the projection expression. The result must be assigned to a table variable with a compatible column layout. For example, the type of first column must be compatible to the type of the first element in the projection list, and so on. The assignment of projection list elements to result table columns is done by position. Note that old content of the table variable gets replaced by the new content of the projection.

The projection expression starts with the SELECT keyword, followed by the projection list and the FOREACH notation used in [FOREACH loops](#). The projection list is a comma-separated list of expressions. For example, these expressions can be an access to attributes of a vertex variable or a lookup into a Sequence or Map container with a scalar variable as positional index or key value.

```
<projection_expr> ::= SELECT <proj_list> FOREACH <variable> IN <expr> [WITH ORDINALITY AS <variable>]
| SELECT <proj_list> FOREACH ( <variable> , <variable> ) IN <expr>
<proj_list> ::= <proj> [, <proj_list>]
<proj> ::= <expr>
```

The projection expression may iterate over containers or maps.

```
MULTISET<VERTEX> myVertices = ...;
-- projecting vertex attributes
myTable1 = SELECT :v."NAME", :v.visited FOREACH v IN :myVertices;

SEQUENCE<EDGE> edgeSequence = ...;
-- projecting one edge attribute and the position of the edge in the sequence
myTable2 = SELECT :e."NAME", :i FOREACH e IN :edgeSequence WITH ORDINALITY AS i;

MAP<VERTEX,DOUBLE> costMap = ...;
-- projecting vertices in a map and associated values, cost in map and vertex attributes
myTable3 = SELECT :v."NAME", :cost FOREACH (v,cost) IN :costMap;
```

i Note

Tables or table types are permitted for IN and OUT parameters.

Examples

```
CREATE TYPE tt AS TABLE(NAME NVARCHAR, ATTR INTEGER);

CREATE PROCEDURE myTableProc (IN i_tab tt, OUT o_tab tt)
LANGUAGE GRAPH READS SQL DATA AS
BEGIN
  INTEGER i = :i_tab.attr[1L];
  GRAPH g = GRAPH("FLIGHTS", "GRAPH")
  WITH TEMPORARY ATTRIBUTES(VERTEX INTEGER ATTR = :i);
  o_tab = SELECT :v."NAME", :v.attr FOREACH v IN VERTICES(:g);
END;
```

Return Table Statement

A GraphScript table function must return a table of the type defined in the return clause of the function definition. The GraphScript return table statement has the following syntax:

```
<return_table_statement> ::= RETURN <projection_expr> ;
```

The return table statement terminates the execution of the current function and returns the specified table to the caller. Return table statements are supported only in GraphScript table functions and not in procedures or anonymous blocks.

Example

```
CREATE OR REPLACE FUNCTION myTableFunction (val INTEGER)
RETURNS TABLE(NAME NVARCHAR, ATTR INTEGER)
LANGUAGE GRAPH AS
```

```

BEGIN
  GRAPH g = GRAPH("FLIGHTS", "GRAPH")
    WITH TEMPORARY ATTRIBUTES(VERTEX INTEGER ATTR = :val);
  RETURN SELECT :v."NAME", :v.attr FOREACH v IN VERTICES(:g);
END;

```

Table Column Cast Expressions

A table column can also be cast to a multiset, a sequence or a map of scalars, provided that their element primitive types match.

```

<multiset_init_expr> ::=  MULTiset '<' <primitive_type> '>' '(' <attr_access_expr> ')'
<sequence_init_expr> ::=  SEQUENCE '<' <primitive_type> '>' '(' <attr_access_expr> ')'
<sequence_init_expr> ::=  MAP '<' <primitive_type>, <primitive_type> '>' '(' <attr_access_expr>, <i

```

Examples

```

SEQUENCE<INTEGER> si = SEQUENCE<INTEGER>(:i_tab."ATTR");
MULTISET<INTEGER> mi = MULTISET<INTEGER>(:i_tab."ATTR");
MAP<INTEGER, BIGINT> m = MAP<INTEGER, BIGINT>(:i_tab."ATTR", :b_tab."BIGATTR")

-- using shorthand notation
SEQUENCE<INTEGER> si(:i_tab."ATTR");
MULTISET<INTEGER> mi(:i_tab."ATTR");
MAP<INTEGER, BIGINT> m(:i_tab."ATTR", :b_tab."BIGATTR")

```

Scalar Data Types and Operations

GraphScript supports scalar data types and operations.

The following table summarizes the supported scalar data types in GraphScript and classifies them by their characteristics:

Classification	Data Type
Numeric types	INTEGER, BIGINT, DOUBLE
Boolean type	BOOLEAN
Character string type	NVARCHAR
Datetime type	DATE, TIMESTAMP
Binary character string type	VARBINARY
Spatial data object	ST_GEOMETRY

All scalar variables support the special value `NULL`.

Numeric Types

Each numeric type has a maximum value and a minimum value. A numeric overflow exception is thrown if a given value is smaller than the minimum allowed value or greater than the maximum allowed value.

INTEGER

The `INTEGER` data type specifies a 32-bit signed integer.

- The minimum value is -2,147,483,648.
- The maximum value is 2,147,483,647.
- Integer literals that aren't suffixed with an 'L' are treated as 32-bit integer literals.
- A variable of type `INTEGER` is default-initialized with the value `NULL`.

BIGINT

The `BIGINT` data type specifies a 64-bit signed integer.

- The minimum value is -9,223,372,036,854,775,808.
- The maximum value is 9,223,372,036,854,775,807.
- Literal values of type `BIGINT` have to be suffixed with an 'L'.
- A variable of type `BIGINT` is default-initialized with the value `NULL`.

DOUBLE

The `DOUBLE` data type specifies a double-precision 64-bit floating-point number.

- The minimum value is -1.7976931348623157E308 and the maximum value is 1.7976931348623157E308.
- The smallest positive `DOUBLE` value is 2.2250738585072014E-308 and the largest negative `DOUBLE` value is -2.2250738585072014E-308.
- A variable of type `DOUBLE` is default-initialized with the value `NULL`.

Non-Numeric Scalar Types

String Type

The NVARCHAR character string data type specifies character strings of variable length. It's used both for storing ASCII and Unicode character strings. A string variable is default-initialized with value NULL.

Boolean Type

The BOOLEAN data type stores Boolean values, which are TRUE and FALSE. A variable of type BOOLEAN is default-initialized with value NULL.

Datetime Type

The type DATE represents a date with day, month and year.

- Its default format is 'YYYY-MM-DD'.
- The range of the date value is between 0001-01-01 and 9999-12-31.
- A variable of type DATE is default-initialized with NULL.

The TIMESTAMP data type consists of date and time information.

- Its default format is 'YYYY-MM-DD HH24:MI:SS.FF7'.
- FF_n represents the fractional seconds where n indicates the number of digits in the fractional part.
- The range of the timestamp value is between 0001-01-01 00:00:00.0000000 and 9999-12-31 23:59:59.9999999.
- A variable of type TIMESTAMP is default-initialized with NULL.

Varbinary Type

The VARBINARY data type stores binary character data of variable length. A varbinary variable is default-initialized with value NULL.

Spatial object type

The ST_GEOMETRY data type stores a spatial object. An ST_GEOMETRY variable is default-initialized with value NULL. Support of ST_GEOMETRY in the Graphscript language is limited. Currently, it is only possible to read ST_GEOMETRY values from vertex or edge attributes or local variables. The values may be assigned to a local ST_GEOMETRY variable or to an output column using cell access or projection statements. More information about the data type can be found in the [SAP HANA Spatial documentation](#).

GraphScript supports all methods of ST_GEOMETRY that are listed in the SAP HANA Spatial documentation which do not return ST_GEOMETRY or VARBINARY. In contrast to the use in SQL, the spatial methods are called as functions in GraphScript.

Examples

```
ST_GEOMETRY my_geom = :inTab.val[1L]; -- inTab contains a geometry like this: LINESTRING(0 0, 0 1)
BOOLEAN isValid = ST_ISVALID(:my_geom);
INTEGER lengthInMeter = ST_LENGTH(:my_geom, 'meter');
BOOLEAN isWithin = ST_WITHINDISTANCE(:my_geom, :my_geom, 0.0);
```

Related Information

[Supported Spatial Data Types and Their Hierarchy](#)

[ST_Geometry Type](#)

Typed Literals

A literal is a symbol that represents a specific fixed data value.

Numeric Literals

A numeric literal is represented by a sequence of digits that aren't enclosed in quotation marks. If a numeric literal contains a decimal point, then it's treated as a DOUBLE precision floating point value. BIGINT literals need to be suffixed with an L.

```
<int_literal> ::= 0 | <pos_digit> {<digit>}
<bigint_literal> ::= <int_literal> L
<double_literal> ::= ( 0 | <pos_digit> {<digit>}) <dot> <digit> {<digit>}
<numeric_literal> ::= <int_literal> | <bigint_literal> | <double_literal>
```

Examples

```
123 -- INTEGER
123L -- BIGINT
123.4 -- DOUBLE
```

String Literals

A character string literal is enclosed in single quotation marks. The character string literal is optionally prefixed by an N.

```
<nvarchar_literal> ::=
    <single_quote> {<any_character>-<single_quote>} <single_quote>
    | N<single_quote> {<any_character>-<single_quote>} <single_quote>
```

Examples

```
'Material' -- NVARCHAR
'23' -- NVARCHAR
N'is used in ' -- NVARCHAR
```

Boolean Literals

A Boolean literal can be either TRUE or FALSE. Both literals are keywords in GraphScript and therefore are case-insensitive.

Datetime Literals

Datetime literals correspond to date or timestamp constructors with a string literal as initializer.

```
<date_literal> ::= DATE <l_paren> <single_quote> YYYY-MM-DD <single_quote> <r_paren>
<timestamp_literal> ::= TIMESTAMP <l_paren> <single_quote> YYYY-MM-DD HH24:MI:SS.FF7 <single_quote>
```

```
                                <r_paren>  
<datetime_literal>      ::= <date_literal> | <timestamp_literal>
```

Examples

```
DATE( '2019-12-19' )  
TIMESTAMP( '2019-12-19 16:30:12.345' )
```

Type Casts

BOOLEAN

Casts a BOOLEAN, INTEGER, BIGINT or NVARCHAR value to a BOOLEAN value.

- `TO_BOOLEAN(BOOLEAN) => BOOLEAN`
- `TO_BOOLEAN(INTEGER) => BOOLEAN`
- `TO_BOOLEAN(BIGINT) => BOOLEAN`
- `TO_BOOLEAN(NVARCHAR) => BOOLEAN`

If value is:

- 1, 'true' or true then `TO_BOOLEAN` returns true.
- 0, 'false', or false then `TO_BOOLEAN` returns false.

'true' and 'false' are not case sensitive, meaning that values of 'TRUE' and 'TruE' are treated the same as a value of true. `TO_BOOLEAN` throws an exception when value is not one of true, false, 0, 1, '0', '1', 'true' or 'false'.

It is also possible to provide a second argument having type BOOLEAN. If the type casting fails or the first argument is NULL, then the second argument is returned. Similar applies to other type casts with second argument.

- `TO_BOOLEAN(BOOLEAN, BOOLEAN) =>` Returns the first argument as long it is not NULL, else it returns the second argument.
- `TO_BOOLEAN(INTEGER, BOOLEAN) =>` Type casts INTEGER to BOOLEAN. If it fails or is NULL, it returns the second argument.
- `TO_BOOLEAN(BIGINT, BOOLEAN) =>` Type casts BIGINT to BOOLEAN. If it fails or is NULL, it returns the second argument.
- `TO_BOOLEAN(NVARCHAR, BOOLEAN) =>` Type casts NVARCHAR to BOOLEAN. If it fails or is NULL, it returns the second argument.

```
BOOLEAN var1 = TO_BOOLEAN(true); -- var1 is true
BOOLEAN nullVal;
BOOLEAN var2 = TO_BOOLEAN(:nullVal, false); -- var2 is false as the first value is null
```

INTEGER

Casts a DOUBLE, BIGINT, NVARCHAR or BOOLEAN value to an INTEGER value. Throws a conversion error in the event of value overflows or underflows.

- `TO_INTEGER(INTEGER) => INTEGER`
- `TO_INTEGER(DOUBLE) => INTEGER`
- `TO_INTEGER(BIGINT) => INTEGER`
- `TO_INTEGER(NVARCHAR) => INTEGER`

- `TO_INTEGER(BOOLEAN) => INTEGER`

It is also possible to provide a second argument having type `INTEGER`. If the type casting fails or the first argument is `NULL`, then the second argument is returned. Similar applies to other type casts with second argument.

- `TO_INTEGER(INTEGER, INTEGER) =>` Returns first argument as long it is not `NULL`, else returns second argument
- `TO_INTEGER(DOUBLE, INTEGER) =>` Type casts `DOUBLE` to `INTEGER`. If it fails or is `NULL`, it returns the second argument.
- `TO_INTEGER(BIGINT, INTEGER) =>` Type casts `BIGINT` to `INTEGER`. If it fails or is `NULL`, it returns the second argument.
- `TO_INTEGER(NVARCHAR, INTEGER) =>` Type casts `NVARCHAR` to `INTEGER`. If it fails or is `NULL`, it returns the second argument.
- `TO_INTEGER(BOOLEAN, INTEGER) =>` Type casts `BOOLEAN` to `INTEGER`. If it is `NULL`, it returns the second argument.

```
INTEGER var1 = TO_INTEGER(5.1, 2); -- var1 is 5
INTEGER var2 = TO_INTEGER(4000000000L, 2); -- var2 is 2 as value overflows during casting
INTEGER var3 = TO_INTEGER('23'); -- var3 is 23
```

BIGINT

Casts an `INTEGER`, `DOUBLE`, `NVARCHAR` OR `BOOLEAN` value to a `BIGINT` value. Throws a conversion error in the event of value overflows or underflows.

- `TO_BIGINT(BIGINT) => BIGINT`
- `TO_BIGINT(INTEGER) => BIGINT`
- `TO_BIGINT(DOUBLE) => BIGINT`
- `TO_BIGINT(NVARCHAR) => BIGINT`
- `TO_BIGINT(BOOLEAN) => BIGINT`

It is also possible to provide a second argument having type `BIGINT`.

- `TO_BIGINT(BIGINT, BIGINT) =>` Returns the first argument as long it is not `NULL`, else it returns the second argument.
- `TO_BIGINT(INTEGER, BIGINT) =>` Type casts `INTEGER` to `BIGINT`. If it is `NULL`, it returns the second argument.
- `TO_BIGINT(DOUBLE, BIGINT) =>` Type casts `DOUBLE` to `BIGINT`. If it fails or is `NULL`, it returns the second argument.
- `TO_BIGINT(NVARCHAR, BIGINT) =>` Type casts `NVARCHAR` to `BIGINT`. If it fails or is `NULL`, it returns the second argument.
- `TO_BIGINT(BOOLEAN, BIGINT) =>` Type casts `BOOLEAN` to `BIGINT`. If it is `NULL`, it returns the second argument.

```
GRAPH g = GRAPH("FLIGHTS", "GRAPH")
  WITH TEMPORARY ATTRIBUTES(VERTEX INTEGER cost = 10);
VERTEX v = VERTEX(:g, 'FRA');
BIGINT var1 = TO_BIGINT(:v.cost, 2L); -- var1 is 10L
```

DOUBLE

Casts an `INTEGER`, `BIGINT` or `NVARCHAR` value to a `DOUBLE` value.

- `TO_DOUBLE(DOUBLE) => DOUBLE`

- `TO_DOUBLE(INTEGER) => DOUBLE`
- `TO_DOUBLE(BIGINT) => DOUBLE`
- `TO_DOUBLE(NVARCHAR) => DOUBLE`

It is also possible to provide a second argument having type `DOUBLE`.

- `TO_DOUBLE(DOUBLE, DOUBLE) =>` Returns first argument as long it is not `NULL`, else it returns the second argument.
- `TO_DOUBLE(INTEGER, DOUBLE) =>` Type casts `INTEGER` to `DOUBLE`. If it fails or is `NULL`, it returns the second argument.
- `TO_DOUBLE(BIGINT, DOUBLE) =>` Type casts `BIGINT` to `DOUBLE`. If it fails or is `NULL`, it returns the second argument.
- `TO_DOUBLE(NVARCHAR, DOUBLE) =>` Type casts `NVARCHAR` to `DOUBLE`. If it fails or is `NULL`, it returns the second argument.

```
DOUBLE var1 = NULL;
DOUBLE var2 = TO_DOUBLE(:var1, 3.5); -- var2 is 3.5
```

NVARCHAR

Casts an `INTEGER`, `BIGINT`, `DOUBLE` or `BOOLEAN` value to an `NVARCHAR` value.

- `TO_NVARCHAR(NVARCHAR) => NVARCHAR`
- `TO_NVARCHAR(INTEGER) => NVARCHAR`
- `TO_NVARCHAR(BIGINT) => NVARCHAR`
- `TO_NVARCHAR(DOUBLE) => NVARCHAR`
- `TO_NVARCHAR(BOOLEAN) => NVARCHAR`

It is also possible to provide a second argument having type `NVARCHAR`.

- `TO_NVARCHAR(NVARCHAR, NVARCHAR) =>` Returns first argument as long it is not `NULL`, else it returns the second argument.
- `TO_NVARCHAR(INTEGER, NVARCHAR) =>` Type casts `INTEGER` to `NVARCHAR`. If `NULL`, it returns the second argument.
- `TO_NVARCHAR(BIGINT, NVARCHAR) =>` Type casts `BIGINT` to `NVARCHAR`. If `NULL`, it returns the second argument.
- `TO_NVARCHAR(DOUBLE, NVARCHAR) =>` Type casts `DOUBLE` to `NVARCHAR`. If `NULL`, it returns the second argument.
- `TO_NVARCHAR(BOOLEAN, NVARCHAR) =>` Type casts `BOOLEAN` to `NVARCHAR`. If `NULL`, it returns the second argument.

```
NVARCHAR var1 = TO_NVARCHAR(3.2, 'bc'); -- var1 is '3.2'
NVARCHAR var2 = NULL;
var1 = TO_NVARCHAR(:var2, 'b'); -- var1 is 'b'
```

TIMESTAMP

Casts a `NVARCHAR` value to a `TIMESTAMP` value.

- `TO_TIMESTAMP(TIMESTAMP) => TIMESTAMP`
- `TO_TIMESTAMP(NVARCHAR) => TIMESTAMP`

It is also possible to provide a second argument having type `TIMESTAMP`.

- `TO_TIMESTAMP(TIMESTAMP, TIMESTAMP)` => Returns the first argument as long it is not `NULL`, else it returns the second argument.
- `TO_TIMESTAMP(NVARCHAR, TIMESTAMP)` => Type casts `NVARCHAR` to `TIMESTAMP`. If it fails or is `NULL`, it returns the second argument.

```
TIMESTAMP var1 = TO_TIMESTAMP('1972-04-01 01:01:01.0000001');
TIMESTAMP var2 = TO_TIMESTAMP('HANA', :var1); -- var2 is '1972-04-01 01:01:01.0000001'
```

DATE

Casts a `NVARCHAR` value to a `DATE` value.

- `TO_DATE(DATE)` => `DATE`
- `TO_DATE(NVARCHAR)` => `DATE`

It is also possible to provide a second argument having type `DATE`.

- `TO_DATE(DATE, DATE)` => Returns the first argument as long it is not `NULL`, else it returns the second argument.
- `TO_DATE(NVARCHAR, DATE)` => Type casts `NVARCHAR` to `DATE`. If it fails or is `NULL`, it returns the second argument.

```
DATE var1 = TO_DATE('1972-04-01');
DATE var2 = TO_DATE('HANA', :var1); -- var2 is '1972-04-01'
```

Arithmetic Expressions for numerical types

Arithmetic expressions are unary or binary expressions that take one or two expressions and an arithmetic operator and produce a result. Both expressions have to be of the same type.

```
<arithmetic_op> ::= <plus> | <minus> | <asterisk> | <slash> | <modulo>
<binary_arithmetic_expr> ::= <expr> <arithmetic_op> <expr>
```

Examples

```
INTEGER suma = 2 + 3;
INTEGER diff = 2 - 3;
BIGINT prod = 2L * 3L;
DOUBLE quot = 2.0 / 3.0;
INTEGER modulo = 10 % 3;
```

Arithmetic Expressions for vector types

For vector types a subset of arithmetic expressions is supported: plus, minus and multiplication with a numeric value

Examples

```
REAL_VECTOR suma = REAL_VECTOR('[1,2,3]') + REAL_VECTOR('[4,5,6]'); -- suma = [5, 7, 9]
REAL_VECTOR diff = REAL_VECTOR('[1,2,3]') - REAL_VECTOR('[4,5,6]'); -- diff = [-3,-3,-3]
REAL_VECTOR prodWithScalar = 3.0 * REAL_VECTOR('[1,2,3]'); -- prodWithScalar = [3, 6, 9]
```

Numerical Functions

GraphScript supports several numerical functions that are similar in name and behavior to their SQL counterparts. The following is a list of the functions and their supported argument and return types. For details about their functionality, please consult the [SAP HANA Cloud, SAP HANA Database SQL Reference Guide](#).

ABS

- `ABS (INTEGER) => INTEGER`
- `ABS (BIGINT) => BIGINT`
- `ABS (DOUBLE) => DOUBLE`

ACOS

- `ACOS (DOUBLE) => DOUBLE`

ASIN

- `ASIN (DOUBLE) => DOUBLE`

ATAN

- `ATAN (DOUBLE) => DOUBLE`

ATAN2

- `ATAN2 (DOUBLE, DOUBLE) => DOUBLE`

BITAND

- `BITAND (INTEGER, INTEGER) => INTEGER`
- `BITAND (BIGINT, BIGINT) => BIGINT`

BITCOUNT

- `BITCOUNT (INTEGER, INTEGER) => INTEGER`
- `BITCOUNT (BIGINT, BIGINT) => BIGINT`

BITNOT

- `BITNOT(INTEGER) => INTEGER`
- `BITNOT(BIGINT) => BIGINT`

BITOR

- `BITOR(INTEGER, INTEGER) => INTEGER`
- `BITOR(BIGINT, BIGINT) => BIGINT`

BITXOR

- `BITXOR(INTEGER, INTEGER) => INTEGER`
- `BITXOR(BIGINT, BIGINT) => BIGINT`

CEIL

- `CEIL(INTEGER) => INTEGER`
- `CEIL(BIGINT) => BIGINT`
- `CEIL(DOUBLE) => DOUBLE`

COS

- `COS(DOUBLE) => DOUBLE`

COSH

- `COSH(DOUBLE) => DOUBLE`

COT

- `COT(DOUBLE) => DOUBLE`

EXP

- `EXP(DOUBLE) => DOUBLE`

FLOOR

- `FLOOR(INTEGER) => INTEGER`
- `FLOOR(BIGINT) => BIGINT`

- `FLOOR(DOUBLE) => DOUBLE`

LN

- `LN(DOUBLE) => DOUBLE`

LOG

- `LOG(DOUBLE, DOUBLE) => DOUBLE`

RAND

- `RAND() => DOUBLE`

ROUND

- `ROUND(INTEGER) => INTEGER`
- `ROUND(INTEGER, INTEGER) => INTEGER`
- `ROUND(BIGINT) => BIGINT`
- `ROUND(BIGINT, INTEGER) => BIGINT`
- `ROUND(DOUBLE) => DOUBLE`
- `ROUND(DOUBLE, INTEGER) => DOUBLE`

NDIV0

- `NDIV0(INTEGER, INTEGER) => INTEGER`
- `NDIV0(BIGINT, BIGINT) => BIGINT`
- `NDIV0(DOUBLE, DOUBLE) => DOUBLE`

POWER

- `POWER(DOUBLE, DOUBLE) => DOUBLE`

SIGN

- `SIGN(INTEGER) => INTEGER`
- `SIGN(BIGINT) => INTEGER`
- `SIGN(DOUBLE) => INTEGER`

SIN

- `SIN(DOUBLE) => DOUBLE`

SINH

- `SINH(DOUBLE) => DOUBLE`

SQRT

- `SQRT(DOUBLE) => DOUBLE`

TAN

- `TAN(DOUBLE) => DOUBLE`

TANH

- `TANH(DOUBLE) => DOUBLE`

UMINUS

- `UMINUS(INTEGER) => INTEGER`
- `UMINUS(BIGINT) => BIGINT`
- `UMINUS(DOUBLE) => DOUBLE`

String Functions

Concatenation

String concatenation is a binary expression that takes two expressions and a concatenation operator and produces a combined string value. Both expressions have to be of character string type.

`<concat_op_expr> ::= <expr> '||' <expr>`

Examples

```
NVARCHAR str1 = 'Basket';  
NVARCHAR str2 = 'ball';  
NVARCHAR str3 = :str1 || :str2;
```

Date and Time Functions

CURRENT_DATE

Returns the current local system DATE value. This function is an analogue to the homonymous function in SAP HANA SQL.

- `CURRENT_DATE()` => DATE

CURRENT_TIMESTAMP

Returns the current local system TIMESTAMP value. This function is an analogue to the homonymous function in HANA SQL.

It is recommended that you use UTC `TIMESTAMP` instead of local `TIMESTAMP`. See to the `CURRENT_UTCTIMESTAMP` function for more information.

- `CURRENT_TIMESTAMP()` => TIMESTAMP

CURRENT_UTCTIMESTAMP

Returns the current UTC `TIMESTAMP` value. This function is an analogue to the homonymous function in SAP HANA SQL.

- `CURRENT_UTCTIMESTAMP()` => TIMESTAMP

DATE

Creates a DATE value from the given character string. Throws a conversion error if the character string isn't a valid DATE value.

- `DATE(NVARCHAR)` => DATE

TIMESTAMP

Creates a TIMESTAMP value from the given character string. Throws a conversion error if the character string isn't a valid TIMESTAMP value.

- `TIMESTAMP(NVARCHAR)` => TIMESTAMP

NANOSECOND

Extracts the nanosecond component from a TIMESTAMP value.

- `NANOSECOND(TIMESTAMP)` => INTEGER

SECOND

Extracts the second component from a `TIMESTAMP` value.

- `SECOND(TIMESTAMP) => INTEGER`

MINUTE

Extracts the minute component from a `TIMESTAMP` value.

- `MINUTE(TIMESTAMP) => INTEGER`

HOUR

Extracts the hour component from a `TIMESTAMP` value.

- `HOUR(TIMESTAMP) => INTEGER`

DAYOFMONTH

Extracts the day-of-month component from a `DATE` or `TIMESTAMP` value.

- `DAYOFMONTH(DATE) => INTEGER`
- `DAYOFMONTH(TIMESTAMP) => INTEGER`

MONTH

Extracts the month component from a `DATE` or `TIMESTAMP` value.

- `MONTH(DATE) => INTEGER`
- `MONTH(TIMESTAMP) => INTEGER`

YEAR

Extracts the year component from a `DATE` or `TIMESTAMP` value.

- `YEAR(DATE) => INTEGER`
- `YEAR(TIMESTAMP) => INTEGER`

LAST_DAY

Returns the `DATE` or `TIMESTAMP` value of the last day of the month of the specified `DATE` or `TIMESTAMP` value.

- `LAST_DAY(DATE) => DATE`
- `LAST_DAY(TIMESTAMP) => TIMESTAMP`

ADD_NANO100

Computes the specified `TIMESTAMP` value, plus or minus the specified number of 0.1 microsecond units. The value to be added or subtracted needs to be a `BIGINT`.

- `ADD_NANO100(TIMESTAMP, BIGINT) => TIMESTAMP`

ADD_SECONDS

Computes the specified `TIMESTAMP` value, plus or minus the specified seconds. The value to be added or subtracted needs to be an `INTEGER` or a `BIGINT`.

- `ADD_SECONDS(TIMESTAMP, INTEGER) => TIMESTAMP`
- `ADD_SECONDS(TIMESTAMP, BIGINT) => TIMESTAMP`

ADD_MINUTES

Computes the specified `TIMESTAMP` value, plus or minus the specified minutes. The value to be added or subtracted needs to be an `INTEGER`.

- `ADD_MINUTES(TIMESTAMP, INTEGER) => TIMESTAMP`

ADD_HOURS

Computes the specified `TIMESTAMP` value, plus or minus the specified hours. The value to be added or subtracted needs to be an `INTEGER`.

- `ADD_HOURS(TIMESTAMP, INTEGER) => TIMESTAMP`

ADD_DAYS

Computes the specified `DATE` or `TIMESTAMP` value, plus or minus the specified days. The value to be added or subtracted needs to be an `INTEGER`.

- `ADD_DAYS(DATE, INTEGER) => DATE`
- `ADD_DAYS(TIMESTAMP, INTEGER) => TIMESTAMP`

ADD_MONTHS

Computes the specified `DATE` or `TIMESTAMP` value, plus or minus the specified months. The value to be added or subtracted needs to be an `INTEGER`.

- `ADD_MONTHS(DATE, INTEGER) => DATE`
- `ADD_MONTHS(TIMESTAMP, INTEGER) => TIMESTAMP`

ADD_YEARS

Computes the specified DATE or TIMESTAMP value, plus or minus the specified years. The value to be added or subtracted needs to be an INTEGER.

- `ADD_YEARS( DATE, INTEGER ) => DATE`
- `ADD_YEARS( TIMESTAMP, INTEGER ) => TIMESTAMP`

NANO100_BETWEEN

Computes the time difference between two specified TIMESTAMP values to the precision of 0.1 microseconds.

- `NANO100_BETWEEN( TIMESTAMP, TIMESTAMP ) => BIGINT`

SECONDS_BETWEEN

Computes the number of seconds between two specified TIMESTAMP values.

- `SECONDS_BETWEEN( TIMESTAMP, TIMESTAMP ) => BIGINT`

DAYS_BETWEEN

Computes the number of days between two specified DATE or TIMESTAMP values.

- `DAYS_BETWEEN( DATE, DATE ) => INTEGER`
- `DAYS_BETWEEN( TIMESTAMP, TIMESTAMP ) => INTEGER`

MONTHS_BETWEEN

Computes the number of months between two specified DATE or TIMESTAMP values.

- `MONTHS_BETWEEN( DATE, DATE ) => INTEGER`
- `MONTHS_BETWEEN( TIMESTAMP, TIMESTAMP ) => INTEGER`

YEARS_BETWEEN

Computes the number of years between two specified DATE or TIMESTAMP values.

- `YEARS_BETWEEN( DATE, DATE ) => INTEGER`
- `YEARS_BETWEEN( TIMESTAMP, TIMESTAMP ) => INTEGER`

Boolean Expressions

Relational Expressions

Relational expressions are binary expressions that compare the values produced by two expressions and return a `BOOLEAN` value. Both expressions must be of the same type. For `VERTEX`, `EDGE`, and `BOOLEAN` types, only equality and inequality comparisons are allowed. For membership testing, the right operand must be a multiset. Comparisons with `NULL` give a false result, that is, if both compared values are `NULL` and the comparison operator is equals, the result is false.

```
<relational_op> ::= = | != | < | > | <= | >= | IN | NOT IN
<relational_expr> ::= <expr> <relational_op> <expr>
```

Examples

```
:intValue > 3
(:doubleValue - 2.0 ) < 17
:v_current == VERTEX(:g, 'ID0001')
```

Logical Expressions

Logical expressions take two expressions and a logical operator and produce a Boolean result value. Both expressions have to be of the same type.

```
<logical_expr> ::= <expr> <logical_op> <expr> | NOT '(' <expr> ') '
<logical_op> ::= AND | OR
```

Examples

```
(3 == 4) AND (4 == 3)
TRUE OR (:e.WEIGHT > 3)
NOT (3 == 2)
```

IS NULL Expressions

The unary `IS NULL` and `IS NOT NULL` expressions allow you to check whether a scalar value or attribute have the value `NULL`. The return type of the expressions is a Boolean value.

```
<is_null_expression> ::= <expr> IS NULL
<is_not_null_expression> ::= <expr> IS NOT NULL
```

Examples

```
INTEGER i; -- default initialization to NULL
BOOLEAN is_i_initialized = :i IS NOT NULL;
```

5/24/26, 12:34 PM

```
BOOLEAN is_weight_attr_null = :v."weight" IS NULL;
```

Restrictions for GraphScript Procedures

Size Limits

The following table shows the maximum allowable limit for each entry.

Description	Limit
Maximum identifier length	127
Maximum number of temporary attributes per graph	64
Maximum number of variables	10000
Maximum number of graph variables	128
Maximum script length (in bytes)	2 GB

General Graph Workspace Limitations

- Caching isn't supported for graph workspaces that refer to virtual tables.
- The usage of cross-database access in a scenario with more than one tenant database in combination with SAP HANA Cloud property graph engine isn't supported.

Complex GraphScript Examples

GraphScript Procedure Example

The following example depicts a more complex example of a GraphScript procedure. It uses the Open Flights data set and computes the shortest path one-to-one between the airports of Philadelphia and Jinan.

```
CREATE TYPE "FLIGHTS"."TT_NODES_SPOO" AS TABLE ("NODE_KEY" NVARCHAR(100));
CREATE TYPE "FLIGHTS"."TT_EDGES_SPOO" AS TABLE (
    "EDGE_KEY" BIGINT, "SOURCE" NVARCHAR(100), "TARGET" NVARCHAR(100));

CREATE OR REPLACE PROCEDURE "FLIGHTS"."GS_SPOO"(
    IN i_startNode NVARCHAR(100),      -- the ID of the start node
    IN i_endNode NVARCHAR(100),        -- the ID of the end node
    OUT o_path_length BIGINT,          -- the hop distance between start and end
    OUT o_path_weight DOUBLE,          -- the path weight/cost based on the WEIGHT attribute
    OUT o_nodes "FLIGHTS"."TT_NODES_SPOO",
    OUT o_edges "FLIGHTS"."TT_EDGES_SPOO"
)
LANGUAGE GRAPH READS SQL DATA AS
BEGIN
    -- create an instance of the graph, referring to the graph workspace object
    GRAPH g = GRAPH("FLIGHTS", "GRAPH");
    INTEGER quot = INTEGER(2.1);
    -- create an instance of the start/end node
    VERTEX v_start = Vertex(:g, :i_startNode);
    VERTEX v_end = Vertex(:g, :i_endNode);
    -- running shortest path using the WEIGHT column as cost
    WeightedPath<DOUBLE> p = Shortest_Path(:g, :v_start, :v_end, (Edge e)
        => DOUBLE{ return :e."DIST_KM"/800.0; });
    -- project the results from the path
    o_path_length = LENGTH(:p);
    o_path_weight = WEIGHT(:p);
    o_nodes = SELECT :v."NODE_KEY" FOREACH v IN Vertices(:p);
    o_edges = SELECT :e."EDGE_KEY", :e."SOURCE", :e."TARGET" FOREACH e IN Edges(:p);
END;

CALL "FLIGHTS"."GS_SPOO"(
    i_startNode => 'PHL', i_endNode => 'TNA',
    o_path_length => ?, o_path_weight => ?, o_nodes => ?, o_edges => ?);
```

The following example uses the Open Flights data set and computes the 3 shortest paths between Philadelphia and Jinan. A public synonym "PUBLIC_SYNONYM_FLIGHTS_GRAPH" for the graph workspace "FLIGHTS"."GRAPH" is created and then used in GraphScript to instantiate the graph.

```
CREATE OR REPLACE PUBLIC SYNONYM "PUBLIC_SYNONYM_FLIGHTS_GRAPH" FOR "FLIGHTS"."GRAPH";

CREATE TYPE "FLIGHTS"."TT_PATHS_TKSP" AS TABLE (
    "PATH_ID" INT, "PATH_LENGTH" BIGINT, "PATH_WEIGHT" DOUBLE,
    "EDGE_KEY" BIGINT, "EDGE_ORDER" INT);

CREATE OR REPLACE PROCEDURE "FLIGHTS"."GS_TKSP"(
    IN i_startNode NVARCHAR(100),      -- the ID of the start node
    IN i_endNode NVARCHAR(100),        -- the ID of the end node
    IN i_k INT,                        -- the number of paths to be returned
    OUT o_paths "FLIGHTS"."TT_PATHS_TKSP"
)
LANGUAGE GRAPH READS SQL DATA AS
BEGIN
    -- create an instance of the graph using a public synonym for the graph workspace
    GRAPH g = Graph("PUBLIC_SYNONYM_FLIGHTS_GRAPH");
    -- create an instance of the start/end node
    VERTEX v_start = Vertex(:g, :i_startNode);
    VERTEX v_end = Vertex(:g, :i_endNode);
```

```

-- running top k shortest paths using the WEIGHT column as cost
SEQUENCE<WeightedPath<DOUBLE>> s_paths = K_Shortest_Paths(
  :g, :v_start, :v_end, :i_k, (Edge e) => DOUBLE{ return :e."DIST_KM"; });
-- project result paths into a table
BIGINT currentResultRow = 1L;
FOREACH result_path IN (:s_paths) WITH ORDINALITY AS path_id {
  FOREACH path_edge IN EDGES(:result_path) WITH ORDINALITY AS edge_order {
    o_paths."PATH_ID"[:currentResultRow] = INTEGER(:path_id);
    o_paths."PATH_LENGTH"[:currentResultRow] = Length(:result_path);
    o_paths."PATH_WEIGHT"[:currentResultRow] = Weight(:result_path);
    o_paths."EDGE_KEY"[:currentResultRow] = :path_edge."EDGE_KEY";
    o_paths."EDGE_ORDER"[:currentResultRow] = INTEGER(:edge_order);
    currentResultRow = :currentResultRow + 1L;
  }
}
END;

CALL "FLIGHTS"."GS_TKSP"(i_startNode => 'PHL', i_endNode => 'TNA', i_k => 3, o_paths => ?);

```

The following example uses the Employees graph based on collection and finds direct neighbors of the input vertex.

```

CREATE OR REPLACE PROCEDURE "GRAPH_SCHEMA"."GS_NEIGHBORS"(
  IN i_startNode BIGINT, -- the ID of the start node
  OUT o_nodes TABLE ("NODE_KEY" BIGINT)
)
LANGUAGE GRAPH READS SQL DATA AS
BEGIN
  -- create an instance of the graph, referring to the graph workspace object
  GRAPH g = GRAPH("GRAPH_SCHEMA", "EMPLOYEES_GRAPH");
  -- create a subgraph using a filter
  GRAPH sub_g = SUBGRAPH(:g, v IN Vertices(:g) WHERE TO_BIGINT(:v."id") < 4L);
  -- create an instance of the start node
  VERTEX v_start = Vertex(:sub_g, :i_startNode);
  -- running neighbors algorithm on start node
  MULTISET<VERTEX> neighborsVertex = NEIGHBORS(:sub_g, :v_start, -1, 1);
  -- project the results
  o_nodes = SELECT TO_BIGINT(:v."id") FOREACH v IN :neighborsVertex;
END;

```

GraphScript Anonymous Block Example

The following example uses the Company graph described in [Appendix A - Open Flights, Company and Employees Graph Examples](#) to determine all the employees who work on projects that produce products bought by a specific customer.

```

DO (IN CUSTOMER_ID BIGINT => 1, -- The key of the customer in the "CUSTOMERS" table
  OUT EMPLOYEES TABLE (FIRST_NAME NVARCHAR(100), LAST_NAME NVARCHAR(100)) => ?)
LANGUAGE GRAPH
BEGIN
  -- Create an instance of the graph, referring to the graph workspace object
  GRAPH g = GRAPH("COMPANY_SCHEMA", "COMPANY_GRAPH");
  -- Create a vertex which corresponds to the customer with the provided CUSTOMER_ID.
  -- For the vertex table "CUSTOMERS" no label has been explicitly defined.
  -- In this case, the label defaults to the name of the table.
  VERTEX v_customer = VERTEX(:g, 'CUSTOMERS', :CUSTOMER_ID);
  -- The maximum hop distance between any two vertices in the given "COMPANY_GRAPH" is three.
  -- Identify all the vertices reachable from the customer vertex regardless of the edge direction.
  MULTISET<VERTEX> m_neighbors = NEIGHBORS(:g, :v_customer, 1, 3, 'ANY');
  -- Filter the employees out of the reachable vertices.
  -- The employees are identified by the label 'Employee'.
  MULTISET<VERTEX> m_employees = v IN :m_neighbors where LABEL(:v) == 'Employee';
  -- Project the FIRST_NAME and the LAST_NAME of all found employees.
  EMPLOYEES = SELECT :v.FIRST_NAME, :v.LAST_NAME FOREACH v IN :m_employees;
END;

```

GraphScript Table User-Defined Function Example

An implementation based on table user-defined functions of the anonymous block example from above, which computes all the employees who work on projects that produce products bought by a specific customer, is shown below. A synonym "SYNONYM_COMPANY_GRAPH" for the graph workspace "COMPANY_SCHEMA"."COMPANY_GRAPH" is created in the schema "ANOTHER_SCHEMA" and then used in GraphScript to instantiate the graph.

```
DROP SCHEMA "ANOTHER_SCHEMA" CASCADE;
CREATE SCHEMA "ANOTHER_SCHEMA";
CREATE OR REPLACE SYNONYM "ANOTHER_SCHEMA"."SYNONYM_COMPANY_GRAPH" FOR "COMPANY_SCHEMA"."COMPANY_GRAPH";

CREATE OR REPLACE FUNCTION "COMPANY_SCHEMA"."CONTRIBUTING_EMPLOYEES" (
    CUSTOMER_ID BIGINT -- The key of the customer in the "CUSTOMERS" table
)
    RETURNS TABLE (FIRST_NAME NVARCHAR(100), LAST_NAME NVARCHAR(100))
LANGUAGE GRAPH AS
BEGIN
    GRAPH g = GRAPH("ANOTHER_SCHEMA", "SYNONYM_COMPANY_GRAPH");
    VERTEX v_customer = VERTEX(:g, 'CUSTOMERS', :CUSTOMER_ID);
    MULTISSET<VERTEX> m_neighbors = NEIGHBORS(:g, :v_customer, 1, 3, 'ANY');
    MULTISSET<VERTEX> m_employees = v in :m_neighbors where LABEL(:v) == 'Employee';
    RETURN SELECT :v.FIRST_NAME, :v.LAST_NAME FOREACH v IN :m_employees;
END;

SELECT * FROM "COMPANY_SCHEMA"."CONTRIBUTING_EMPLOYEES"(?) ORDER BY LAST_NAME, FIRST_NAME;
```

openCypher Pattern Matching

openCypher is a declarative graph query language for graph pattern matching developed by the openCypher Implementers Group. (Cypher is a registered trademark of Neo4j, Inc.)

SAP HANA Cloud property graph engine allows you to use openCypher directly in SQL. This topic describes the SQL interface and the currently supported subset of the openCypher query language.

OPENCYPHER_TABLE SQL Function

The OPENCYPHER_TABLE SQL function enables the embedding of an openCypher query in an SQL query. The result is returned as a table. The column names and types of the resulting table are determined by both the [openCypher Query Language](#) and the [Graph Workspaces](#).

OPENCYPHER_TABLE is robust with respect to inconsistent graph workspaces. Errors can occur in rare cases, but usually the inconsistent parts don't show in the result.

Syntax

```
OPENCYPHER_TABLE (
    <graph_workspace_spec>
    <opencypher_query_spec>
    [<parameters_spec>]
)
```

Syntax Elements

<graph_workspace_spec>

Specifies the graph workspace used with the openCypher query.

```
<graph_workspace_spec> ::= GRAPH WORKSPACE <graph_workspace>
```

<graph_workspace> can be any graph workspace present in the system, identified by an optional schema name and a workspace name.

<opencypher_query_spec>

Specifies a query in the openCypher language.

```
<opencypher_query_spec> ::= QUERY <opencypher_query_string>
```

<opencypher_query_string> is a string or nstring literal that corresponds to a pattern matching query in the openCypher language. Although <opencypher_query_string> appears to be a normal SQL string, it's actually part of the query syntax and can't be replaced with a string expression like most SQL strings. To parameterize it, dynamic SQL has to be used.

<parameters_spec>

Specifies the parameters used with the openCypher query.

```
<parameters_spec> ::= PARAMETERS <l_paren> <parameters> <r_paren>
```

```
<parameters> ::= <parameter>
| <parameters> <comma> <parameter>
```

<parameter> ::= <single_quote> <parameter_name> <single_quote> <equals> <questionmark>

The parameters within the query specification have to be preceded by a dollar sign.

i Note

Since a single-quoted SQL string is used for the openCypher query, single quotes (') inside the openCypher query have to be escaped by using two single quotes (").

Examples

The example is based on the Open Flights graph from section [Appendix A - Open Flights Graph Example](#).

```
SELECT DISTINCT * FROM OPENCYPHER_TABLE( GRAPH WORKSPACE "FLIGHTS"."GRAPH" QUERY
MATCH (a)-[e]-(b)
WHERE a.NAME = ''John F Kennedy International Airport'' RETURN b.NAME AS Name
ORDER BY b.NAME
) ORDER BY "Name"
```

Name
Amsterdam Airport Schiphol
Dubai International Airport
Frankfurt am Main Airport
Indira Gandhi International Airport
Philadelphia International Airport

This example shows how to use parameters in the query above.

```
SELECT DISTINCT * FROM OPENCYPHER_TABLE( GRAPH WORKSPACE "FLIGHTS"."GRAPH" QUERY
MATCH (a)-[e]-(b)
WHERE a.NAME = $airport RETURN b.NAME AS Name
ORDER BY b.NAME
PARAMETERS ('airport' = ?)
)
```

Related Information

- [openCypher Query Language](#)
- [Graph Workspaces](#)
- [Appendix A - Open Flights, Company and Employees Graph Examples](#)

openCypher Query Language

An openCypher query searches for subgraphs that match the specification given in **MATCH** clauses and returns a result table as specified in the **RETURN** clause. If the graph workspace consists of multiple vertex or edge tables, then the query result may contain columns, which are not present in all the vertex or edge tables, respectively. Missing entries are filled with **NULL** values.

```
<opencypher_query> ::= <match_clauses> <return_clause>
```

```
<match_clauses> ::= <match_clause> [{match_clause}]
```

The total number of **MATCH** clauses in one openCypher query is currently limited to five. OpenCypher expressions can include vertex or edge attributes only of the following types: **INT**, **BIGINT**, **DOUBLE**, **NVARCHAR**, **DATE**, and **TIMESTAMP**. Vertex or edge attributes of other types can be used in the **RETURN**-clause, but not within aggregation functions or other expressions.

MATCH Clause

The MATCH clause consists of topological constraints that define the topology (vertices, edges, and paths) of the matching subgraphs, and an optional filter condition, which allows the user to add additional (non-topological) constraints. These constraints must be fulfilled by each match in the graph workspace to become a part of the result.

```
<match_clause> ::= MATCH <topology_constraints>
    [WHERE <non_topology_constraints>]
<topology_constraints> ::= <topology_constraint>
    [{<comma> <topology_constraint>}]
```

Topological Constraints

Topological constraints are a comma-separated list of vertices and edges. Additionally, one path can be used.

```
<topology_constraint> ::=
<vertex>
| <edge>
| <path>
```

Pattern for vertices

The simplest pattern consists of a single vertex variable enclosed in parentheses. Additionally, a vertex label can also be provided.

```
<vertex> ::= <l_paren> <variable_name> <r_paren>
| <l_paren> <variable_name> <colon> <label_name> <r_paren>
```

```
(a)
(a:vertex_label)
```

The following query illustrates the pattern with a vertex label:

```
MATCH (a:label) RETURN a.NAME AS name
```

Pattern for related vertices

A pattern can consist of two vertices and an edge connecting them. In the following example, vertices a and b are connected with an edge e. The direction of the relationship between the vertices can be specified using an arrow ->. Additionally, an edge label can also be provided.

```
<edge> ::= <source> <directed_edge> <target>
| <source> <undirected_edge> <target>
<directed_edge> ::= <minus> <edge_var> <minus> <greater>
<undirected_edge> ::= <minus> <edge_var> <minus>
<vertex_var> ::= <l_paren> <variable_name> <r_paren>
<edge_var> ::= <l_square> <variable_name> <r_square>
| <l_square> <variable_name> <colon> <label_name> <r_square>
```

```
(a)-[e]-(b)          -- any edge
(a)-[e]->(b)         -- directed edge
```

(a) - [e:edge_label] - (b) -- edge with label

The following query illustrates the use of an undirected edge:

```
MATCH (a) - [e] - (b)
WHERE a.NAME = 'John F Kennedy International Airport' RETURN b.NAME AS Name
ORDER BY b.NAME
```

The result of this query is the following table:

Name
Amsterdam Airport Schiphol
Amsterdam Airport Schiphol
Dubai International Airport
Dubai International Airport
Frankfurt am Main Airport
Frankfurt am Main Airport
Indira Gandhi International Airport
Indira Gandhi International Airport
Philadelphia International Airport
Philadelphia International Airport

The following query illustrates the use of multiple MATCH clauses:

```
MATCH (a) - [e1] -> (b)
MATCH (b) - [e2] -> (c)
WHERE a.NAME = 'Philadelphia International Airport' RETURN c.NAME AS Name
ORDER BY c.NAME
```

This query returns all vertices laying at a distance of two hops from Philadelphia International Airport, considering the direction of edges:

Name
Amsterdam Airport Schiphol
Dubai International Airport
Frankfurt am Main Airport
Indira Gandhi International Airport
Philadelphia International Airport

i Note

Note that Philadelphia International Airport is also in the results, which means that edge variables e1 and e2 match the same edge in the data graph. This would not be possible if e1 and e2 are defined in the same MATCH clause.

Pattern for Paths

A pattern for path connects two vertices which are a variable number of hops away. The length of the path must be specified using an asterisk, followed by the minimum path length, double dots and the maximum path length

[*minimum_length..maximum_length]. The minimal and maximal length of a path must be in the range of 1 to 15. It is mandatory to assign this pattern to a path variable. Additionally, the vertex labels and an edge label can also be provided.

```
<path> ::= <variable_name> <equal> <source> <directed_path> <target>
| <variable_name> <equal> <source> <undirected_path> <target>
<directed_path> ::= <minus> <l_square> <asterisk> <uint> <double_dot> <uint>
<r_square> <minus> <greater>
| <minus> <l_square> <colon> <label_name> <asterisk> <uint> <double_dot> <uint>
<r_square> <minus> <greater>
<undirected_path> ::= <minus> <l_square> <asterisk> <uint> <double_dot> <uint>
<r_square> <minus>
| <minus> <l_square> <colon> <label_name> <asterisk> <uint> <double_dot> <uint>
<r_square> <minus>
```

```
MATCH p = (a)-[*1..2]->(b)
MATCH p = (a)-[:edge_label*1..2]->(b:vertex_label)
```

The following query illustrates the use of a directed path:

```
MATCH p = (a)-[*1..2]->(b)
WHERE a.NAME = 'Philadelphia International Airport'
RETURN b.NAME AS Name
ORDER BY b.NAME
```

The path with the name p starts from vertex a and ends at vertex b. It is one or two edges long. This query returns all vertices that are related to Philadelphia International Airport by outgoing edges. The result is:

Name
Amsterdam Airport Schiphol
Dubai International Airport
Frankfurt am Main Airport
Indira Gandhi International Airport
John F Kennedy International Airport
Philadelphia International Airport

The following query illustrates the use of vertex labels and an edge label in a variable-length path pattern, applied on a heterogenous graph COMPANY_GRAPH:

```
MATCH p = (v1:Project)-[:Assignments*1..2]-(v2:Employee)
RETURN v1.NAME AS Project_Name, [e IN RELATIONSHIPS(p) | e.PROJECT_ID] AS Project_ID,
v2.FIRST_NAME AS Employee_Name
ORDER BY v1.NAME')
```

The above query returns the project vertices, the project IDs and the employees working on these projects.

Project_Name	Project_ID	Employee_Name
DWC	[2]	Maria
DWC	[2]	John
HANA	[1]	Paul
HANA	[1]	John

Non-Topological Constraints

non-topological constraints is a Boolean expression that is evaluated on each matched subgraph. This expression can contain many predicates that are combined by the logical connectives AND, OR, and NOT.

```
<non_topology_constraints> ::= <condition>
```

```
<condition> ::= <condition> OR <condition>
| <condition> AND <condition>
| NOT <condition>
| <builtin_function>
| <l_paren> <condition> <r_paren>
| <predicate>
```

Predicates include comparisons (=, <, >, <=, >=, <=>), IS NULL, and other Boolean built-in functions.

```
<predicate> ::= <comparison_predicate>
| <attribute access> IS NULL
| <attribute access> CONTAINS <nvarchar constant>
| <attribute access> STARTS WITH <nvarchar constant>
| <attribute access> ENDS WITH <nvarchar constant>

<comparison_predicate> ::=
  <expression> <equals> <expression>
| <expression> <unequal> <expression>
| <expression> <greater> <expression>
| <expression> <smaller> <expression>
| <expression> <greater_equal> <expression>
| <expression> <smaller_equal> <expression>

<expression> ::=
  <attribute access>
| <constant>
| <parameter>
| <expression> <operator> <expression>
<attribute access> ::= <variable_name> <dot> <attribute_name>
<variable_name> ::= <identifier>
<attribute_name> ::= <identifier>
<label_name> ::= <identifier>
<constant> ::= <integer_constant>
| <bigint_constant>
| <double_constant>
| <varchar_constant>
<parameter> ::= <dollar_sign> <identifier>
<operator> ::= <plus>
| <minus>
| <asterix>
| <slash>
| <modulo>
| <circumflex>
```

For numerical expressions the operators correspond to the common arithmetic functions. For string values the plus operator corresponds to the string concatenation. The following query illustrates string concatenation and the use of the logical connective NOT. Note that NOT has higher precedence than AND and OR. Therefore, NOT is evaluated first.

```
MATCH (a)-[e]->(b)
WHERE a.NAME = ''John F Kennedy'' + '' International Airport''
      AND (e.EDGE_KEY < 4 OR NOT b.NAME <> ''Philadelphia International Airport'')
RETURN b.NAME AS Name
ORDER BY b.NAME
```

The following query illustrates the use of the logical connective OR. Note that OR has lower precedence than AND.

```
MATCH (a)-[e]->(b)
WHERE a.NAME = 'John F Kennedy International Airport'
      AND e.EDGE_KEY > 4 OR b.NAME = 'Philadelphia International Airport'
RETURN b.NAME AS Name
ORDER BY b.NAME
```

The following query illustrates the use of parentheses to evaluate the OR connective before the AND connective.

```
MATCH (a)-[e]->(b)
WHERE a.NAME = ''John F Kennedy International Airport''
      AND (e.EDGE_KEY > 4 OR b.NAME = ''Philadelphia International Airport'')
RETURN b.NAME AS Name
ORDER BY b.NAME
```

Name
Amsterdam Airport Schiphol
Dubai International Airport
Indira Gandhi International Airport
Philadelphia International Airport

The following query illustrates the use of string-matching predicates for evaluating international airports.

```
MATCH (a)
WHERE a.NAME CONTAINS 'International'
RETURN a.NAME AS Name
ORDER BY a.NAME
```

Name
Beijing Capital International Airport
Dubai International Airport
Indira Gandhi International Airport
John F Kennedy International Airport
Philadelphia International Airport

RETURN Clause

The RETURN clause lists expressions over the matched subgraphs that need to be projected into the resulting table. The optional DISTINCT specifier eliminates duplicate rows from the resulting table. The optional ORDER BY clause allows you to sort the resulting rows in ascending or descending order. The optional LIMIT clause of the RETURN clause truncates the resulting table to the given number of rows (from the top) if the result has more rows than the given number. Otherwise, all the rows are displayed. The optional SKIP clause excludes the given number of rows from the top of the result. When all three optional clauses are present, the ORDER-BY clause is applied first, then the SKIP clause, and finally the LIMIT clause.

```

<return_clause> ::= RETURN [ DISTINCT ] <return_list>
                  [<order_by_clause>] [<limit_skip_clause>]

<return_list> ::= <return_item> [{<comma> <return_item>}]
<return_item> ::= <return_expression> AS <alias>
<alias> ::= <identifier>

<return_expression ::=
    <attribute_access>
    | <string_concatenation>
    | <aggregate_function> <l_paren> <attribute_access> <r_paren>
    | <list_comprehension>
    | <object_constructor>

<aggregate_function> ::= COUNT | MAX | MIN | SUM
<order_by_clause> ::= ORDER BY <order_by_list>
<order_by_list> ::= <order_by_item> [{<comma> <order_by_item>}]
<order_by_item> ::= <variable_name> <dot> <attribute_name> [ ASC | DESC ]

<list_comprehension> ::= <l_square> <variable_name> IN
    <relationships_function> <pipe> <expression> <r_square>
<object_constructor> ::= <l_curly> <identifier> <colon> <expr>
    [{<comma> <identifier> <colon> <expr>}] <r_curly>

<limit_skip_clause> ::= LIMIT <uint> [ SKIP <uint> ]

```

If there's more than one return item and at least one aggregation, the result is grouped by the not aggregated return items. The MAX, MIN, and SUM functions need a numeric type as input and the result type corresponds to the input type. The COUNT function can take any type as input and the result type is always BIGINT. MAX(NULL) and MIN(NULL) return NULL, whereas COUNT(NULL) and SUM(NULL) return 0.

An example with a complex return clause is:

```

MATCH (a)-[e]->(b)
WHERE e.DIST KM > 5000.0
RETURN a.NAME AS name, COUNT(b.NAME) AS numLongDistanceConnections
ORDER BY a.NAME ASC
LIMIT 5
SKIP 1

```

List comprehension is useful for extracting edge attributes of a path. List comprehension constructs a new list from the given path function, returning a list of edges and the given expression that can use the given iterator variable. It's only permitted in the RETURN clause. The resulting column is a string representation of the list comprehension result in JSON format. The following expressions are allowed in the list comprehension: attribute accesses, string concatenations, and constants. The following example shows how to use the list comprehension.

```

MATCH p = (a)-[*1..2]->(b)
WHERE a.NAME STARTS WITH 'Philadelphia'
RETURN [e IN RELATIONSHIPS(p) | e.TARGET] AS result

```

The result is:

result
["JFK"]
["JFK", "FRA"]
["JFK", "AMS"]
["JFK", "DXB"]
["JFK", "DEL"]
["JFK", "PHL"]

The object constructor can group several expressions into one expression. It's only permitted in the RETURN clause. The resulting column is a string representation of the object in JSON format. The following example shows how to use the object constructor.

```
MATCH (a) - [e] -> (b)
WHERE a.NAME STARTS WITH 'Dubai'
RETURN { from: e.SOURCE, to: e.TARGET } AS result
```

The result is:

result
{"from":"DXB","to":"FRA"}
{"from":"DXB","to":"AMS"}
{"from":"DXB","to":"PEK"}
{"from":"DXB","to":"JFK"}
{"from":"DXB","to":"DEL"}

The following openCypher query returns the values of the "NAME" attribute of all vertices in the given graph.

```
MATCH (a)
RETURN a.NAME AS name
ORDER BY a.NAME ASC
```

The result of this query is a table containing 8 rows:

Name
Amsterdam Airport Schiphol
Beijing Capital International Airport
Dubai International Airport
Frankfurt am Main Airport
Indira Gandhi International Airport

Name
John F Kennedy International Airport
Philadelphia International Airport
Yaoqiang Airport

The following query illustrates the use of the LIMIT clause applied to the result from the previous example:

```
MATCH (a)
RETURN a.NAME AS name
ORDER BY a.NAME ASC
LIMIT 5
```

The result of this query is a table containing 5 rows:

Name
Amsterdam Airport Schiphol
Beijing Capital International Airport
Dubai International Airport
Frankfurt am Main Airport
Indira Gandhi International Airport

The following query illustrates the use of the SKIP clause applied to the result from the previous example:

```
MATCH (a)
RETURN a.NAME AS name
ORDER BY a.NAME ASC
LIMIT 5
SKIP 1
```

The result of this query is the following table:

Name
Beijing Capital International Airport
Dubai International Airport
Frankfurt am Main Airport
Indira Gandhi International Airport
John F Kennedy International Airport

Keywords

All keywords are case insensitive.

Keywords

MATCH	WHERE	OR	AND	NOT
ALL	IN	RETURN	AS	ORDER
BY	ASC	DESC	LIMIT	SKIP
WITH	CONTAINS	STARTS	ENDS	SUM
AVG	COUNT	MAX	MIN	

Reserved Keywords

optional	distinct	unique	detach	union
unwind	merge	create	set	delete
remove	on	ascending	descending	xor
null	true	false	case	when
then	else	end	foreach	call
constraint	assert	load	csv	headers
fieldterminator	index	yield	using	drop
profile	explain	start	is	

Attribute Names, Variable Names, and Aliases

Attribute names, variable names, and aliases are case-sensitive.

The following query illustrates the case sensitivity of variable names (a and A aren't the same).

```
MATCH (a), (A)
RETURN a.NAME AS name
```

This query also illustrates a disconnected graph pattern, since vertices a and A aren't connected by any edge. In other words, there are two connected components, each of them containing one vertex. From a performance perspective, we recommend that you avoid matching disconnected subgraphs wherever possible, because the result is the Cartesian product of matches of all connected components (225 rows), which can easily get large and use up system resources.

The following query illustrates the case sensitivity of aliases and returns the values of the "NAME" attribute of all pairs of vertices in the given graph connected by an edge.

```
MATCH (A) - [e] -> (a)
RETURN a.NAME AS name, A.NAME as NAME
ORDER BY a.NAME ASC, A.NAME DESC
```

Built-In Functions

SAP HANA Cloud property graph engine supports a subset of openCypher built-in functions and offers a set of SAP HANA-specific built-in functions.

```
<builtin_function> ::= <opencypher_builtin_function> | <hana_builtin_function>
```

openCypher built-in functions are case-insensitive. SAP HANA-specific built-in functions are defined in the SYS namespace and are case-sensitive.

```
<hana_builtin_function> ::= SYS '.' <text_contains_function>
```

```
<opencypher_builtin_function> ::= <all_function> | <relationships_function>
```

TOBOOLEAN

The `TOBOOLEAN()` function takes an integer, boolean or a string value and converts it to a boolean value.

```
<toboolen_function> ::= TOBOOLEAN '(' <variable_name> '.' <attribute_name> ')'
```

`TOBOOLEAN(NULL)` returns NULL. An error is thrown if the parsing fails.

TOFLOAT

The `TOFLOAT()` function takes an integer, floating point or a string value and converts it to a floating point number.

```
<tofloat_function> ::= TOFLOAT '(' <variable_name> '.' <attribute_name> ')'
```

`TOFLOAT(NULL)` returns NULL. An error is thrown if the parsing fails.

TOINTEGER

The `TOINTEGER()` function takes a boolean, integer, floating point or a string value and converts it to an integer.

```
<tointeger_function> ::= TOINTEGER '(' <variable_name> '.' <attribute_name> ')'
```

`TOINTEGER(NULL)` returns NULL. An error is thrown if the parsing fails.

The following example shows how to use the `TOINTEGER` function.

```
MATCH (a)-[e]->(b)
WHERE a.NODE_KEY = 'FRA' AND b.NODE_KEY = 'PEK'
RETURN TOINTEGER(e.DIST_KM) as Distance
```

Distance

Distance
7808

TOSTRING

The `TOSTRING()` function takes a boolean, integer, floating point, date, time or a string value and converts it to a string.

```
<tostring_function> ::= TOSTRING '(' <variable_name> '.' <attribute_name> ')'
```

`TOSTRING(NULL)` returns `NULL`. An error is thrown if the parsing fails.

DATETIME

The `DATETIME()` function takes a string value in `YYYY-MM-DD HH24:MI:SS.FF7` format and converts it to a datetime value with year, month, day, hour, minute, seconds and fractional seconds component values.

```
<datetime_function> ::= DATETIME '(' <variable_name> '.' <attribute_name> ')'
```

`DATETIME(NULL)` returns `NULL`. An error is thrown if the parsing fails.

DATE

The `DATE()` function takes a string value in `YYYY-MM-DD` format and converts it to date value with year, month and day component values.

```
<datetime_function> ::= DATETIME '(' <variable_name> '.' <attribute_name> ')'
```

`DATE(NULL)` returns `NULL`. An error is thrown if the parsing fails.

RELATIONSHIPS

The `RELATIONSHIPS` function takes a path variable and returns a collection of all edges in the path.

```
<relationships_function> ::= RELATIONSHIPS '(' <variable_name> ')'
```

ALL

The `ALL` function returns a Boolean result and can be used as a predicate in a `WHERE` clause. This function returns true if the given condition is true for all elements in the given collection. Otherwise, it returns false.

```
<all_function> ::= ALL '('
    <variable_name> IN <relationships_function> WHERE <condition> ')'
```

The following example shows how to use the `ALL` function. If you're only interested in Dubai's direct and indirect ultra-long range connections, you can use an additional edge filter for all edges of a path.

```
MATCH p = (a)-[*1..2]->(b)
WHERE a.NAME STARTS WITH ''Dubai'' AND ALL(e IN RELATIONSHIPS(p) WHERE e.DIST_KM > 8000.0)
RETURN DISTINCT b.NAME AS Name
ORDER BY b.NAME
```

The result is:

Name
Dubai International Airport
Indira Gandhi International Airport
John F Kennedy International Airport

LABEL

The LABEL function takes a vertex or an edge variable and returns the label of that variable. If no label is specified for the vertex or edge table in the graph workspace, then the name of the table is returned.

```
<label_function> ::= LABEL '(' <variable_name> ')'
```

The following query illustrates the use of label function on a vertex variable of the heterogenous graph COMPANY_GRAPH:

```
MATCH (v)
WHERE v.NAME = ''SAP HANA Cloud''
RETURN LABEL(v) as Label
```

Label
Product

The following query illustrates the use of label function on an edge variable of the heterogenous graph COMPANY_GRAPH:

```
MATCH (v1:Project)-[e]-(v2:Employee)
WHERE v1.NAME = ''HANA'' OR v1.NAME = ''DWC''
RETURN v1.NAME as Project_Name, LABEL(e) as Label, v2.FIRST_NAME AS Employee_Name
ORDER BY v1.NAME
```

Project_Name	Label	Employee_Name
DWC	Assignments	Maria
DWC	Assignments	John
HANA	Assignments	Paul
HANA	Assignments	John

SYS.TEXT_CONTAINS

The function SYS.TEXT_CONTAINS makes advanced text search capabilities available in openCypher.

```
<text_contains_function> ::= TEXT_CONTAINS '('
  <variable_name> '.' <attribute_name> ',' <varchar_constant> ',' <varchar_constant> ')'
```

This function returns a Boolean result and can be used as a predicate in a WHERE clause of an openCypher query.

The first parameter is a vertex or directed edge attribute of type NVARCHAR. The second parameter is a string literal that specifies the search pattern, and the third parameter is a string literal specifying one of the two available search modes: EXACT or FUZZY. The search pattern can contain reserved operators (-,?,",*,OR,%) that have special meanings, depending on the search mode.

Here are two examples of how to apply the TEXT_CONTAINS function:

```
MATCH (a)
WHERE SYS.TEXT_CONTAINS(a.name, 'Dubbai', 'FUZZY(0.8)')
RETURN a.id
```

```
MATCH (a) - [e] -> (b)
WHERE SYS.TEXT_CONTAINS(e.color, 'b*', 'EXACT')
RETURN a.id
```

Additionally, the user can create a full-text index for any NVARCHAR column in the vertex or edge table to transform the search into a full-text search. More information about the full-text search and how to create a full-text index can be found in the [SAP HANA Cloud Search Developer Guide](#) and in the [SAP HANA Cloud SQL Reference Guide](#).

There are the following limitations:

- Undirected edges aren't supported.
- The function only supports graph workspaces on top of column store tables.
- The search pattern must not contain single quotes or brackets.

Exact search

In the EXACT search mode, the search term must match the entire search pattern to return a column entry as a match. In this search mode, the user can specify the search pattern as a complex predicate using the following specific operators. Some operators are useful only in a full-text search.

- With a minus sign (-), SAP HANA searches in columns for matches that don't contain the term immediately following the minus sign.
- The question mark (?) replaces a single character in a search term (for example, cat? would match cats).
- Terms within the quotation marks (" ") aren't tokenized and are handled as a string. Therefore, all search matches must be exact.
- The or-operator (OR) matches contain at least one of the terms joined by the OR operator.
- The asterisk sign (*) replaces 0 or more characters in a search term (for example, cat* would match cats and catalogs).
- The space operator () matches contain both of the terms joined by a space operator.

Examples for complex search patterns in the EXACT search mode are:

```
SYS.TEXT_CONTAINS(a.color, 'red', 'EXACT')
SYS.TEXT_CONTAINS(a.color, '-red', 'EXACT')
SYS.TEXT_CONTAINS(a.color, 'red*', 'EXACT')
SYS.TEXT_CONTAINS(a.color, '?ed', 'EXACT')
```

```
SYS.TEXT_CONTAINS(a.color, 'red OR blue', 'EXACT')  
SYS.TEXT_CONTAINS(a.color, 'red blue', 'EXACT')  
SYS.TEXT_CONTAINS(a.color, '"red blue"', 'EXACT')
```

Fuzzy search

Fuzzy search is a fast and fault-tolerant search feature in SAP HANA. A fuzzy search returns records even if the search term contains additional or missing characters or other types of spelling errors. The first parameter, the fuzzy score, defines how different the search pattern can be. It is a floating-point number from 0 through 1 and the larger the number, the less difference is allowed. Moreover, the fuzzy search mode takes many additional parameters that define how the fuzzy score is calculated.

```
SYS.TEXT_CONTAINS(a.name, 'Dubai', 'FUZZY(0.8)')  
SYS.TEXT_CONTAINS(a.name, 'Dubai', 'FUZZY(0.8, option1=value1, option2=value2)')
```

A fuzzy search for 'Dubai' with an appropriate fuzzy score also returns vertices containing the word 'Dubbai' or 'Dubay'. This provides the user the possibility to query dirty data, for example, querying misspelled names.

The following table shows a few additional options to influence fuzzy search results. Many options conflict with each other or need to be combined with different options. For more detailed information about these options and additional options, see the [SAP HANA Cloud Search Developer Guide](#).

OPTIONS	VALUES	DESCRIPTION
similarCalculationMode	typeahead, symmetricsearch, substringsearch, searchcompare, search, compare	Defines the impact of wrong characters, additional characters in search patterns, and additional characters in data for the fuzzy score
termMappingTable	unquoted sql identifier	Defines terms that are used to extend a search to generate additional results
searchMode	alphanumeric, housenumber, postcode	Searches for special formats

Related Information

[SAP HANA Cloud, SAP HANA Database Search Developer Guide](#)

[SAP HANA Cloud, SAP HANA Database SQL Reference Guide](#)

Basic Building Blocks

Basic Rules

```

<identifier> ::= <simple_identifier> | <special_identifier>

<simple_identifier> ::= (<letter> | <underscore>)
    {<letter> | <underscore> | <digit>}

<special_identifier> ::=
<backtick> { (<any_char> - <backtick>) | <backtick><backtick> }<backtick>

<integer_constant>::
<uint>
| <minus> <uint>

<bigint_constant> ::= <uint>
| <minus> <uint>

<double_constant> ::= ( <zero> | <pos_digit> {<digit>} ) <dot> <digit> [{<digit>}]
| <minus> ( <zero> | <pos_digit> {<digit>} ) <dot> <digit> [{<digit>}]

<uint> ::= <zero>
| <positive_digit> [{<digit>}]

<varchar_constant> ::= <single_quote> { ( <any_char> -
    ( <single_quote> | <backslash> ) ) | <escaped_char> } <single_quote>

<escaped_character> ::= <backslash> <single_quote>
| <backslash> <backslash>

```

BNF Terminal Symbols

```

<backtick> ::= `
<dot> ::= .
<double_dot> ::= ..
<comma> ::= ,
<minus> ::= -
<l_paren> ::= (
<r_paren> ::= )
<l_curly> ::= {
<r_curly> ::= }
<l_square> ::= [
<r_square> ::= ]
<greater> ::= >
<equal> ::= =
<lower> ::= <
<lower_equal> ::= <=
<greater> ::= >
<greater_equal> ::= >=
<unequal> ::= <>
<asterisk> ::= *
<single_quote> ::= '
<double_quote> ::= "
<backslash> ::= \
<zero> ::= 0
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<positive_digit> ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<letter> ::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q |
    r | s | t | u | v | w | x | y | z | A | B | C | D | E | F | G | H | I | J |
    K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z
<any_char> ::= any character !!

```

Additional Information

This section provides additional information about syntax and examples.

Appendix A - Open Flights, Company and Employees Graph Examples

Use the following SQL statements to create the Open Flights, Company and Employees graph examples.

Open Flights Graph Example

Use the following SQL statements to create the Open Flights graph example.

Create Vertex and Edge Tables (Open Flights)

```
CREATE SCHEMA "FLIGHTS";
SET SCHEMA "FLIGHTS";

CREATE TABLE "NODES" (
  "NODE_KEY" NVARCHAR(100) PRIMARY KEY,
  "TYPE" NVARCHAR(100),
  "NAME" NVARCHAR(100),
  "CITY" NVARCHAR(100),
  "COUNTRY" NVARCHAR(100),
  "LOCATION" ST_GEOMETRY(4326)
);

CREATE TABLE "EDGES" (
  "EDGE_KEY" BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  "SOURCE" NVARCHAR(100) NOT NULL REFERENCES "NODES" ("NODE_KEY")
    ON UPDATE CASCADE ON DELETE CASCADE,
  "TARGET" NVARCHAR(100) NOT NULL REFERENCES "NODES" ("NODE_KEY")
    ON UPDATE CASCADE ON DELETE CASCADE,
  "DIST_KM" DOUBLE,
  "PASSENGER_CAPACITY" INTEGER
);
```

Create Graph Workspace (Open Flights)

```
CREATE GRAPH WORKSPACE GRAPH
  EDGE TABLE EDGES
    KEY EDGE_KEY
    SOURCE SOURCE
    TARGET TARGET
  VERTEX TABLE NODES
    KEY NODE_KEY;
```

Insert Rows into Vertex Table (Open Flights)

```
INSERT INTO "NODES" VALUES (
  'FRA', 'Airport', 'Frankfurt am Main Airport', 'Frankfurt am Main', 'Germany',
  ST_GEOMFROMTEXT('POINT(8.570556 50.033333)', 4326));
INSERT INTO "NODES" VALUES (
  'JFK', 'Airport', 'John F Kennedy International Airport', 'New York', 'United States',
  ST_GEOMFROMTEXT('POINT(-73.77890015 -73.77890015)', 4326));
INSERT INTO "NODES" VALUES (
  'DXB', 'Airport', 'Dubai International Airport', 'Dubai', 'United Arab Emirates',
  ST_GEOMFROMTEXT('POINT(55.3643989563 25.2527999878)', 4326));
INSERT INTO "NODES" VALUES (
```

```

'AMS', 'Airport', 'Amsterdam Airport Schiphol', 'Amsterdam', 'Netherlands',
ST_GEOMFROMTEXT('POINT(4.76389 52.308601)', 4326));
INSERT INTO "NODES" VALUES (
'PEK', 'Airport', 'Beijing Capital International Airport', 'Beijing', 'China',
ST_GEOMFROMTEXT('POINT(116.58499908447266 40.080101013183594)', 4326));
INSERT INTO "NODES" VALUES (
'DEL', 'Airport', 'Indira Gandhi International Airport', 'Delhi', 'India',
ST_GEOMFROMTEXT('POINT(77.103104 28.5665)', 4326));
INSERT INTO "NODES" VALUES (
'PHL', 'Airport', 'Philadelphia International Airport', 'Philadelphia', 'United States',
ST_GEOMFROMTEXT('POINT(-75.24109649658203 -75.24109649658203)', 4326));
INSERT INTO "NODES" VALUES (
'TNA', 'Airport', 'Yaoqiang Airport', 'Jinan', 'China',
ST_GEOMFROMTEXT('POINT(117.21600341796875 36.857200622558594)', 4326));

```

Insert Rows into Edge Table (Open Flights)

```

INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('FRA', 'PEK', 7808.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('DEL', 'FRA', 6131.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('JFK', 'FRA', 6206.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('PEK', 'FRA', 7808.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('AMS', 'FRA', 367.2
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('DXB', 'FRA', 4848.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('FRA', 'JFK', 6206.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('JFK', 'AMS', 5863.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('PEK', 'AMS', 7848.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('DXB', 'AMS', 5174.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('DEL', 'AMS', 6375.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('FRA', 'AMS', 367.2
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('FRA', 'DXB', 4848.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('FRA', 'DEL', 6131.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('DEL', 'PEK', 3815.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('DXB', 'PEK', 5855.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('AMS', 'PEK', 7848.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('DEL', 'DXB', 2187.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('PEK', 'DXB', 5855.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('AMS', 'DXB', 5174.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('JFK', 'DXB', 1102.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('DEL', 'JFK', 1177.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('AMS', 'JFK', 5863.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('DXB', 'JFK', 1102.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('AMS', 'DEL', 6375.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('PEK', 'DEL', 3815.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('DXB', 'DEL', 2187.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('JFK', 'DEL', 1177.
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('PHL', 'JFK', 150.8
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('JFK', 'PHL', 150.8
INSERT INTO "EDGES" ("SOURCE", "TARGET", "DIST_KM", "PASSENGER_CAPACITY") VALUES ('PEK', 'TNA', 361.9

```

Company Graph Example

Use the following SQL statements to create the Company graph example.

Create Vertex and Edge Tables (Company)

```

DROP SCHEMA COMPANY_SCHEMA CASCADE;
CREATE SCHEMA COMPANY_SCHEMA;
SET SCHEMA COMPANY_SCHEMA;

CREATE TABLE EMPLOYEES(EID NVARCHAR(10) PRIMARY KEY, FIRST_NAME NVARCHAR(100), LAST_NAME NVARCHAR(
INSERT INTO EMPLOYEES VALUES ('I123456', 'John', 'May');
INSERT INTO EMPLOYEES VALUES ('I123457', 'Paul', 'Evans');
INSERT INTO EMPLOYEES VALUES ('D123456', 'Maria', 'Peters');

```

```

CREATE TABLE PROJECTS(PID INTEGER PRIMARY KEY, NAME NVARCHAR(50), BUDGET BIGINT);
INSERT INTO PROJECTS VALUES (1, 'HANA', 500000);
INSERT INTO PROJECTS VALUES (2, 'DWC', 450000);

CREATE TABLE PRODUCTS(PID INTEGER PRIMARY KEY, NAME NVARCHAR(50), PRICE INTEGER);
INSERT INTO PRODUCTS VALUES (100, 'SAP HANA Platform', 300);
INSERT INTO PRODUCTS VALUES (101, 'SAP HANA Cloud', 400);
INSERT INTO PRODUCTS VALUES (200, 'SAP Data Warehouse Cloud', 500);

CREATE TABLE CUSTOMERS(CID BIGINT PRIMARY KEY, COMPANY_NAME NVARCHAR(100), ADDRESS NVARCHAR(200));
INSERT INTO CUSTOMERS VALUES (1, 'XYZ Corp', 'Address XYZ Corp');
INSERT INTO CUSTOMERS VALUES (2, 'ABC Inc', 'Address ABC Inc');

CREATE TABLE PROJECT_ASSIGNMENTS(
    EMPLOYEE_ID NVARCHAR(10) REFERENCES EMPLOYEES(EID),
    PROJECT_ID INTEGER REFERENCES PROJECTS(PID),
    HOURS SMALLINT,
    PRIMARY KEY (EMPLOYEE_ID, PROJECT_ID));
INSERT INTO PROJECT_ASSIGNMENTS VALUES ('I123456', 1, 300);
INSERT INTO PROJECT_ASSIGNMENTS VALUES ('I123457', 1, 400);
INSERT INTO PROJECT_ASSIGNMENTS VALUES ('I123456', 2, 100);
INSERT INTO PROJECT_ASSIGNMENTS VALUES ('D123456', 2, 400);

CREATE TABLE PROJECT_OUTCOMES(
    PROJECT_ID INTEGER REFERENCES PROJECTS(PID),
    PRODUCT_ID INTEGER REFERENCES PRODUCTS(PID),
    PRIMARY KEY (PROJECT_ID, PRODUCT_ID));
INSERT INTO PROJECT_OUTCOMES VALUES (1, 100);
INSERT INTO PROJECT_OUTCOMES VALUES (1, 101);
INSERT INTO PROJECT_OUTCOMES VALUES (2, 200);

CREATE TABLE SALES(
    PRODUCT_ID INTEGER REFERENCES PRODUCTS(PID),
    CUSTOMER BIGINT REFERENCES CUSTOMERS(CID),
    UNITS INTEGER,
    PRIMARY KEY (PRODUCT_ID, CUSTOMER));
INSERT INTO SALES VALUES (100, 1, 3);
INSERT INTO SALES VALUES (101, 1, 2);
INSERT INTO SALES VALUES (200, 2, 7);

```

Create Graph Workspace (Company)

```

CREATE GRAPH WORKSPACE COMPANY_GRAPH
    VERTEX TABLE EMPLOYEES
        KEY EID
        LABEL 'Employee'
    VERTEX TABLE PROJECTS
        LABEL 'Project'
        KEY PID
    EDGE TABLE PROJECT_ASSIGNMENTS
        SOURCE EMPLOYEE_ID REFERENCES EMPLOYEES
        TARGET PROJECT_ID REFERENCES PROJECTS
        KEY EMPLOYEE_ID, PROJECT_ID
        LABEL 'Assignments'
    EDGE TABLE PROJECT_OUTCOMES
        KEY PROJECT_ID, PRODUCT_ID
        SOURCE PROJECT_ID REFERENCES PROJECTS
        TARGET PRODUCT_ID REFERENCES PRODUCTS
    VERTEX TABLE PRODUCTS
        KEY PID
        LABEL 'Product'
    VERTEX TABLE CUSTOMERS
        KEY CID
    EDGE TABLE SALES
        KEY PRODUCT_ID, CUSTOMER
        LABEL 'Sales'
        SOURCE PRODUCT_ID REFERENCES PRODUCTS
        TARGET CUSTOMER REFERENCES CUSTOMERS;

```

Employees Graph Example

Use the following SQL statements to create the Employees graph example.

Create the collection containing the source data

```
DROP SCHEMA A_SCHEMA CASCADE;
CREATE SCHEMA A_SCHEMA;
SET SCHEMA A_SCHEMA;
CREATE COLLECTION EMPLOYEES;
```

Insert the vertices into the collection

```
INSERT INTO EMPLOYEES VALUES({'id': 0, "type": "vertex", "name": "Mary", "role": "senior manager"});
INSERT INTO EMPLOYEES VALUES({'id': 1, "type": "vertex", "name": "John", "role": "manager"});
INSERT INTO EMPLOYEES VALUES({'id': 2, "type": "vertex", "name": "Sara", "role": "architect"});
INSERT INTO EMPLOYEES VALUES({'id': 3, "type": "vertex", "name": "Paul", "role": "senior developer"});
INSERT INTO EMPLOYEES VALUES({'id': 4, "type": "vertex", "name": "Alex", "role": "developer"});
```

Insert the edges into the collection

```
INSERT INTO EMPLOYEES VALUES({'id': 5, "type": "edge", "src": 1, "tgt": 0});
INSERT INTO EMPLOYEES VALUES({'id': 6, "type": "edge", "src": 2, "tgt": 1});
INSERT INTO EMPLOYEES VALUES({'id': 7, "type": "edge", "src": 3, "tgt": 1});
INSERT INTO EMPLOYEES VALUES({'id': 8, "type": "edge", "src": 4, "tgt": 1});
```

Create the graph workspace

```
DROP SCHEMA GRAPH_SCHEMA CASCADE;
CREATE SCHEMA GRAPH_SCHEMA;
CREATE OR REPLACE GRAPH WORKSPACE GRAPH_SCHEMA.EMPLOYEES_GRAPH COLLECTION A_SCHEMA.EMPLOYEES
  TYPE "type"
  VERTEX
    KEY "id"
  EDGE
    KEY "id"
    SOURCE "src"
    TARGET "tgt";
```

The SAP HANA system will automatically create a corresponding hash and an adjacency collection indexes on the fields defining the graph topology before creating the graph workspace. The names of the system generated hash and adjacency indexes will have the prefix **SYS_HASH_INDEX** and **SYS_ADJACENCY_INDEX**, respectively. Alternatively, the graph workspace can also be defined on an explicitly created adjacency index as shown below.

```
CREATE HASH INDEX EMPLOYEES_HASH_INDEX ON A_SCHEMA.EMPLOYEES("id");
CREATE ADJACENCY INDEX EMPLOYEES_ADJ_INDEX ON A_SCHEMA.EMPLOYEES(ID "id", SOURCE "src", TARGET "tgt");
CREATE OR REPLACE GRAPH WORKSPACE GRAPH_SCHEMA.EMPLOYEES_GRAPH ADJACENCY INDEX A_SCHEMA.EMPLOYEES_ADJ_INDEX;
```

Appendix B - Notation

The syntactic notation used in this guide is an extended version of BNF ("Backus Naur Form").

In a BNF language definition, each syntactic element of the language (known as a BNF nonterminal symbol) is defined by means of a production rule. This rule defines the element in terms of a formula consisting of the characters, character strings, and syntactic elements that can be used to form an instance of the formula.

The following table explains the symbols used and their meanings:

Symbol	Meaning
<>	A character string enclosed in angle brackets is the name of a syntactic element (BNF nonterminal) of the GraphScript language.
::=	The definition operator is used in a production rule to separate the element defined by the rule from its definition. The element being defined appears to the left of the operator and the formula that defines the element appears to the right.
[]	Square brackets indicate optional elements in a formula. The portion of the formula within the brackets can be explicitly specified or can be omitted.
{ }	Braces group elements in a formula. Curly braces indicate that the expression can be repeated zero or more times.
	The alternative operator. The vertical bar indicates that the portion of the formula following the bar is an alternative to the portion preceding the bar. If the vertical bar appears at a position where it isn't enclosed in braces or square brackets, it specifies a complete alternative for the element defined by the production rule. If the vertical bar appears in a portion of a formula enclosed in braces or square brackets, it specifies alternatives for the contents of the innermost pair of these braces or brackets.
!!	Introduces normal English text. This is used when the definition of a syntactic element isn't expressed in BNF.
-	The minus operator. The horizontal bar excludes the result of the formula following the bar from valid rules that are defined in front of the bar. The scope of this operator is equivalent to the scope of the alternative operator.

Spaces are used to separate syntactic elements. Multiple spaces and line breaks are treated as a single space. Apart from those symbols with special functions (see previous table), other characters and character strings in a formula stand for themselves. In addition, if the symbols to the right of the definition operator in a production consist entirely of BNF symbols, then these symbols stand for themselves and don't take on their special meaning.

Pairs of braces and square brackets can be nested to any depth, and the alternative operator can appear at any depth within such a nest.

A character string, which forms an instance of any syntactic element, can be generated from the BNF definition of that syntactic element by performing the following steps:

- Select any one option from those defined in the right-hand side of a production rule for the element and replace the element with this option.
- Replace each ellipsis, and the object it applies to, with one or more instances of that object.
- For every portion of the string enclosed in square brackets, either delete the brackets and their contents or change the brackets to braces.
- For every portion of the string enclosed in braces, apply steps 1 through 5 to the substring between the braces, then remove the braces.

- Apply steps 1 through 5 to any BNF nonterminal symbol that remains in the string.

The expansion or production is complete when no further nonterminal symbols remain in the character string.

By applying the previous steps 1 through 5 to NT, and always selecting the leftmost BNF nonterminal in step 5, the left normal form derivation of a character string CS in the source language character set from a BNF nonterminal NT is obtained.

Appendix C - GraphScript Cheat Sheet

GraphScript is the imperative extension language of SAP HANA Cloud property graph engine. It's designed to ease the development of application-specific graph algorithms and to integrate them into the overall data processing workflow.

General Structure

Graphscript Procedures

```
CREATE PROCEDURE GRAPH_PROC (OUT out_val DOUBLE)
LANGUAGE GRAPH READS SQL DATA AS
BEGIN
  -- GraphScript coding goes here
END
```

Anonymous Blocks

```
DO (IN in_val BIGINT => ?, OUT out_val BIGINT => ?) LANGUAGE GRAPH
BEGIN
  -- GraphScript coding goes here
END
```

Table User Defined Functions (TUDF)

```
CREATE OR REPLACE FUNCTION GRAPH_FUNC (in_val NVARCHAR(10))
  RETURNS TABLE (col1 NVARCHAR(10), col2 INTEGER)
LANGUAGE GRAPH AS
BEGIN
  -- GraphScript coding goes here
  RETURN SELECT :v.VERTEX_ID, :v.VERTEX_DEPTH FOREACH v IN VERTICES(:g);
END
```

Scalar Data Types

Integer (32-bit, signed, nullable)

```
INTEGER i = 23;
i = TO_INTEGER(45L);      -- cast BigInt to Integer
i = TO_INTEGER(23.5);     -- cast Double to Integer
```

BigInt (64-bit, signed, nullable)

```
BIGINT b = 23L;
b = TO_BIGINT(45);        -- cast Integer to BigInt
b = TO_BIGINT(5.2, 6L);   -- cast Double to BigInt. If the cast fails or the first argument is NULL,
                           then the second argument is returned. Hence, b = 5L.
```

Double (64-bit, signed, nullable)

```
DOUBLE d = 23.5;
d = TO_DOUBLE(12L);      -- cast BigInt to Double
d = TO_DOUBLE('23.5');   -- cast Nvarchar to Double
INTEGER i = NULL;
d = TO_DOUBLE(:i, 3.5);   -- cast Integer to Double. If the cast fails or the first argument is NULL
                           then the second argument is returned. Hence, d = 3.5.
```

String Type

```
NVARCHAR n = 'Test';
n = TO_NVARCHAR(3, 'abc'); -- cast Integer to Nvarchar. If the first argument is NULL, then the
                           second argument is returned. Hence, n = '3'.
```

All strings are variable-length and support unicode characters.

Boolean Type

```
BOOLEAN b = TRUE;
b = TO_BOOLEAN(0); -- cast Integer to Boolean
b = TO_BOOLEAN('falsee', false) -- cast Nvarchar to Boolean. If the cast fails or the first argument
                                is NULL, then the second argument is returned. Hence, b = false.
```

Defines a boolean variable with initial value TRUE. The default value is FALSE.

Timestamp

```
TIMESTAMP t = TO_TIMESTAMP(
    '2010-03-17 12:34:57.1234567');
```

Defines a timestamp variable from a string. The format is yyyy-mm-dd HH:MM:SS.NNNNNNN.

Date

```
Date d = TO_DATE('2020-02-02');
```

Defines a date variable from a string. The format is yyyy-mm-dd.

Graph Types

```
GRAPH g1 = GRAPH("my_graph_workspace");
GRAPH g2 = GRAPH("my_schema", "workspace");
```

Defines a graph based on a workspace catalog object.

```
GRAPH g3 = GRAPH(:edgeTab, "srcCol", "tgtCol",
    "edgeIdCol", :vtxTab, "vtxIdCol");
```

Defines a graph based on edge and vertex tables.

Temporary Attributes

```
GRAPH g3 = GRAPH("my_schema", "workspace")
  WITH TEMPORARY ATTRIBUTES(EDGE INTEGER attr = 2 * 1,
                           VERTEX DOUBLE otherattr DENSE);
```

Defines a graph with additional temporary attributes.

Vertex and Edge Types

```
VERTEX v1(:g, 1);
EDGE e1(:g, 2, 'keyVal2');    -- edge constructor with composite key
VERTEX v2(:path, 'label', 3); -- vertex constructor with label
```

Constructs vertex and edge references from a graph or a weighted path, and a (composite) key. For graphs with multiple vertex or edge tables, a vertex or edge label is necessary in the constructor.

Variables

Variable names are case-insensitive and unique within a block scope and all its subscopes.

```
INTEGER a = 23;
INTEGER b = 0;
b = :a;
```

In contrast to write access, read access to a local variable has to be prefixed by a colon.

FOREACH Loop

Iterate over all vertices in graph g:

```
FOREACH v IN VERTICES(:g) { ... }
```

Iterate over all edges in graph g:

```
FOREACH e IN EDGES(:g) { ... }
```

WHILE Loop

```
INTEGER i = 0;
WHILE (:i < 10) {
  i = :i + 1;
}
```

Conditional Statements

```
IF (2 < 3) {
  IF (TRUE) {
  } ELSE {
  }
}
```

Comments

```
-- This is a one-line comment
/* This is
   a multi-line comment */
```

Attribute Access

```
INTEGER a = :v.attr;
v.attr = 23;
```

Relational Expression

```
2 < 3
3 == 4
:a.weight >= 5
:v.attr != 'blue'
```

Arithmetic Expression

```
2 + 3
3 - 1
2 * :v.weight
2 / 2
```

Logical Expressions

```
(3 == 4) AND (4 == 3)
TRUE OR (:a.weight > 3)
NOT (3 == 2)
(:a.attr IS NULL) AND (:b.attr IS NOT NULL)
```

Membership Expression

```
:i IN :mset_int
:str NOT IN :mset_nvarchar
```

Existence Expression

```

VERTEX_EXISTS(:g, 'keyVal')
EDGE_EXISTS(:g, 'label', 3)

```

Filter Expression

```
v IN VERTICES(:g) WHERE :v.attr > 23
```

Projection Expression

```

tab = SELECT :v.id FOREACH v IN VERTICES(:g);
RETURN SELECT :v.id FOREACH v IN VERTICES(:g);

```

Multisets

Constructors

```
MULTISET<VERTEX> m1(:g);
```

Constructs an empty vertex multiset.

```
MULTISET<VERTEX> m2 = { :v1, :v2 };
```

Constructs vertex multiset with vertices :v1 and :v2.

```

MULTISET<INTEGER> m3;           -- empty multiset
MULTISET<INTEGER> m4 = {};      -- empty multiset
MULTISET<INTEGER> m5(:seq_int); -- creates a multiset from the given sequence

```

Constructs a collection of scalars.

```
MULTISET<MULTISET<BIGINT>> m6 = {{2L, 2L}, {1L, 3L}, {3L, 1L}};
```

Constructs a nested multiset of scalars.

Operations

```

DISTINCT(:m1)
:m1 INTERSECT [ALL] :m2
:m1 UNION [ALL] :m2
:m1 EXCEPT [ALL] :m2

```

Removing ALL is equivalent to a subsequent DISTINCT.

```
BIGINT sz = COUNT(:m1);
```

Calculates the size of the Multiset.

```

BOOLEAN b1 = :m1 == :m2;
BOOLEAN b2 = :m1 != :m3;

```

Compares two multisets for equality or inequality.

Sequences

Constructors

```

SEQUENCE<VERTEX> s1(:g);

```

Constructs an empty vertex sequence.

```

SEQUENCE<VERTEX> s2 = [:v1, :v2];

```

Constructs vertex sequence with vertices :v1 and :v2.

```

SEQUENCE<INTEGER> si(:tab."colInt");

```

Constructs scalar sequence from table column.

```

SEQUENCE<INTEGER> s3;           -- empty sequence
SEQUENCE<SEQUENCE<INTEGER>> s4 = []; -- empty nested sequence

```

Constructs a collection of scalars.

```

SEQUENCE<SEQUENCE<EDGE>> s5 = [[:e1], [:e2, :e3]];

```

Constructs a nested sequence of edges.

Operations

```

VERTEX v1   = :s1[1L];    -- read
s5[2L, 1L] = :e1;         -- write
BIGINT sz   = COUNT(:s1); -- size
BOOLEAN b1  = :s1 == :s2; -- comparison

```

Index starts at 1.

```

SEQUENCE<VERTEX> s6 = :s1 || :s2;
SEQUENCE<SEQUENCE<EDGE>> s7 = :s5 || [:e2, :e1];

```

Concatenates two sequences or a sequence with an element.

```

SEQUENCE<INT> s8 = [2,3,1] ORDER ASC;

```

Sorts the elements of a sequence.

Map

Constructors

```
MAP<BIGINT, DOUBLE> map1(10L);
```

Constructs an empty map with memory allocated for 10 entries.

```
MAP<VERTEX, BIGINT> map2(:g, 100L, 2L);
```

Constructs a map with 100 entries and default value 2L.

Operations

```
BIGINT r1 = :map2[:v1];    -- read
map1[3L] = 3.8;           -- write
BIGINT sz = COUNT(:map1); -- size
```

Tables

Operations

```
INTEGER i = :tab."colInt"[1L]; -- read
tab."colInt"[1L] = 23; -- write
SEQUENCE<INTEGER> s3(:tab."colInt"); -- sequence from column
BIGINT sz = COUNT(:tab); -- row count
MAP<INTEGER, INTEGER> map1(:inTable."colInt1", :inTable."colInt2"); -- map from two table columns
```

Graph Algorithms

IS_REACHABLE

```
BOOLEAN b = IS_REACHABLE(:g, :v1, :v2);
```

Determines the reachability between vertices :v1 and :v2.

VERTICES / EDGES

```
MULTISET<VERTEX> vs = VERTICES(:g);
SEQUENCE<VERTEX> sv = VERTICES(:weighted_path);
MULTISET<EDGE> me = EDGES(:g);
MULTISET<EDGE> es = EDGES(:g, :vsSrc, :vsTgt);
MULTISET<EDGE> mi = IN_EDGES(:v);
MULTISET<EDGE> mo = OUT_EDGES(:v);
```

SOURCE / TARGET

```
VERTEX v = SOURCE(:e);
VERTEX v = TARGET(:e);
```

NEIGHBORS

```
MULTISET<VERTEX> nb1 = NEIGHBORS(:g, :v, -1, 2);
MULTISET<VERTEX> nb2 = NEIGHBORS(:g, :v, 0, 2, 'OUTGOING');
MULTISET<VERTEX> nb3 = NEIGHBORS(:g, :v, 1, 15, 'INCOMING');
MULTISET<VERTEX> nb4 = NEIGHBORS(:g, :v, 2, 2, 'ANY');
```

Degree Functions

```
BIGINT numIncomingEdges = IN_DEGREE(:v);
BIGINT numOutgoingEdges = OUT_DEGREE(:v);
BIGINT numEdgesAnyDirection = DEGREE(:v);
```

LABEL

```
NVARCHAR n = LABEL(:v);
```

Returns the label of a given vertex or edge.

RANDOM_ELEMENT

```
INTEGER random_int = RANDOM_ELEMENT(:ms);
```

Returns a randomly chosen element from a given multiset or sequence.

IS_EMPTY

```
BOOLEAN b = IS_EMPTY(:seq);
```

Returns true if the given multiset, sequence, map or weighted path has no elements.

SUBGRAPH

```
MULTISET<VERTEX> mv= VERTICES(:g);
GRAPH g1 = SUBGRAPH(:g, :mv);
```

Induces a subgraph from a set of vertices.

INVERSEGRAPH

```
GRAPH g1 = INVERSEGRAPH(:g);
```

Retrieves an inverse graph with all edges flipped.

SHORTEST_PATH

```
WEIGHTEDPATH<BIGINT> p1
  = SHORTEST_PATH(:g, :v1, :v2);
WEIGHTEDPATH<INTEGER> p2
  = SHORTEST_PATH(:g, :v1, :v2,
    (EDGE e) => INTEGER { return :e.attr; }));
```

Determines the shortest path between two vertices based either on hop distance or on a custom weight function.

```
WEIGHTEDPATH<BIGINT> p3
  = SHORTEST_PATH(:g, :v1, :v2, 'ANY');
```

Determines the shortest path between two vertices irrespective of the connecting edges' direction.

```
WEIGHTEDPATH<INTEGER> p4
  = SHORTEST_PATH(:g, :v1, :v2, (EDGE e, INTEGER w, BIGINT h) =>
    INTEGER{ IF(:w + :e.price < 500 AND :h < 2L) {return :e.price;}
    ELSE {END TRAVERSE;}});
```

Determines the shortest path between two vertices using a custom weight function with bounds on current weight distance and hop distance. In this example, 'w' is the weight and 'h' is the number of hops until the previous edge.

K_SHORTEST_PATHS

```
SEQUENCE<WEIGHTEDPATH<BIGINT>> paths
  = K_SHORTEST_PATHS(:g, :v1, :v2, :k);
SEQUENCE<WEIGHTEDPATH<INTEGER>> paths2
  = K_SHORTEST_PATHS(:g, :v1, :v2, :k,
    (EDGE e) => INTEGER { return :e.attr; }));
```

Determines the top k shortest paths between two vertices either based on hop distance or on a custom weight function.

SHORTEST_PATHS_ONE_TO_ALL

```
GRAPH g2 = SHORTEST_PATHS_ONE_TO_ALL (
  :g, :startVertex, "WEIGHT",
  (EDGE e) => DOUBLE { RETURN :e.len; },
  'INCOMING'
);
```

Determines the shortest paths between a start vertex and all other reachable vertices within a given graph traversing the connecting edges in reverse direction.

MAX_FLOW

```
INTEGER flow_value = 0;
SEQUENCE<WEIGHTEDPATH<INTEGER>> flow_paths(:g);

CLOSURE capacityFct(:g, (EDGE e) => INTEGER {
  IF (:e."DIST_KM" < 7000.0) {
    RETURN :e."PASSENGER_CAPACITY";
  } ELSE {
    END TRAVERSE;
  }
});

MAP<EDGE, BIGINT>> flow_map = MAX_FLOW(:g, :v_source, :v_sink, :capacityFct, :flow_value, :flow_pa
```

Determines the maximum flow from a start vertex to a sink vertex.

STRONGLY_CONNECTED_COMPONENTS

```
SEQUENCE<SEQUENCE<VERTEX>> ssv = STRONGLY_CONNECTED_COMPONENTS(:g);
```

Computes the strongly connected components in a graph.

Weighted Path Functions

WEIGHT

```
BIGINT costs = WEIGHT(:weighted_path_bigint);
```

Gets the weight of the given path.

LENGTH

```
BIGINT len = LENGTH(:weighted_path);
```

Gets the length (number of edges) for a given weighted path.

Datetime Functions

Signature	Return	Description
YEAR(TIMESTAMP)	INTEGER	extract year
MONTH(TIMESTAMP)	INTEGER	extract month
DAYOFMONTH(TIMESTAMP)	INTEGER	extract day
HOUR(TIMESTAMP)	INTEGER	extract hour
MINUTE(TIMESTAMP)	INTEGER	extract minute
SECOND(TIMESTAMP)	INTEGER	extract second

Signature	Return	Description
NANOSECOND (TIMESTAMP)	INTEGER	extract nanosecond
CURRENT_TIMESTAMP ()	TIMESTAMP	extract current timestamp
CURRENT_UTCTIMESTAMP ()	TIMESTAMP	extract UTC timestamp
LAST_DAY (TIMESTAMP)	TIMESTAMP	extract last day of the month
ADD_NANO100 (TIMESTAMP, BIGINT)	TIMESTAMP	add or subtract specified 0.1 microsecond units
ADD_SECONDS (TIMESTAMP, INTEGER)	TIMESTAMP	add or subtract seconds
ADD_MINUTES (TIMESTAMP, INTEGER)	TIMESTAMP	add or subtract minutes
ADD_HOURS (TIMESTAMP, INTEGER)	TIMESTAMP	add or subtract hours
ADD_DAYS (TIMESTAMP, INTEGER)	TIMESTAMP	add or subtract days
ADD_MONTHS (TIMESTAMP, INTEGER)	TIMESTAMP	add or subtract months
ADD_YEARS (TIMESTAMP, INTEGER)	TIMESTAMP	add or subtract years
NANO100_BETWEEN (TIMESTAMP, TIMESTAMP)	BIGINT	difference in 0.1 microsecond units
SECONDS_BETWEEN (TIMESTAMP, TIMESTAMP)	BIGINT	difference in seconds
DAYS_BETWEEN (TIMESTAMP, TIMESTAMP)	INTEGER	difference in days
MONTHS_BETWEEN (TIMESTAMP, TIMESTAMP)	INTEGER	difference in months
YEARS_BETWEEN (TIMESTAMP, TIMESTAMP)	INTEGER	difference in years

Traversal Hooks

```

TRAVERSE BFS :g FROM VERTEX(:g, 1)
  ON VISIT VERTEX (VERTEX v) {
    INTEGER attrVal = :v.attr;
  };

```

Breadth-first traversal with vertex hook.

```

TRAVERSE BFS :g FROM VERTEX(:g, 1)
  ON VISIT VERTEX (VERTEX v, BIGINT lvl) {
    v.attr = :lvl;
  };

```

Breadth-first traversal with vertex hook and traversal depth.

```

TRAVERSE DFS :g FROM VERTEX(:g, 1)
  ON VISIT EDGE (EDGE e) {
    INTEGER attrVal = :e.attr;
  };

```

Depth-first traversal with edge hook.

```

TRAVERSE BFS :g FROM VERTEX(:g, 1)
  ON VISIT EDGE (EDGE e, BIGINT lvl) {

```

```

    e.attr = :lvl;
};

```

Breadth-first traversal with edge hook and traversal depth.

```

TRAVERSE BFS :g FROM VERTEX(:g, 1)
ON VISIT VERTEX (VERTEX v, BIGINT lvl) {
    v.attr = :lvl;
}
ON VISIT EDGE (EDGE e) {
    INTEGER attrVal = :e.attr;
};

```

Breadth-first traversal with vertex and edge hooks and traversal depth.

```

TRAVERSE DFS :g FROM VERTEX(:g, 1)
ON VISIT VERTEX (VERTEX v) {
    IF (:v.attrVal == 42) {
        END TRAVERSE ALL;
    }
};

```

Depth-first traversal with END TRAVERSE ALL statement in vertex hook.

```

TRAVERSE BFS :g FROM VERTEX(:g, 1)
ON VISIT EDGE (EDGE e) {
    IF (:e.attrVal != 'open') {
        END TRAVERSE;
    }
};

```

Breadth-first traversal with END TRAVERSE statement in edge hook.

Appendix D - openCypher Cheat Sheet

openCypher is a declarative graph query language for graph pattern matching. An openCypher query searches for subgraphs that match the pattern given in MATCH clauses and returns a result table as specified in the RETURN clause.

General Structure

```
SELECT * FROM OPENCYPHER_TABLE( GRAPH WORKSPACE "SCHEMA"."GRAPH" QUERY
  \ -- openCypher query goes here \
);
```

An example:

```
SELECT * FROM OPENCYPHER_TABLE( GRAPH WORKSPACE "FLIGHTS"."GRAPH" QUERY
  ' MATCH (a)-[e]-(b)
    WHERE a.NAME = ''John F Kennedy International''
    RETURN b.NAME AS Name
    ORDER BY b.NAME '
);
```

MATCH clause

Consists of topological constraints that define the topology (vertices, edges, and paths) of the matching subgraphs, and an optional filter condition, which allows the user to add additional (non-topological) constraints.

Topological Constraints

MATCH (a)	-- any vertex
MATCH (a:label)	-- vertex with label
MATCH (a)-[e]-(b)	-- any edge between two vertices
MATCH (a)-[e]->(b)	-- directed edge
MATCH (a)-[e]-(b:vertex_label)	-- edge with label for one vertex
MATCH (a:vertex_label)-[e:edge_label]->(b)	-- directed edge with label for edge and one vertex
MATCH p = (a)-[*1..2]->(b)	-- variable-length path
MATCH p = (a)-[:edge_label*3..5]->(b)	-- variable-length path with edge label

```
MATCH (a)-[e1]->(b)
MATCH (b)-[e2]->(c)
```

Multiple match clauses form different patterns. The final result is a combination of results of each individual pattern.

```
MATCH (a)-[e1]->(b), (b)-[e2]->(c)
```

Comma separated match clauses are part of the same pattern. Each relationship appears only once in the result.

Non-Topological Constraints

```
WHERE a.NAME = ''John F Kennedy International Airport''
  AND (e.EDGE_KEY > 4 OR b.NAME = ''Philadelphia International Airport'')
```

Adds constraints to the patterns using comparisons (=, <>, >, <, >=, <=), IS NULL, and other boolean built-in functions, combined by the logical connectives AND, OR, and NOT.

RETURN Clause

DISTINCT

```
RETURN DISTINCT a.NAME AS Name
```

Eliminates duplicate rows from the result.

ORDER BY

```
RETURN a.NAME AS name
ORDER BY a.NAME DESC
```

Sorts the resulting rows in ascending or descending order.

LIMIT

```
RETURN a.NAME AS name
LIMIT 5
```

Truncates the resulting table to the given number of rows.

SKIP

```
RETURN a.NAME AS name
SKIP 2
```

Excludes the given number of rows from the top of the result.

Aggregate functions

```
RETURN COUNT(a.NAME) AS numCount    -- returns the number of rows
RETURN MAX(a.ID) AS numMax           -- returns the max value
RETURN MIN(a.ID) AS numMin           -- returns the min value
RETURN SUM(a.ID) AS numSum           -- returns the sum
```

List comprehension

```
RETURN [e IN RELATIONSHIPS(p) | e.TARGET] AS result
```

Constructs a new list from the given path function in JSON format, returning a list of edges and the given expression that uses the given iterator variable.

```
RETURN { from: e.SOURCE, to: e.TARGET } AS result
```

String representation of the result in JSON format.

OpenCypher functions

TOBOOLEAN

```
TOBOOLEAN(value)
```

Converts the input to a boolean.

TOFLOAT

```
TOFLOAT(value)
```

Converts the input to a floating point number.

TOINTEGER

```
TOINTEGER(value)
```

Converts the input to an integer.

TOSTRING

```
TOSTRING(value)
```

Converts the input to a string.

DATETIME

```
DATETIME(value)
```

Converts the input to a datetime.

DATE

```
DATE(value)
```

Converts the input to a date.

RELATIONSHIPS

RELATIONSHIPS(p)

Takes a path variable and returns a collection of all edges in the path.

ALL

WHERE ALL(e IN RELATIONSHIPS(path) WHERE e.DIST_KM > 8000.0)

Returns true if the given condition is true for all elements in the given collection.

LABEL (v)

LABEL(v)

Takes a vertex or an edge variable and returns its label.

SYS.TEXT_CONTAINS

```
WHERE SYS.TEXT_CONTAINS(a.name, 'Dubbai', 'FUZZY(0.8)') -- fuzzy search
WHERE SYS.TEXT_CONTAINS(e.color, 'b*', 'EXACT')         -- exact search
```

Allows advanced text search capabilities.